Омский государственный технический университет

О. Б. Малков, М. П. Маркова, М. В. Девятерикова

РАБОТА С СИСТЕМОЙ УПРАВЛЕНИЯ БАЗАМИ ДАННЫХ MongoDB

Учебное пособие

Учебное текстовое локальное электронное издание

Рекомендовано редакционно-издательским советом
Омского государственного технического университета

Омск
Издательство ОмГТУ
2025

Сведения об издании: 1, 2, 3

© ОмГТУ, 2025

ISBN 978-5-8149-4044-5

УДК 004.65(075)
ББК 32.973.26-018.2я73
М19

Рецензенты:

В. А. Мещеряков, д-р техн. наук,
профессор кафедры «Цифровые технологии» СибАДИ;

А. М. Мызников, канд. физ.-мат. наук,
руководитель проекта ООО «Магма-Компьютер»

Малков, О. Б. Работа с системой управления базами данных MongoDB : учеб. пособие / О. Б. Малков, М. П. Маркова, М. В. Девятерикова ; Ом. гос. техн. ун-т. – Омск : Изд-во ОмГТУ, 2025. – 1 CD-ROM (2,30 Мб). – Минимальные систем. требования: процессор с частотой 800 МГц и выше ; 128 Мб RAM и более ; свободное место на жестком диске 300 Мб и более ; Linux / Windows XP и выше ; MacOS X 10.4 и выше ; CD/DVD-ROM-дисковод ; ПО для просмотра pdf-файлов. – Загл. с титул. экрана. – ISBN 978-5-8149-4044-5.

Описаны основы работы с системой управления базами данных MongoDB. Рассмотрен процесс установки системы управления базами данных и создания документной базы данных, основные операции с базой данных, коллекциями и документами, взаимодействие с приложениями.

Пособие предназначено для обучения студентов по образовательным программам высшего образования для топ-специалистов в сфере информационных технологий (ТОП-ИТ), а также по направлениям 09.03.01 «Информатика и вычислительная техника», 09.03.02 «Информационные системы и технологии», 09.03.03 «Прикладная информатика» и 09.03.04 «Программная инженерия».

© ОмГТУ, 2025

Учебное текстовое локальное электронное издание

Малков Олег Брониславович
Маркова Марина Петровна
Девятерикова Марина Владимировна

РАБОТА С СИСТЕМОЙ УПРАВЛЕНИЯ БАЗАМИ ДАННЫХ MongoDB

Учебное пособие

Издание поставляется на одном CD-ROM-диске

Воспроизведение издания автоматическое –
без установки на жесткий диск компьютера

Для корректной работы с изданием на компьютере должны быть установлены
CD/DVD-ROM-дисковод и программное обеспечение
для просмотра pdf-файлов

Редактор *А. Ю. Леонтьева*
Компьютерная верстка *О. Г. Белименко*

Для дизайна первичной упаковки и этикетки носителя издания
использованы материалы из открытых интернет-источников

Иллюстрации для издания предоставлены авторами

Сводный темплан 2025 г.
Подписано к использованию 26.11.2025.
Объем 2,30 Мб. Тираж 10 эл. опт. дисков.

Издательство ОмГТУ. 644050, г. Омск, пр. Мира, 11; т. 8(3812)23-02-12.
Эл. почта: publisher@omgtu.ru

ОГЛАВЛЕНИЕ

ВВЕДЕНИЕ 5

Лабораторная работа № 1. УСТАНОВКА И НАСТРОЙКА MONGODB 7

Лабораторная работа № 2. БАЗЫ ДАННЫХ, КОЛЛЕКЦИИ И ДОКУМЕНТЫ 15

Лабораторная работа № 3. БАЗОВЫЕ ОПЕРАЦИИ CRUD В MONGODB 37

Лабораторная работа № 4. ОПРЕДЕЛЕНИЕ ЛОГИЧЕСКИХ СВЯЗЕЙ МЕЖДУ КОЛЛЕКЦИЯМИ 47

Лабораторная работа № 5. ПОИСК, СОРТИРОВКА И ОБНОВЛЕНИЕ ДАННЫХ В MONGODB 57

Лабораторная работа № 6. РАБОТА СО ВЛОЖЕННЫМИ СТРУКТУРАМИ И МАССИВАМИ 74

Лабораторная работа № 7. ФРЕЙМВОРК АГРЕГАЦИИ В MONGODB....87

Лабораторная работа № 8. ИСПОЛЬЗОВАНИЕ РЕГУЛЯРНЫХ ВЫРАЖЕНИЙ В MONGODB.....102

Лабораторная работа № 9. РАБОТА С КУРСОРОМ.....112

Лабораторная работа № 10. ИНТЕГРАЦИЯ MONGODB С ПРИЛОЖЕНИЯМИ 127

ЗАКЛЮЧЕНИЕ 147

БИБЛИОГРАФИЧЕСКИЙ СПИСОК.....148

ВВЕДЕНИЕ

> Мы вступаем в новый мир, в котором данные могут оказаться важнее программного обеспечения.
>
> *Тим О'Рейли, основатель издательства O'Reilly, активист движения за свободное программное обеспечение*

Современный мир основан на информации, поэтому решающее значение имеет возможность эффективно хранить большие объемы данных и управлять ими. База данных (БД) NoSQL MongoDB – важный инструмент разработчиков, стремящихся к гибкости, масштабируемости и производительности. От реляционных баз данных ее отличает документоориентированная модель данных, упрощающая обработку сложных структур.

Документоориентированные БД направлены на хранение данных и работу с ними в формате документов. Данные хранятся непосредственно в коллекциях документов, как правило, с применением форматов JSON / XML / BSON. При этом одна запись может включать в себя множество данных любых типов: ограничения отсутствуют. Внутренняя структура одного документа может отличаться от внутренней структуры другого. Кроме того, документы можно вкладывать друг в друга.

Особенности документоориентированных баз данных:

- отсутствие схемы данных. Позволяет добавлять новые данные в записи. При этом не требуется, чтобы все остальные записи имели одинаковую структуру;
- адресация документов с помощью уникального ключа. Обычно это строка, которая генерируется автоматически. По ключу можно извлекать запись или ссылаться на другие документы;
- наличие своего языка запросов.

Такими документоориентированными БД являются MongoDB, CouchDB, Couchbase, MarkLogic, eXist.

MongoDB является документоориентированной системой управления базами данных (СУБД) NoSQL с открытым исходным кодом. Данные в ней хранятся с использованием структур, подобных JSON. Каждый документ может иметь собственную структуру. В MongoDB представлены методы и команды, отличные от SQL, для управления данными и их запроса программным или интерактивным способами. Система известна высокой производительностью, масштабируемостью и быстрыми темпами разработки. Ее особенности:

- индексирование по любому атрибуту, позволяющее ускорить поиск данных и оптимизировать производительность запросов;
- избыточность данных за счет поддержания нескольких копий (репликация), обеспечивающая отказоустойчивость и непрерывную доступность даже при сбоях сервера;
- автоматическое сегментирование, распределяющее данные по нескольким серверам, обеспечивая горизонтальное масштабирование и эффективную обработку больших наборов данных;
- работа с большими данными и приложениями, требующими обновления данных в реальном времени;
- поддержка сложных запросов с различными операторами, позволяющих извлекать, фильтровать данные и манипулировать ими;
- быстрые обновления документов на месте с минимальным перемещением данных, сокращающие затраты на запись;
- использование в приложениях Интернета вещей для хранения и анализа данных датчиков в различных форматах, хорошо согласующихся со структурой документов.

Настоящее пособие знакомит с основами MongoDB, начиная с настройки среды и заканчивая вопросами взаимодействия с приложениями. Его можно рассматривать как первый шаг в изучении данной СУБД. Ответы на более продвинутые вопросы можно найти в специальной литературе [1, 3, 5, 6, 7, 8, 9].

Лабораторная работа № 1
УСТАНОВКА И НАСТРОЙКА MONGODB

Цель работы:

– познакомиться с документоориентированной СУБД NoSQL MongoDB;

– изучить способы установки и подключения к серверу СУБД MongoDB.

Теоретические сведения

Установка MongoDB и подключение к серверу

Установка MongoDB возможна на различных платформах: Windows, Linux, MacOS, Solaris. Для каждой из них доступно несколько дистрибутивов. Имеется два вида серверов – бесплатный *Community Edition* и платный *Enterprise Advanced*. Следует выбрать вариант *Community*.

В настоящей работе использовали версию СУБД 8.0.6, другие версии могут несколько отличаться. Для загрузки всех необходимых файлов следует выбрать нужную операционную систему и подходящий тип пакета.

Для Windows доступна загрузка установщика msi и zip-пакета. Достаточно загрузить zip-архив и распаковать его в нужной папке. Если использовалась более ранняя версия MongoDB, то ее нужно удалить. Процедура установки достаточно проста: следует принять все предлагаемые по умолчанию настройки. В процессе установки создается папка *bin* (рис. 1) в каталоге *C:\Program Files\MongoDB\Server\8.0*.

Имя	Дата изменения	Тип	Размер
InstallCompass.ps	20.02.2024 23:08	Сценарий Windo...	2 КБ
mongod.cfg	16.03.2024 22:32	Исходный файл ...	1 КБ
mongod.exe	21.02.2024 1:36	Приложение	64 702 КБ
mongod.pdb	21.02.2024 1:36	Program Debug D...	1 014 476 КБ
mongos.exe	21.02.2024 1:32	Приложение	39 041 КБ
mongos.pdb	21.02.2024 1:32	Program Debug D...	575 748 КБ

Рис. 1

Эта папка содержит несколько важных приложений:

- *mongod.exe* – сервер баз данных MongoDB, который обрабатывает запросы, управляет форматом данных и выполняет в фоновом режиме операции по управлению БД;
- *mongos.exe* – служба маршрутизации MongoDB, помогающая обрабатывать запросы и определять положение данных в кластере MongoDB.

В зависимости от операционной системы и параметров установки можно использовать следующие клиентские приложения:

- *mongosh* – новая оболочка MongoDB (в ранних версиях в папке *bin* находился консольный клиент *mongo.exe*, но начиная с MongoDB 6.0 устаревшая оболочка *mongo* больше не предлагается);
- *MongoDB Compass* – графический интерфейс администратора, включающий встроенную версию *mongosh*.

Клиент командной строки MongoDB Shell

Наиболее простым клиентом для отправки запросов к серверу является консольная оболочка MongoDB Shell *(mongosh)*. Это самый быстрый способ подключения к MongoDB и работы с ним. Представляет собой современный расширяемый интерфейс командной строки, оснащенный подсветкой синтаксиса, интеллектуальным автозаполнением, контекстной справкой и сообщениями об ошибках.

> ВНИМАНИЕ! Клиент MongoDB Shell *(mongosh)* скачивается как zip-пакет из раздела *Tools* на сайте и устанавливается независимо от сервера MongoDB

Загруженный zip-пакет нужно распаковать в папку *C:\mongosh*. Если в распакованном архиве войти в папку *bin (C:\mongosh\bin)*, то там можно найти консольную утилиту *mongosh* (рис. 2), которая и будет применяться для работы с сервером MongoDB.

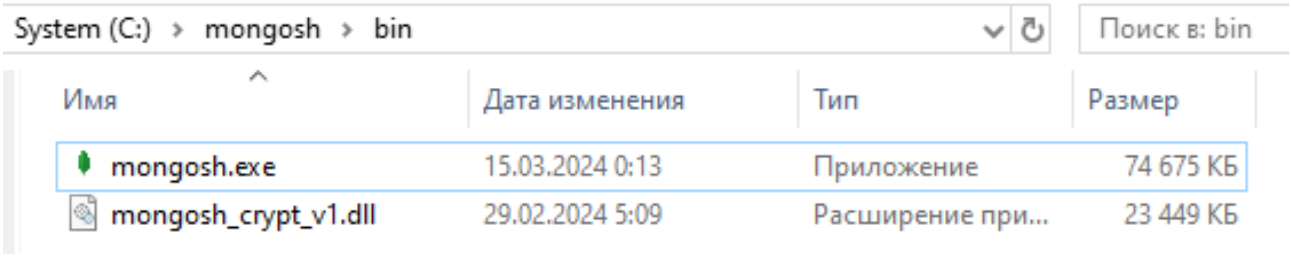


Рис. 2

Для подключения к серверу MongoDB необходимо выполнить следующие шаги (сервер должен быть запущен):

1. Запустите файл *mongosh.exe*, расположенный в папке установки. Программа *mongosh* спросит, какую строку подключения использовать для подключения к серверу MongoDB.

2. Нажмите *Enter*, согласившись на строку подключения по умолчанию. По умолчанию сервер использует порт 27017, а полная строка подключения выглядит следующим образом: *MongoDB://localhost:27017* или *MongoDB://127.0.0.1:27017*.

3. После подключения консоль отобразит служебную информацию и автоматически осуществит подключение к БД *test* (рис. 3).

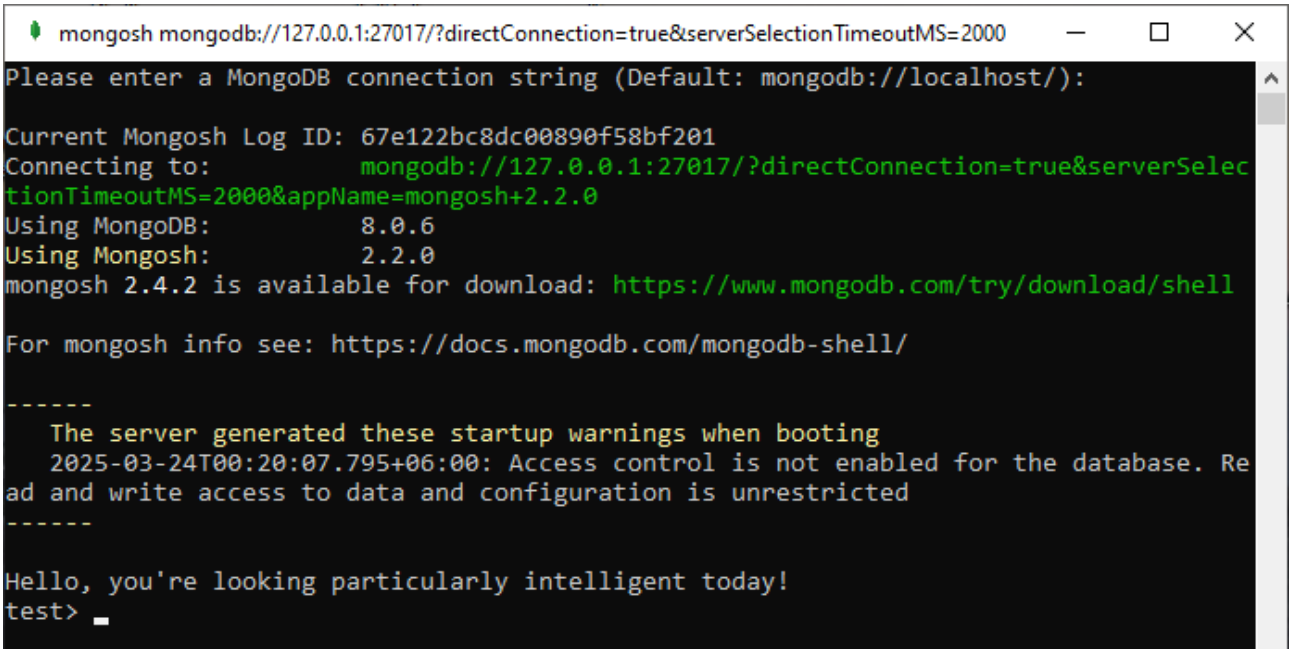


Рис. 3

Ввод в командной строке команды *help* даст справочную информацию о командах клиента командной строки:

- *use* – установить текущую БД;
- *show databases / show dbs* – показать список доступных БД;
- *show collections / show tables* – показать список всех коллекций для текущей БД;
- *show profile* – показать информацию *system.profile*;
- *show users* – показать список всех пользователей для текущей БД;
- *show roles* – показать список всех ролей для текущей БД;
- *show log <type>* – показать журнал для текущего подключения (если тип не задан, то используется *global*);
- *show logs* – показать все журналы;
- *exit / exit()* – выйти из оболочки MongoDB Shell;
- *quit / quit()* – выйти из оболочки MongoDB Shell;
- *Mongo* – создать и протестировать новое подключение;
- *connect* – создать новое подключение и вернуть объект БД: *connect(URI, username [optional], password [optional])*;
- *version* – узнать версию оболочки;
- *load* – загрузить и запустить файл JavaScript в среде оболочки;
- *enableTelemetry* – включить сбор анонимных данных об использовании для улучшения интерфейса mongodb CLI;
- *disableTelemetry* – отключить сбор анонимных данных об использовании для улучшения интерфейса mongodb CLI;
- *passwordPrompt* – запросить у пользователя пароль;
- *sleep* – перейти в спящий режим на указанное число миллисекунд;
- *print()* – вывести содержимое объекта на печать;
- *printjson* – задать псевдоним для *print()*;
- *convertShardKeyToHashed* – возвратить хешированное значение для ввода (использует ту же функцию хеширования, что и хешированный ин-

декс, и его можно применить, чтобы увидеть, каким будет хешированное значение для ключа);

- *cls* – очистить экран (как *console.clear()*);
- *isInteractive* – возвратить логическое значение, указывающее, работает ли текущий сеанс в интерактивном режиме или в режиме сценария.

Для примера рассмотрим несколько операций с сервером в консольной оболочке *mongosh*.

Команда *use example* (рис. 4) установит в качестве текущей БД *example*. Если такой базы нет, то она будет создана автоматически. Далее *db* будет представлять текущую БД, т. е. *example*. После *db* указана коллекция *users*, в которую мы добавим новый объект. Если в реляционной БД таблицы следует создавать заранее, то коллекции при их отсутствии MongoDB создает сама.

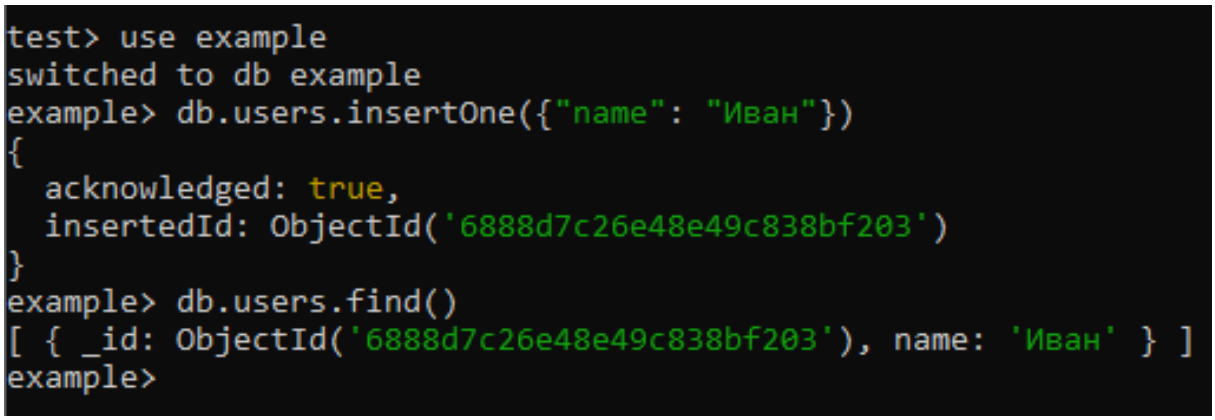


Рис. 4

Метод *db.users.insertOne()* добавляет объект *{"name": "Иван"}* в коллекцию *users* БД *example*. Если объект был успешно добавлен, то консоль выведет результат операции – идентификатор добавленного объекта (создается автоматически).

Команда *db.users.find()* выведет все объекты из БД *example*.

ВНИМАНИЕ! Для вставки в командную строку *mongosh* какого-либо текста (например, из Блокнота) нужно скопировать этот текст в буфер, а затем щелкнуть правой кнопкой мыши в командной строке *mongosh*

Графический интерфейс MongoDB Compass

Работа с сервером MongoDB возможна также через графический клиент *MongoDB Compass*. Интуитивно понятный и гибкий графический интерфейс *Compass* предоставляет подробную визуализацию схемы, показатели производительности в реальном времени и многое другое. Поставляется в трех версиях:

- полнофункциональная версия (*Compass*);
- версия только для чтения без возможности записи или удаления (*Readonly Edition*);
- изолированная версия, единственное сетевое подключение которой осуществляется к экземпляру MongoDB (*Isolated Edition*).

Графический клиент Compass может быть установлен независимо или во время установки сервера. В последнем случае на одном из шагов установщика следует активировать опцию *Install MongoDB Compass*.

Для подключения к серверу MongoDB из клиента *Compass* необходимо предпринять следующие шаги (сервер должен быть запущен):

1. Запустите файл *MongoDBCompass.exe*.
2. В окне приложения соединитесь с существующим сервером или нажмите кнопку *Add new connection (Добавить новое соединение)*.
3. Выбрав новое соединение, отредактируйте при необходимости строку подключения и нажмите кнопку *Save & Connect*.
4. По умолчанию используется строка подключения *mongodb://localhost:27017* или *mongodb://127.0.0.1:27017*.
5. При успешном подключении откроется список БД на сервере (рис. 5).
6. Дальше можно в БД *test* работать с коллекцией *users*, которая была создана ранее. Для базовых операций создания и удаления коллекций, добавления, просмотра, изменения и удаления документов есть соответствующие элементы графического интерфейса.
7. При выборе коллекции и отображении ее документов над списком данных появляется кнопка добавления документа, в том числе путем импорта из внешнего файла. Напротив каждого документа есть опции для его просмотра, изменения и удаления.

8. Если графических возможностей недостаточно для управления данными, то в *Compass* есть встроенный консольный клиент *Mongosh* (рис. 6), который открывается нажатием на кнопку *Open MongoDB shell*.

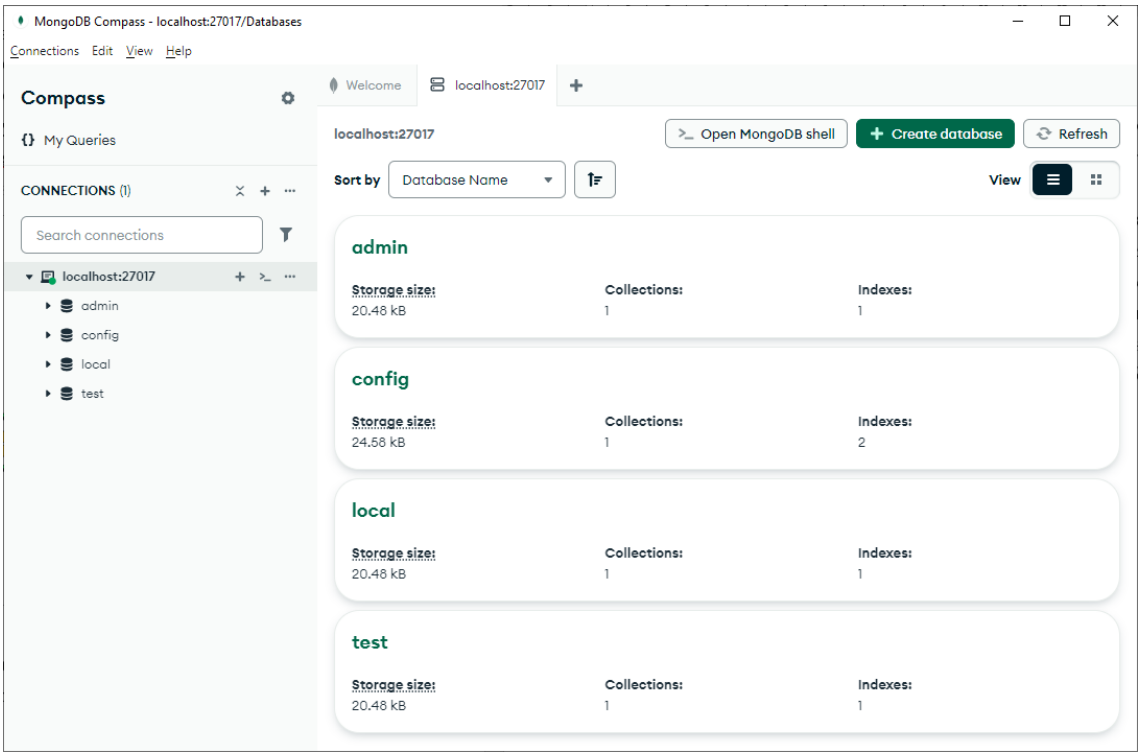


Рис. 5

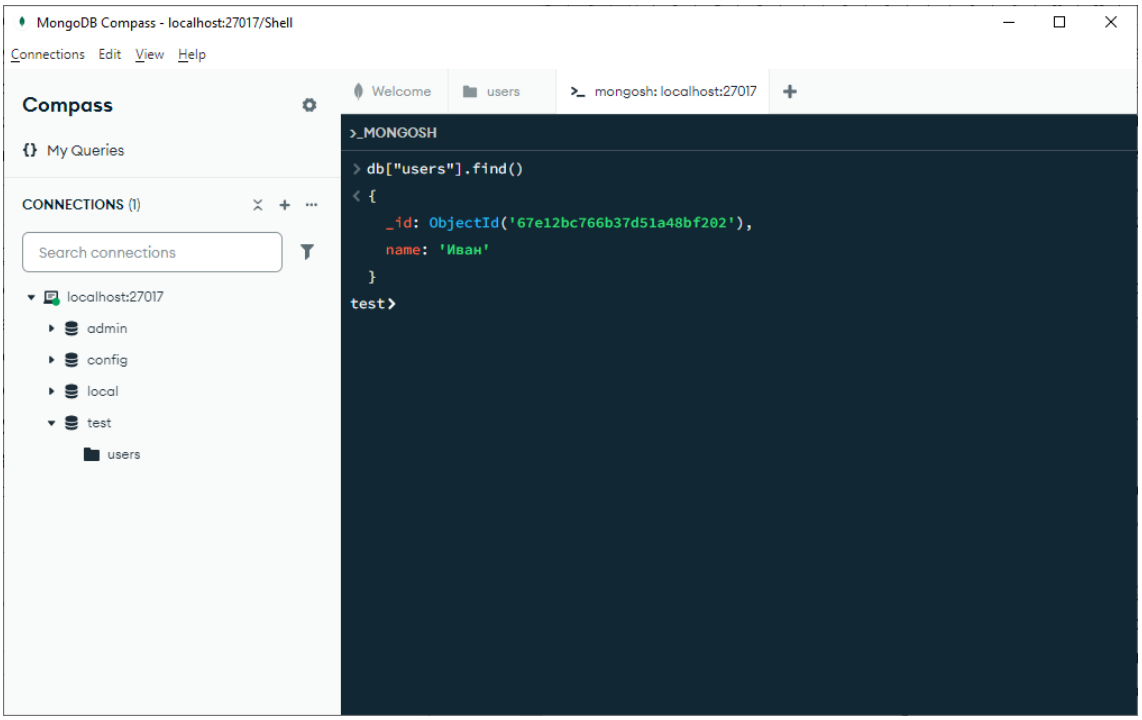


Рис. 6

Консольный клиент в *Compass* упрощает написание запросов, позволяя копировать и вставлять запросы в *mongosh*. Есть функция автодополнения, предлагающая варианты доступных функций, уведомление об устаревших функциях.

Выполнение работы

В ходе выполнения лабораторной работы необходимо:

- установить сервер MongoDB;
- установить клиент командной строки *MongoDB Shell*;
- установить графический клиент *MongoDB Compass*;
- осуществить подключение к БД *test*;
- получить практику работы с командами клиента командной строки, справочную информацию по которым дает команда *help*.

Содержание отчета

1. Номер и название лабораторной работы.
2. Цель лабораторной работы.
3. Снимки экранных форм, показывающие порядок выполнения лабораторной работы.
4. Результаты лабораторной работы.

Лабораторная работа № 2
БАЗЫ ДАННЫХ, КОЛЛЕКЦИИ И ДОКУМЕНТЫ

Цель работы:

– изучить принципы построения БД, коллекций и документов;

– изучить типы данных, используемых в MongoDB.

Теоретические сведения

Основными строительными блоками для хранения и управления данными в MongoDB являются базы данных, коллекции и документы. Документы организованы в коллекции, которые хранятся в базах данных.

База данных в MongoDB – это контейнер для данных, который содержит ряд коллекций. MongoDB позволяет создавать несколько БД на одном сервере, причем каждая из них логически изолирована от других. Это самый высокий уровень структуры в данной системе.

При запуске MongoDB создаются три БД по умолчанию: *admin*, *config* и *local*, которые используются для внутренних целей.

Ограничения на именование баз данных:

- имена не должны учитывать регистр;
- имена не могут содержать специальные символы (/, . , $, *, | и т. д.);
- имена не могут содержать символы *null* (в системах Windows, Unix и Linux);
- имена не могут быть пустыми и должны содержать не более 64 символов.

Коллекция в MongoDB похожа на таблицу в реляционных БД. Она представляет собой контейнер, содержащий группу документов, и является частью БД. Коллекции обеспечивают структуру данных, но они не имеют схемы (БД MongoDB тоже не имеют схемы). Поэтому схема одного документа в коллекции не обязательно должна быть похожей на схему другого. Одна коллекция может содержать разные типы документов, и од-

на БД может содержать несколько коллекций, каждая из которых хранит разные типы документов.

Коллекции могут иметь семантическое значение, но в целом они представляют собой просто способ разделения различных наборов документов в системе. Можно назначать разные свойства разным коллекциям и выполнять действия с конкретными коллекциями документов.

Ограничения на именование коллекций:

- имя должно начинаться с символа подчеркивания (_) или буквы (a–z или A–Z);
- имя не должно начинаться с цифры, префикса *system.* (зарезервирован для системных коллекций MongoDB), содержать символ $, пустую строку, символ *null*;
- максимальная длина имени составляет 120 байт (включая имя БД, точку-разделитель и имя коллекции).

Документ – это контейнер для информации, представляющей одну запись или объект, содержащий связанные описательные данные. Документы могут иметь гибкую структуру, не обязательно совпадающую со структурой других документов той же коллекции, а также быть вложенными.

Документы MongoDB используют формат сериализации данных BSON (двоичное представление нотации JSON JavaScript). Это обеспечивает организованную структуру с определенными типами данных, которые можно запрашивать и обрабатывать программно.

Документы BSON представлены парой фигурных скобок {}, которые содержат пары «ключ-значение» и «поле-значение». Поле идет первым и отображается в виде строки. Значение может иметь любой допустимый тип данных BSON. Между полем и его значением стоит двоеточие. Запятые отделяют пары полей и значений друг от друга:

```
{ _id: 80380,
vehicle.type: "car",
mileage: 7377.80.
```

```
color: "blue",
markers: ["US","UK"],
options: {
       transmission: "automatic",
       num_doors: 4,
       power_windows: true }
}
```

Поля обозначают следующее: *_id* – целое число, *vehicle.type* и *color* – строки, *mileage* – число с плавающей точкой, *markers* – массив строк, *options* – содержит вложенный документ со значениями из строки, целого числа и логического значения.

Во многих примерах в названии поля используют двойные кавычки:

```
{ "count": 123 }
```

Они также присутствуют вокруг строковых значений в массиве:

```
{ "colors": ["red", "white", "blue"] }
```

Эти двойные кавычки не всегда строго необходимы. Есть примеры, когда имя поля не заключено в двойные кавычки, или когда строки заключаются в одинарные кавычки. Это связано с различиями между используемым оболочкой *mongosh* языком JavaScript и JSON (BSON). Пример в JavaScript:

```
> var test = ["one", "two", "three"]
> console.log(test)
['one', 'two', 'three']
```

Мы выделили элементы массива двойными кавычками, но при выходе из системы они будут заключены в одинарные, поскольку в JavaScript допустимы оба варианта. Но если есть имя поля, начинающееся с цифры 2, мы получим сообщение об ошибке:

```
> var object = { first: "1st", 2nd: 'second' }
```

```
var object = { first: "1st", 2nd: 'second' }
                                ^^
Uncaught SyntaxError: Invalid or unexpected token
```

Первое поле с именем *first* допустимо, второе поле с именем *2nd* – нет (начинается с цифры). Если же заключить имя этого поля в кавычки, оно станет допустимым (теперь это явная строка):

```
>  var object = { first: "1st", 2nd: 'second' }
> console.log(object)
{ first: "1st", 2nd: 'second' }
```

ВНИМАНИЕ! По этим причинам рекомендуется заключать имена полей и строки в двойные кавычки, что и делают в большинстве случаев [6]

Просмотр существующих в системе БД выполняется для оценки ее текущего состояния. Отобразить все БД, к которым у пользователя есть доступ, позволяет метод *show dbs*.

Ответ сервера MongoDB содержит название БД, отправлявшей запрос, а также краткую информацию о текущем пространстве хранения. При этом можно обнаружить, что в текущий момент используется БД, не показанная командой *show dbs*. Пока первый документ не будет записан в БД, она в MongoDB фактически не будет создана. Узнать, с какой БД в данный момент идет работа, можно с помощью команды *db.getName()* или ее сокращенного псевдонима *db*.

Переключиться на другую БД позволяет команда *use <имя БД>*.

Получить базовую информацию о текущей БД можно с помощью метода *db.stats()*. В выходных данных будет отображена информация о количестве коллекций в ней, статистика хранения, информация об индексах и т. п.

Просмотреть доступные коллекции в текущей БД можно с помощью метода *show collections*. Альтернативой является метод *db.getCollectionNames()*.

Показать дополнительную информацию о коллекциях в текущей БД позволяет метод *db.getCollectionInfos()*. Получить сведения о конкретной коллекции можно с помощью фильтрации результатов.

Получить статистику по коллекции позволяет метод *db.<коллекция>.countDocuments()*. Для просмотра базовой статистики о коллекциях в текущей БД используют метод *db.printCollectionStats()*, который выводит огромный объем информации, полезный для глубокого изучения коллекции.

Типы данных в MongoDB

Пользователь MongoDB способен гибко выбирать структуру данных. Он не привязан к определенной схеме и для любого поля документа может использовать любой доступный тип данных MongoDB. При этом следует применить и схему, чтобы добавить проверку коллекций. Если схема есть, то она может повлиять на типизацию данных. При планировании БД важно понимать, как выбрать нужный тип данных. Это будет определять работу с данными и способ их хранения.

Рассмотрим типы данных, используемых в MongoDB.

1. JSON и BSON.

MongoDB и другие документные БД NoSQL применяют формат JSON (нотацию объектов JavaScript) для представления записей данных в виде документов. Преимущества использования JSON:

- простота чтения, изучения и узнаваемость конструкций;
- гибкость формата, будь то разреженный, иерархический или глубоко вложенный;
- самоописание, облегчающее приложениям работу с данными;
- возможность применять минимальное количество основных типов.

Документы в MongoDB рассматривают как JSON-подобные в том смысле, что они концептуально похожи на объекты в JavaScript. JSON является простым представлением данных – спецификация может быть описана в одном параграфе и содержит только шесть типов данных:

- *null*;
- логический тип;
- число;
- строка;
- массив;
- объект.

Благодаря этому JSON легко понимать, конвертировать и запоминать. С другой стороны, выразительные возможности данного формата ограничены:

- нет типа даты;
- есть только один тип числа (нельзя различить числа с плавающей точкой и целые числа, не говоря уже о различии между 32-битными и 64-битными числами);
- невозможно обозначить другие часто используемые типы, такие как регулярные выражения или функции.

MongoDB использует формат BSON, допускающий дополнительные типы данных, недоступные для формата JSON. При этом BSON сохраняет существенную природу пары типа «ключ-значение» JSON, но в двоичном кодировании. Как и JSON, BSON поддерживает встраивание документов и массивов в другие документы и массивы. В MongoDB каждый тип BSON имеет как целочисленные, так и строковые идентификаторы (табл. 1) [5].

Таблица 1

Type	Number	Alias	Notes
Double	1	`double`	
String	2	`string`	
Object	3	`object`	
Array	4	`array`	

Окончание табл. 1

Type	Number	Alias	Notes
Binary data	5	`binData`	
ObjectID	7	`objectId`	
Boolean	8	`bool`	
Date	9	`date`	
Null	10	`null`	
Regular expression	11	`regex`	
JavaScript	13	`javascript`	
32-bit integer	16	`int`	
Timestamp	17	`timestamp`	
64-bit integer	18	`long`	
Decimal128	19	`decimal`	New in version 3.4
Min. key	–1	`minKey`	
Max. key	127	`maxKey`	
Undefined	6	`undefined`	Deprecated
DBPointer	12	`dbPointer`	Deprecated
Symbol	14	`symbol`	Deprecated
JavaScript (with scope)	15	`JavascriptWithScope`	Deprecated

2. Строковый тип данных String – *{"x": "текст"}*.

Данный тип чаще всего используется в MongoDB. Любое значение, записанное в кавычках (апострофах) в JSON, является строковым значением.

Любое значение, которое нужно сохранить в виде текста, лучше всего вводить в формате String. Строки BSON имеют формат UTF-8, поэтому в них могут храниться разные символы из разных языков. Размер строковых полей указывать не нужно (этот вопрос решит сама MongoDB). Драйверы языков программирования преобразуют строковый формат языка в UTF-8 при сериализации и десериализации BSON.

При вставке документа с типом данных String используется метод *insertOne* (рис. 7).

```
test> db.person.insertOne({"name": "Ольга"})
{
  acknowledged: true,
  insertedId: ObjectId('686cec00cc2aafeb2a8bf203')
}
test>
```

Рис. 7

Запрос к коллекции осуществляют с помощью методов *find().pretty()* (рис. 8).

```
test> db.person.find().pretty()
[ { _id: ObjectId('686cec00cc2aafeb2a8bf203'), name: 'Ольга' } ]
test>
```

Рис. 8

Для проверки значений при выполнении вставок можно использовать оператор *$type*. Например, нужно вставить в коллекцию *person* несколько дополнительных документов с полем *name*. Проверить, что для *name* применили тип String, можно, используя в запросе псевдоним или числовое значение типа (рис. 9).

```
test> db.person.find( {"name": {$type: "string"} } )
[ { _id: ObjectId('686cec00cc2aafeb2a8bf203'), name: 'Ольга' } ]
test> db.person.find( {"name": {$type: 2} } )
[ { _id: ObjectId('686cec00cc2aafeb2a8bf203'), name: 'Ольга' } ]
test>
```

Рис. 9

Оба запроса возвращают выходные данные всех документов, имеющих сохраненное значение типа String для *name*. Если же запросить тип, который не сохранен в поле *name* какого-либо документа, например *Null* (рис. 10), то результата не будет. Это указывает на то, что в *name* хранится другой тип данных.

```
test> db.person.find( {"name": {$type: ["string", "null"] } } )
[ { _id: ObjectId('686cec00cc2aafeb2a8bf203'), name: 'Ольга' } ]
test>
```

Рис. 10

С помощью оператора *$type* можно запрашивать несколько типов данных одновременно.

3. Числа и числовые значения.

MongoDB поддерживает ряд числовых типов данных для различных сценариев. Решение о выборе типа зависит от вариантов использования данных. В JSON все числа называются *Number*. Система должна преобразовать их в ближайший собственный тип данных. По умолчанию оболочка использует 64-битные числа с плавающей точкой. Следовательно, в оболочке эти числа выглядят привычным образом: *{"x": 3.14}, {"x": 3}.*

4. Тип данных Integer.

Этот тип используется для хранения целых чисел. Его значения могут быть как положительными, так и отрицательными. В MongoDB есть два типа: 32-Bit Integer и 64-Bit Integer. Их можно представить, используя целочисленные или строковые идентификаторы (см. табл. 1).

При вставке типа Integer в MongoDB используется метод *insertOne* (рис. 11).

```
test> db.person.insertOne({"age": 26})
{
  acknowledged: true,
  insertedId: ObjectId('686cef99cc2aafeb2a8bf204')
}
test> db.person.find().pretty()
[
  { _id: ObjectId('686cec00cc2aafeb2a8bf203'), name: 'Ольга' },
  { _id: ObjectId('686cef99cc2aafeb2a8bf204'), age: 26 }
]
test>
```

Рис. 11

Тип 32-Bit Integer имеет 32-битное целочисленное представление, что полезно для хранения меньших целочисленных значений. При увеличении размера числа можно перейти к 64-битному целочисленному значению 64-Bit Integer. В случае с целыми числами используются конструкторы *NumberInt()* или *NumberLong()*, которые обозначают 4-байтовые или 8-байтовые целые числа со знаком соответственно: *{"x": NumberInt ("3")}, {"x": NumberLong ("3")}.*

Добавим в коллекцию документ с полем *height*, описав тип данных этого поля как 32-Bit Integer, и проверим, что мы применили для *height* именно этот тип, используя в запросе оператор *$type* и числовое значение этого типа – 16 (рис. 12).

```
test> db.person.insertOne({"height": NumberInt( "17") } )
{
  acknowledged: true,
  insertedId: ObjectId('686cf191cc2aafeb2a8bf205')
}

test> db.person.find( {"height": {$type: 16} } )
[ { _id: ObjectId('686cf191cc2aafeb2a8bf205'), height: 17 } ]
test>
```

Рис. 12

Добавим в коллекцию документ с полем *width*, описав тип данных поля как 64-Bit Integer, и проверим, что для *width* применен именно этот тип, используя в запросе оператор *$type* и числовое значение типа – 18 (рис. 13).

```
test> db.person.insertOne({"width" : NumberLong( "41") } )
{
  acknowledged: true,
  insertedId: ObjectId('686cf367cc2aafeb2a8bf206')
}
test> db.person.find( {"width" : {$type: 18} } )
[ { _id: ObjectId('686cf367cc2aafeb2a8bf206'), width: Long('41') } ]
test>
```

Рис. 13

5. Тип данных Double.

В BSON заменой типа JSON Number по умолчанию является тип Double. Он используется для хранения значения с плавающей запятой, которое может работать с большим количеством десятичных знаков, но не всегда точно (рис. 14).

```
test> db.person.insertOne({"score": 89.6})
{
  acknowledged: true,
  insertedId: ObjectId('686cf471cc2aafeb2a8bf207')
}
test>
```

Рис. 14

При расчете с двойной точностью могут быть небольшие различия между входными и выходными данными, что потенциально может привести к неожиданному поведению.

6. Тип данных Decimal128.

Для выполнения операций с очень большими числами и большим диапазоном чисел с плавающей запятой MongoDB имеет более точный тип BSON – Decimal128.

Это наиболее полезный тип для значений, требующих высокой точности (например, при использовании денежных операций). Тип Decimal128 обеспечивает 128-битное десятичное представление для хранения чисел, где важно точное округление десятичных дробей. Он поддерживает 34 десятичных цифры значащего числа и диапазон экспоненты от −6143 до +6144.

Для вставки значения с использованием этого типа данных следует использовать конструктор *NumberDecimal()* с числом, представленным как String (рис. 15), чтобы MongoDB не использовала тип по умолчанию Double.

```
test> db.person.insertOne({"pay": NumberDecimal("150.99")})
{
  acknowledged: true,
  insertedId: ObjectId('686cf55dcc2aafeb2a8bf208')
}
test> db.person.find( {"pay": {$type: 19} } )
[
  {
    _id: ObjectId('686cf55dcc2aafeb2a8bf208'),
    pay: Decimal128('150.99')
  }
]
test>
```

Рис. 15

Рассмотрим возможную ситуацию с потерей точности. Допустим, нужно вставить число со многими десятичными знаками типа Double в MongoDB (рис. 16).

```
test> db.person.insertOne({"value": 9999999.4999999999})
{
  acknowledged: true,
  insertedId: ObjectId('686cf639cc2aafeb2a8bf209')
}
test>
```

Рис. 16

На рис. 17 отображен результат запроса.

```
test> db.person.find().pretty()
[
  {
    _id: ObjectId('686cf55dcc2aafeb2a8bf208'),
    pay: Decimal128('150.99')
  },
  { _id: ObjectId('686cf639cc2aafeb2a8bf209'), value: 9999999.5 }
]
test>
```

Рис. 17

Это число округлено до 9999999.5 с потерей точного значения, которое мы вводили. Это делает тип Double непригодным для хранения чисел со многими десятичными знаками.

Покажем, как теряется точность с Decimal128 при неявной передаче Double вместо String. Вставим число типа Double, но с *NumberDecimal()*, чтобы сделать его типом Decimal128 (рис. 18).

```
test> db.person.insertOne({"num": NumberDecimal(9999999.4999999999)})
Warning: NumberDecimal: specifying a number as argument is deprecated and may
 lead to loss of precision, pass a string instead
{
  acknowledged: true,
  insertedId: ObjectId('686cfa3ecc2aafeb2a8bf20a')
}
test>
```

Рис. 18

При выполнении этой вставки выдается предупреждающее сообщение, которое указывает на то, что передаваемое число может иметь потерю точности. Предлагается использовать *NumberDecimal()* и String.

Если мы проигнорируем предупреждение и вставим документ, потеря точности будет видна в результатах запроса из-за округления значения (рис. 19).

```
test> db.person.find().pretty()
[
  {
    _id: ObjectId('686cf55dcc2aafeb2a8bf208'),
    pay: Decimal128('150.99')
  },
  { _id: ObjectId('686cf639cc2aafeb2a8bf209'), value: 9999999.5 },
  {
    _id: ObjectId('686cfa3ecc2aafeb2a8bf20a'),
    num: Decimal128('9999999.5')
  }
]
test>
```

Рис. 19

Результат будет неожиданным. Тип данных поля *num* – Decimal128, однако введенное число было округлено.

В любом случае использования, требующем точных и безошибочных значений, этот возврат может вызвать проблемы. Действия, связанные с денежными операциями, являются примером ситуации, где точность чрезвычайно важна, а наличие точных значений имеет решающее значение для точных расчетов.

7. Тип данных Date – *{"x": new Date()}.*

Тип данных BSON Date – это 64-битное целое число, которое представляет количество миллисекунд, прошедших с начала эпохи Unix (01 января 1970 г.). Этот тип данных хранит текущую дату или время и может быть возвращен как объект даты или как строка.

Даты в MongoDB хранятся в UTC (Coordinated Universal Time – Всемирное координированное время). Тип BSON Date – числа со знаком. Отрицательные значения представляют даты до 01 января 1970 г. Допускается диапазон в 135 миллионов лет до и после 1970 г. У Date нет поля временной метки с типом данных часового пояса, как в некоторых реляционных БД.

Существует три метода для возврата значений даты:

- *Date()* – возвращает строку;
- *new Date()* – возвращает объект даты, используя обертку *ISODate();*
- *ISODate()* – также возвращает объект даты, используя обертку *ISODate().*

При создании нового объекта Date следует всегда использовать метод *new Date()* (рис. 20), а не просто *Date().* Вызов конструктора в качестве функции (не используя слово *new*) возвращает строковое представление даты, а не фактический объект Date. Это не выбор MongoDB, так работает JavaScript. Неосторожное использование конструктора Date может привести к смешению из строк и дат. Строки не совпадают с датами, и наоборот, что может вызвать проблемы с удалением, обновлением, запросами и т. д.

```
test> var date1 = Date()

test> var date2 = new Date()

test> var date3 = ISODate()

test> db.person.insertOne({"firstDate": date1, "secondDate": date2, "thirdDate": date3})
{
  acknowledged: true,
  insertedId: ObjectId('686cfc9ecc2aafeb2a8bf20b')
}
test>
```

Рис. 20

При возвращении мы видим, что в первом случае действительно возвращается строковое значение, а во втором и третьем случаях – объект даты (рис. 21).

```
test> db.person.find().pretty()
[
  {
    _id: ObjectId('686cfc9ecc2aafeb2a8bf20b'),
    firstDate: 'Tue Jul 08 2025 17:05:13 GMT+0600 (Омск, стандартное время)',
    secondDate: ISODate('2025-07-08T11:05:32.267Z'),
    thirdDate: ISODate('2025-07-08T11:05:55.606Z')
  }
]
test>
```

Рис. 21

8. Тип данных Timestamp.

Тип данных Timestamp также применяется для представления времени, однако не связан с типом Date и наиболее полезен для внутреннего употребления. Он представляет собой последовательность символов, используемую для описания даты и времени возникновения события. Это 64-битное значение, в котором:

- наиболее значимые 32 бита являются значением time_t (секунды с эпохи Unix);
- наименее значимые 32 бита упорядоченно увеличиваются для операций в течение данной секунды.

При вставке документа, содержащего поля с пустыми метками времени, MongoDB заменит пустое значение такой метки текущим. Исключением является случай, когда пустую метку времени содержит поле *_id*. Значение временной метки всегда вставляется как есть и не заменяется.

Вставка нового значения Timestamp использует функцию *new Timestamp()* (рис. 22).

```
test> db.person.insertOne({"ts": new Timestamp()})
{
  acknowledged: true,
  insertedId: ObjectId('686d07a0cc2aafeb2a8bf20c')
}
test> db.person.find().pretty()
[
  {
    _id: ObjectId('686d07a0cc2aafeb2a8bf20c'),
    ts: Timestamp({ t: 1751975840, i: 1 })
  }
]
test>
```

Рис. 22

9. Тип данных Object.

Данный тип используется для хранения встроенных документов. Такой документ представляет собой серию вложенных документов в парном формате «поле-значение» (рис. 23).

```
test> var classGrades = {"Физика": 88, "Информатика": 92, "Литература": 79}

test> db.person.insertOne({"student_name": "Иван Петров", "report_card": classGrades})
{
  acknowledged: true,
  insertedId: ObjectId('686d0a1fcc2aafeb2a8bf20d')
}
test> db.person.find().pretty()
[
  {
    _id: ObjectId('686d0a1fcc2aafeb2a8bf20d'),
    student_name: 'Иван Петров',
    report_card: { 'Физика': 88, 'Информатика': 92, 'Литература': 79 }
  }
]
test>
```

Рис. 23

В объектах можно хранить все, что можно хранить в документах, в том числе связанные документы меньшего размера вместе.

Соответственно, тип Object оптимизирован для хранения данных, к которым лучше всего обращаться вместе. Это обеспечивает эффективность хранения, скорость и долговечность в отличие от хранения каждого значения класса отдельно.

10. Тип данных Binary data (BinData).

Тип BinData сохраняет двоичные данные в значении поля. Ими нельзя манипулировать из оболочки. Двоичные данные – единственный способ сохранять строки в БД не в формате UTF-8 (рис. 24). Тип BinData используют при хранении нетекстовых данных (изображений, файлов и т. п.).

```
test> db.project.insertOne({"name": "num", "binaryValue": "110011001"})
{
  acknowledged: true,
  insertedId: ObjectId('686d0d0acc2aafeb2a8bf20e')
}
test> db.person.find().pretty()
[
  {
    _id: ObjectId('686d0d0acc2aafeb2a8bf20e'),
    name: 'num',
    binaryValue: '110011001'
  }
]
test>
```

Рис. 24

11. Тип данных ObjectId.

Тип ObjectId (идентификатор объекта) специфичен для MongoDB и хранит уникальный идентификатор документа. Каждый документ имеет поле *_id* от начала до конца существования. Если для вставляемого документа оно оставлено пустым, то MongoDB автоматически сгенерирует ObjectId для этого поля. Значение ключа *_id* может быть любого типа, но по умолчанию используется ObjectId.

В одной коллекции каждый документ должен иметь уникальное значение *_id*. Если коллекций несколько, то каждая из них может иметь документ с *_id* = 123. Но ни одна коллекция не может содержать более одного документа с *_id* = 123.

Что-либо делать с ObjectId нежелательно, но можно (с осторожностью!) использовать его в своих целях. ObjectId имеет размер 12 байт и состоит из трех частей:

- 4-байтовое значение временной метки создания ObjectId в секундах, прошедших с эпохи Unix (01 января 1970 года);
- 5-байтовое случайное значение (первые три – идентификатор машины, другие два – идентификатор процесса);
- 3-байтовый инкрементный счетчик, инициализированный случайным значением.

Структура ObjectId представлена на рис. 25.

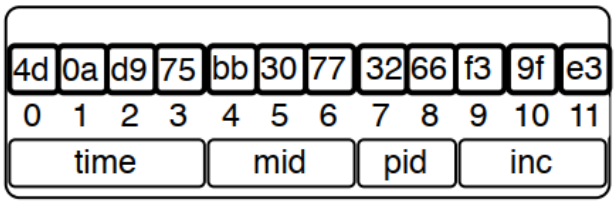


Рис. 25

Достоинство схемы автоматической нумерации – любой процесс на любой машине сам отвечает за генерацию идентификаторов и не вступает в конфликт с другими экземплярами сервера. Это проектное решение – прямой намек на распределенную природу MongoDB.

Некоторые правила использования ObjectId для *_id*:

- в *mongosh* время создания ObjectId доступно с помощью метода *ObjectId.getTimestamp();*
- сортировка по полю *_id*, в котором хранятся типы данных ObjectId, приведет к сортировке по времени создания для каждого документа с точностью до 1 секунды.

Значения ObjectId должны со временем увеличиваться, но не обязательно стать монотонными, поскольку они:

- содержат только 1 секунду временного разрешения, поэтому значения, созданные в течение секунды, не имеют гарантированного порядка;
- генерируются клиентами, которые могут иметь разные системные часы.

12. Тип данных Boolean – *{"x": true}*.

MongoDB имеет собственный тип данных Boolean для хранения значений *true* и *false* в коллекции (рис. 26).

```
test> db.project.insertOne({"isCorrect": true, "isInCorrect": false})
{
  acknowledged: true,
  insertedId: ObjectId('686d0faccc2aafeb2a8bf20f')
}
test> db.project.find().pretty()
[
  {
    _id: ObjectId('686d0faccc2aafeb2a8bf20f'),
    isCorrect: true,
    isInCorrect: false
  }
]
test>
```

Рис. 26

13. Тип данных Regular Expression – *{"x": / текст / i}*.

Этот тип позволяет хранить регулярные выражения в качестве значения поля. MongoDB использует в качестве языка регулярных выражений PCRE (Perl-совместимые регулярные выражения) (рис. 27). BSON позволяет избежать типичного шага «конвертировать из строки», который обычно встречается при работе с регулярными выражениями и БД.

Этот тип наиболее полезен при написании объектов БД, требующих шаблонов проверки. Шаблоны хранятся в виде регулярных выражений, а не строк. Это позволяет запрашивать определенную строку и получать в ответ документы, регулярное выражение которых соответствует нужной строке.



Рис. 27

14. Тип данных JavaScript (без области видимости).

Этот тип, как и ранее упомянутый тип BSON Regular Expression, позволяет MongoDB хранить функции JavaScript без области действия как отдельный тип. Добавление документа в коллекцию с типом данных JavaScript представлено на рис. 28.

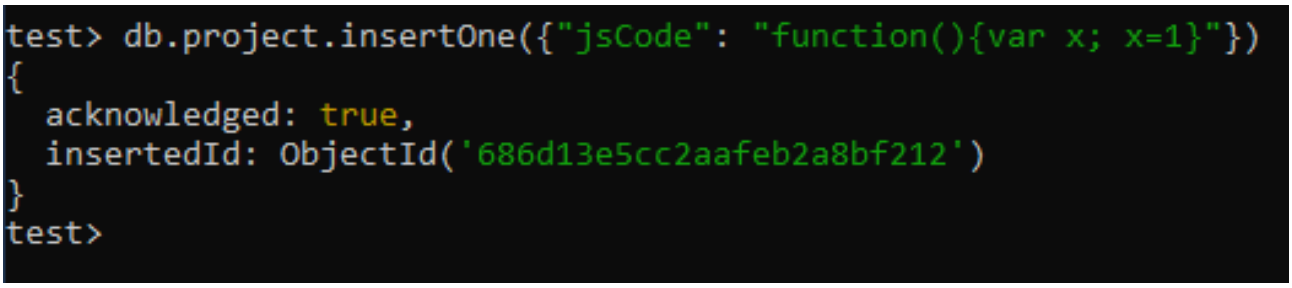


Рис. 28

Эта функциональность позволяет хранить функции JavaScript внутри коллекций MongoDB, если это необходимо для конкретного случая использования.

15. Тип данных Null – *{"x": null}*.

Данный тип можно использовать в MongoDB для обозначения как неопределенного значения, так и несуществующего поля. Если поле в доку-

менте установлено на Null или его не существует, то MongoDB рассматривает его как неопределенное. При поиске неопределенного значения MongoDB сопоставляет как документы, содержащие поле, установленное на Null, так и документы, которые не содержат это поле.

16. Тип данных Array.

Наборы или списки значений могут быть представлены в виде массивов: *{ "x": ["a", "b", "c"] }*.

Практически все, что можно сделать с массивами в JavaScript, можно сделать и в MongoDB. Они могут содержать в качестве значений различные типы данных. Значения массива могут иметь любой поддерживаемый тип данных для обычных пар типа «поле-значение». Возможен массив чисел, строк, сочетание того и другого, массивы внутри массивов, массивы объектов и т. д.

Массивы – это значения, которые могут использоваться взаимозаменяемо как для упорядоченных (как если бы они были списками, стеками или очередями), так и для неупорядоченных операций (как если бы они были наборами). В приведенном документе поле *things* имеет значение массива: *{ "things": ["pie", 3.14] }*.

MongoDB понимает их структуру и знает, как добраться до элементов массивов для выполнения операций с их содержимым. Это позволяет делать запросы к массивам и создавать индексы, используя их содержимое. Можно запрашивать все документы, где 3.14 является элементом массива *things*.

MongoDB допускает атомарные обновления, которые изменяют содержимое массивов (например, изменение значения *"pie"* на *"pi"*).

17. Вложенные документы – *{"x": {"y": "текст"}}*.

Документ может использоваться как значение поля. Такой документ называется вложенным. Вложенные документы можно использовать для организации данных более естественным образом, чем просто плоская структура пар «поле-значение».

Например, если есть документ, обозначающий человека, и мы хотим сохранить его адрес, то можно поместить эту информацию во вложенный документ *address*. Значением поля *address* здесь является вложенный до-

кумент со своими собственными парами «поле-значение» для *street*, *city* и *state*.

MongoDB понимает структуру вложенных документов и может использовать их для создания индексов, выполнения запросов и обновлений.

Вложенные документы могут изменить способ работы с данными. В реляционных СУБД документ моделируется как две строки в двух разных таблицах (*person* и *addresses*). В MongoDB можно встраивать документ *address* в документ *person*.

Однако в случае с MongoDB избыточность данных может быть выше. Допустим, что *addresses* были отдельной таблицей в реляционной БД, и нужно было исправить опечатку в адресе. При соединении таблиц *person* и *addresses* получали обновленный адрес для всех, кто использует его. При работе с MongoDB нужно исправлять опечатку в документе каждого человека.

Выполнение работы

В ходе выполнения лабораторной работы необходимо:

- изучить принципы хранения и управления данными в MongoDB;
- изучить принципы построения БД, коллекций и документов;
- изучить типы данных, используемых в MongoDB;
- изучить форматы представления данных JSON и BSON;
- научиться переключаться между БД;
- научиться получать базовую информацию о текущей БД;
- научиться просматривать коллекции в БД и получать базовую статистику по заданной коллекции.

Содержание отчета

1. Номер и название лабораторной работы.
2. Цели лабораторной работы.
3. Снимки экранных форм, показывающие порядок выполнения лабораторной работы.
4. Результаты лабораторной работы.

Лабораторная работа № 3
БАЗОВЫЕ ОПЕРАЦИИ CRUD В MONGODB

Цель работы:

– освоить основные операции создания, чтения, обновления и удаления документов (CRUD) в MongoDB;

– рассмотреть способы задания логических связей между коллекциями.

ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ

В MongoDB нет явной команды для создания новой базы данных. Но это можно сделать неявно, указав, что требуется записать новые документы в новую базу. Чтобы подготовить MongoDB к записи в новую базу данных, введите команду *use*. После этого можно создать новую коллекцию, а в ней документ.

БД удаляется командой *db.dropDatabase()*. Перед этим следует убедиться, что выбрана нужная БД. Удаленная БД больше не отображается в списке доступных.

Операции CRUD необходимы, чтобы манипулировать данными и просматривать их в оболочке. Выделяют четыре операции:

- *Create* (создать) – добавить новые документы в коллекцию;
- *Read* (читать) – извлечь документы из коллекции;
- *Update* (обновить) – изменить существующие документы;
- *Delete* (удалить) – удалить документы из коллекции.

Операции создания или вставки (табл. 2) используются для добавления новых документов в коллекцию. Если она не существует, то эти операции создадут новую в БД.

Таблица 2

Метод	Описание
db.collection.insertOne()	Вставка отдельного документа в коллекцию
db.collection.insertMany()	Вставка нескольких документов в коллекцию
db.createCollection()	Создание пустой коллекции

Таким образом, есть два варианта создания новой коллекции – неявный и явный. Как и в случае с БД, в MongoDB коллекция появляется автоматически при первой записи в ней документа. Команда *insertOne()* создаст один документ. При этом MongoDB выполнит три действия:

- создаст БД, указанную в команде *use*;
- создаст новую коллекцию в этой БД;
- создаст документ в коллекции, используя данные, предоставленные команде *insertOne()*.

Явный способ – использование метода *db.createCollection()*, который позволяет создавать коллекции без добавления в них каких-либо документов. Новая коллекция отобразится при запросе, но в ней не будет документов.

Метод *db.createCollection()* позволяет указывать различные параметры при создании. Например, можно задать ограничение, которое поддерживает верхний предел выделенного размера коллекции, удаляя самый старый документ при заполнении. Создадим ограниченную коллекцию *notifications* с максимальным объемом не более 10240 байт информации:

```
db.createCollection ("notifications",
   { capped: true, size: 10240 } )
```

Операции чтения (табл. 3) используются для извлечения документов из коллекции, иначе говоря, для запроса коллекции на наличие документа.

Таблица 3

Метод	Описание
db.collection.find()	Извлечение документов из коллекции
db.collection.findOne()	Извлечение одного документа, соответствующего критериям запроса

Метод *pretty()* используется для оформления результата запроса таким образом, чтобы его было легче читать. Методам *find()* и *findOne()* можно

передавать критерии в виде документа запроса. Это ограничит документы, соответствующие запросу. Оболочка автоматически отобразит до 20 подходящих документов, но можно получить и другие.

Операции обновления (табл. 4) используются для обновления или изменения существующего документа в коллекции. Можно обновить один документ или несколько, соответствующих заданному запросу.

Таблица 4

Метод	Описание
db.collection.updateOne()	Обновление в коллекции отдельного документа, удовлетворяющего заданным критериям
db.collection.updateMany()	Обновление в коллекции нескольких документов, удовлетворяющих заданным критериям
db.collection.replaceOne()	Замена в коллекции отдельного документа, удовлетворяющего заданным критериям

Изменить документ можно методом *updateOne()*. Метод принимает (как минимум) два параметра: первый – критерий для поиска документа, который нужно обновить, второй – документ, описывающий обновления.

Операции удаления (табл. 5) используются для удаления или изъятия документов из коллекции. Можно удалить все документы или только соответствующие определенным критериям.

Таблица 5

Метод	Описание
db.collection.deleteOne()	Удаление из коллекции одного документа, удовлетворяющего заданным критериям
db.collection.deleteMany()	Удаление из коллекции нескольких документов, удовлетворяющих заданным критериям
db.collection.drop()	Удаление всей коллекции

Методы *deleteOne()* и *deleteMany()* навсегда удаляют документы из БД. Оба метода принимают документ фильтра с указанием критериев уда-

ления. Если единственная коллекция в БД удаляется методом *drop()*, то вместе с ней удаляется и вся БД.

Рассмотрим создание БД *e_shop*, содержащей коллекцию *customers*, которая представляет собой список клиентов интернет-магазина из разных городов страны. Здесь собраны сведения о клиентах (Ф. И. О., электронная почта, город, дата рождения, бонусные баллы) и их покупках по различным категориям, сделанных за последние годы. При этом покупки оформляются встроенным массивом документов, каждый из которых включает имя категории, год и суммарный объем покупок в этой категории.

Введенные данные первого клиента представлены на рис. 29.

```
e_shop> db.customers.insertOne(
... { "customerId": 1,
...   "name": "Баранов Владимир Николаевич",
...   "email": "baranov@mail.ru",
...   "city": "Новосибирск",
...   "date_of_birth": ISODate("1975-10-01"),
...   "loyalty_points": 340,
...   "purchases": [
...     { "name": "Электроника",
...       "year": 2023,
...       "value": 60000 },
...     { "name": "Одежда и обувь",
...       "year": 2021,
...       "value": 13000 },
...     { "name": "Одежда и обувь",
...       "year": 2022,
...       "value": 18000 },
...     { "name": "Билеты и отели",
...       "year": 2021,
...       "value": 130000 },
...     { "name": "Дом и сад",
...       "year": 2024,
...       "value": 40000 },
...     { "name": "Книги",
...       "year": 2024,
...       "value": 5500 }
...   ]
... })
```

Рис. 29

Теперь данные первого клиента хранятся в документе (рис. 30).

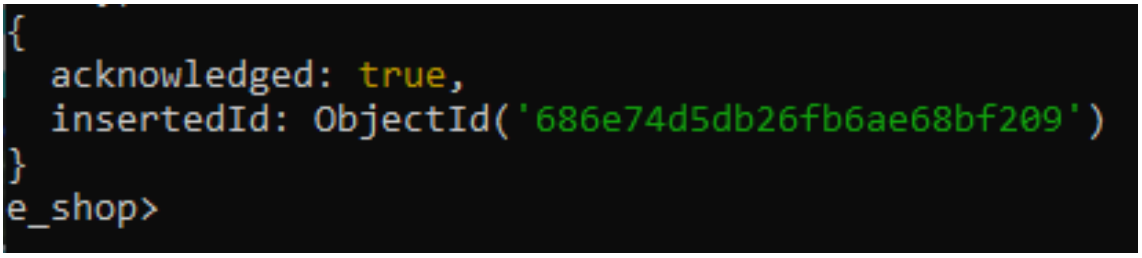


Рис. 30

Введенные данные второго клиента представлены на рис. 31.



Рис. 31

Теперь данные второго клиента хранятся в документе (рис. 32).

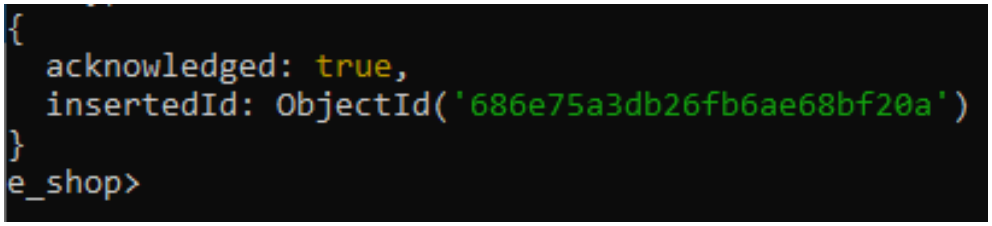


Рис. 32

Введенные данные третьего клиента представлены на рис. 33.



Рис. 33

Теперь данные третьего клиента хранятся в документе (рис. 34).



Рис. 34

Введенные данные четвертого клиента представлены на рис. 35.

```
e_shop> db.customers.insertOne(
... { "customerId": 4,
...   "name": "Медведев Антон Александрович",
...   "email": "medvedev@internet.ru",
...   "city": "Москва",
...   "date_of_birth": ISODate("1980-11-27"),
...   "loyalty_points": 280,
...   "purchases": [
...       { "name": "Электроника",
...         "year": 2022,
...         "value": 105000 },
...       { "name": "Одежда и обувь",
...         "year": 2023,
...         "value": 57000 },
...       { "name": "Детские товары",
...         "year": 2025,
...         "value": 35000 }
...     ]
... })
```

Рис. 35

Теперь данные четвертого клиента хранятся в документе (рис. 36).

```
{
  acknowledged: true,
  insertedId: ObjectId('686e77e1db26fb6ae68bf20c')
}
e_shop>
```

Рис. 36

Введенные данные пятого клиента представлены на рис. 37.

```
e_shop> db.customers.insertOne(
... { "customerId": 5,
...   "name": "Петренко Анастасия Максимовна",
...   "email": "petrenko@mail.ru",
...   "city": "Москва",
...   "date_of_birth": ISODate("2000-08-14"),
...   "loyalty_points": 120,
...   "purchases": [
...       { "name": "Путешествия",
...         "year": 2021,
...         "value": 150000 },
...       { "name": "Одежда и обувь",
...         "year": 2023,
...         "value": 75000 },
...       { "name": "Детские товары",
...         "year": 2024,
...         "value": 14500 },
...       { "name": "Детские товары",
...         "year": 2025,
...         "value": 3500 }
...     ]
... })
```

Рис. 37

Теперь данные пятого клиента хранятся в документе (рис. 38).

```
{
  acknowledged: true,
  insertedId: ObjectId('686e78cddb26fb6ae68bf20d')
}
e_shop>
```

Рис. 38

Введенные данные шестого клиента представлены на рис. 39.

```
e_shop> db.customers.insertOne(
... { "customerId": 6,
...   "name": "Романова Василиса Викторовна",
...   "email": "romanova@inbox.ru",
...   "city": "Санкт-Петербург",
...   "date_of_birth": ISODate("2003-10-01"),
...   "loyalty_points": 100,
...   "purchases": [
...       { "name": "Дом и сад",
...         "year": 2021,
...         "value": 25000 },
...       { "name": "Дом и сад",
...         "year": 2022,
...         "value": 54000 },
...       { "name": "Дом и сад",
...         "year": 2023,
...         "value": 35000 },
...       { "name": "Билеты и отели",
...         "year": 2024,
...         "value": 130000 },
...       { "name": "Путешествия",
...         "year": 2025,
...         "value": 60000 }
...     ]
... })
```

Рис. 39

Теперь данные шестого клиента хранятся в документе (рис. 40).

```
{
  acknowledged: true,
  insertedId: ObjectId('686e7a81db26fb6ae68bf20e')
}
e_shop>
```

Рис. 40

Выполнение работы

В соответствии с выданным описанием предметной области следует создать БД, содержащую нужное количество коллекций и документов.

Содержание отчета

1. Номер и название лабораторной работы.
2. Цели лабораторной работы.
3. Снимки экранных форм, показывающие порядок выполнения лабораторной работы.
4. Результаты лабораторной работы.

Лабораторная работа № 4
ОПРЕДЕЛЕНИЕ ЛОГИЧЕСКИХ СВЯЗЕЙ МЕЖДУ КОЛЛЕКЦИЯМИ

Цель работы: освоить способы задания логических связей между коллекциями различными способами.

Теоретические сведения

При создании документной базы данных не требуется предварительное проектирование схемы, но в соответствии с правилами ER-моделирования выделить можно сущности, атрибуты и связи. Если есть схема реляционной базы данных, следует ориентироваться на нее. Необходимо определить состав коллекций и логические связи между ними.

В MongoDB, в отличие от реляционных СУБД, нет встроенных механизмов для JOIN-операций между коллекциями, но есть несколько способов организации связей между документами разных коллекций. Это следующие способы:

1. Встраивание документов (*Embedding*).
2. Ссылки по идентификатору *ObjectId* (*Referencing*).
3. Гибридный подход.

В лабораторной работе № 3 мы применяли встраивание документов. Встроенные структуры являются гибкими и позволяют хранить больше информации в одном документе.

Преимущества встраивания:

- не нужно делать два запроса к разным коллекциям;
- есть возможность использования атомарных операций обновления;
- есть возможность создания сложных структур с несколькими уровнями вложенности.

Недостатком встраивания является то, что максимальный размер документа в MongoDB составляет 16 Мб, поэтому подход нельзя использовать

для произвольного, постоянно растущего числа атрибутов. Хранение сотен элементов во встроенных массивах также снижает производительность.

Рекомендуется использовать встраивание в следующих случаях:

- отношение «один-ко-многим»;
- частые запросы данных вместе;
- редкое изменение данных.

При использовании ссылок по идентификаторам *ObjectId* важную роль играет мощность связи. Пример отношения «один-к-одному» – связь сущностей *Человек* и *Адрес*. В реляционной БД нужны две таблицы – *Person* и *Address* с внешним ключом *person_id* в таблице *Address* (рис. 41).

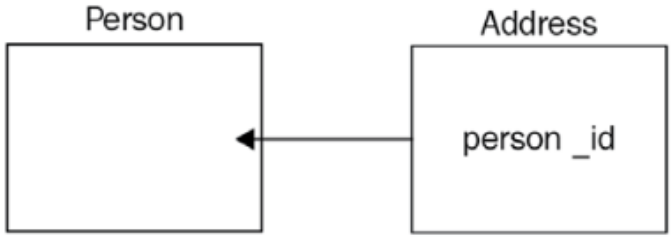


Рис. 41

Моделируем ситуацию, когда один человек имеет один адрес. Создадим новую коллекцию *person* и заполним ее тремя документами (рис. 42).

```
test> db.person.insertMany([
...   { "first_name": "Иван",
...     "last_name": "Ипатьев",
...     "dob": ISODate("2010-10-30")
...   },
...   { "first_name": "Ольга",
...     "last_name": "Птицына",
...     "dob": ISODate("2008-12-04")
...   },
...   { "first_name": "Андрей",
...     "last_name": "Растеряев",
...     "dob": ISODate("2011-03-03") }
... ])
```

Рис. 42

Результат заполнения коллекции *person* представлен на рис. 43.

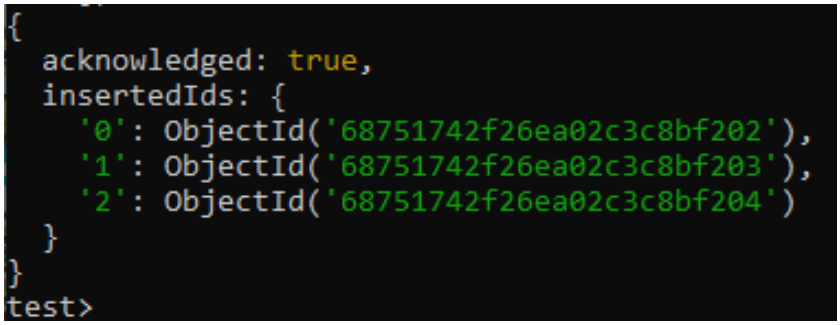


Рис. 43

Зафиксируем идентификаторы объектов, присвоенные системой, затем создадим новую коллекцию *address*, также заполнив ее тремя документами (рис. 44).



Рис. 44

ВНИМАНИЕ! При описании идентификаторов в связанной коллекции следует обязательно использовать метод *ObjectId()*

Результат заполнения коллекции *address* представлен на рис. 45.

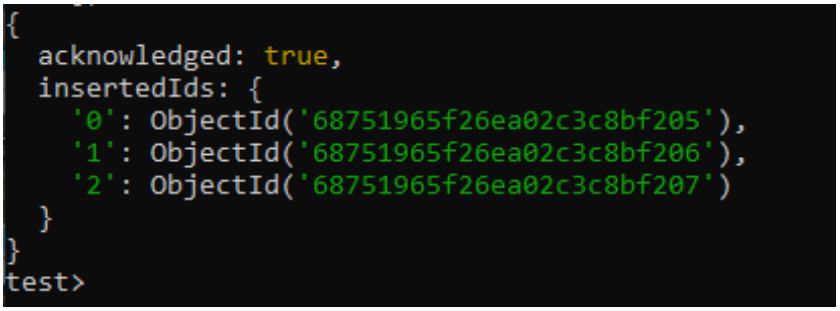


Рис. 45

Теперь смоделируем ситуацию, когда один человек имеет два адреса, а двое других – один адрес (отношение «один-ко-многим»). Вновь создадим коллекцию *address*, заполним ее тремя документами и зафиксируем идентификаторы объектов, присвоенные системой (рис. 46).



Рис. 46

Результат фиксации идентификаторов объекта представлен на рис. 47.

```
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('6875232df26ea02c3c8bf208'),
    '1': ObjectId('6875232df26ea02c3c8bf209'),
    '2': ObjectId('6875232df26ea02c3c8bf20a')
  }
}
test>
```

Рис. 47

Затем обновим коллекцию *person* и также заполним ее тремя документами (рис. 48).

```
test> db.person.insertMany([
...   { "first_name": "Иван",
...     "last_name": "Ипатьев",
...     "dob": ISODate("2010-10-30"),
...     "addresses":
...      [ObjectId("6875232df26ea02c3c8bf208"),
...       ObjectId("6875232df26ea02c3c8bf209")]
...   },
...   { "first_name": "Ольга",
...     "last_name": "Птицына",
...     "dob": ISODate("2008-12-04"),
...     "addresses":
...      [ObjectId("6875232df26ea02c3c8bf20a")]
...   },
...   { "first_name": "Андрей",
...     "last_name": "Растеряев",
...     "dob": ISODate("2011-03-03"),
...     "addresses":
...      [ObjectId("6875232df26ea02c3c8bf20a")] }
... ])
```

Рис. 48

Результат обновления коллекции *person* представлен на рис. 49.

```
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('68752618f26ea02c3c8bf20b'),
    '1': ObjectId('68752618f26ea02c3c8bf20c'),
    '2': ObjectId('68752618f26ea02c3c8bf20d')
  }
}
test>
```

Рис. 49

Чтобы преобразовать связь «один-ко-многим» в связь «многие-ко-многим», достаточно сохранить массивы на обоих концах отношения, т. е. в коллекциях *person* и *address*.

Преимущества использования ссылок по идентификатору *ObjectId* заключаются в большой гибкости и отсутствии дублирования данных. Недостатком является необходимость дополнительных запросов.

Рекомендуется использовать ссылки в следующих случаях:

- отношения «один-ко-многим» и «многие-ко-многим»;
- превышение предельного размера документа;
- частое обновление данных.

Гибридный подход предполагает денормализацию (дублирование часто используемых данных). Примером могут служить те же коллекции *address* и *person*, но коллекция *person* содержит частичную (чаще всего используемую) информацию об адресах (название города) (рис. 50).

```
test> db.person.insertMany([
... { "first_name": "Иван",
...   "last_name": "Ипатьев",
...   "dob": ISODate("2010-10-30"),
...   "addresses": [
...     {"id": ObjectId("6875232df26ea02c3c8bf208"), "city": "Тюмень"},
...     {"id": ObjectId("6875232df26ea02c3c8bf209"), "city": "Кемерово"} ]
... },
... { "first_name": "Ольга",
...   "last_name": "Птицына",
...   "dob": ISODate("2008-12-04"),
...   "addresses": [
...     {"id": ObjectId("6875232df26ea02c3c8bf20a"), "city": "Кемерово"} ]
... },
... { "first_name": "Андрей",
...   "last_name": "Растеряев",
...   "dob": ISODate("2011-03-03"),
...   "addresses": [
...     {"id": ObjectId("6875232df26ea02c3c8bf20a"), "city": "Кемерово"}] }
... ])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('688b51970c5d05b8be8bf202'),
    '1': ObjectId('688b51970c5d05b8be8bf203'),
    '2': ObjectId('688b51970c5d05b8be8bf204')
  }
}
test>
```

Рис. 50

Денормализация рекомендуется, когда часто нужны только некоторые поля или требуется высокая производительность чтения.

Переход от реляционной базы данных к документной

Моделирование данных в документной БД менее детерминировано, чем в реляционной: не существует эквивалента третьей нормальной формы, определяющей «правильную» модель. Кроме того, моделирование в реляционной БД определяется в первую очередь характером хранимых данных, а в документной гораздо большее значение имеет характер выполняемых запросов.

В качестве примера рассмотрим реляционную БД фильмов и актеров. Актеры и фильмы представлены в отдельных таблицах, а таблица связи показывает, какие актеры в каких фильмах снимались (рис. 51).

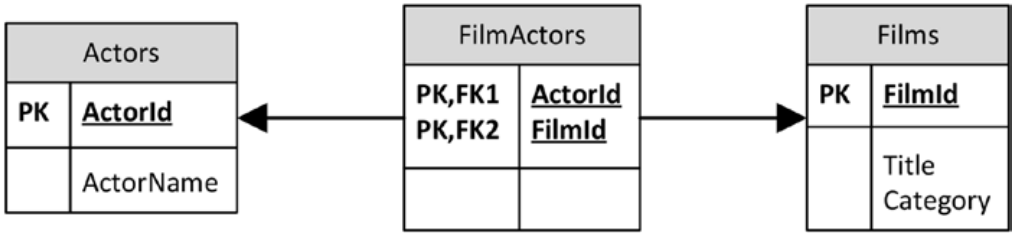


Рис. 51

В документной БД можно создать три коллекции с парами «поле-значение», соответствующими столбцам реляционной схемы, но это нерационально. Документные БД обычно моделируют данные, используя меньшее количество коллекций со встроенными документами (рис. 52).

```
test> db.film.insertMany( [
...    { "_id": 1,
...      "title": "По щучьему велению",
...      "category": "фэнтези",
...      "actors":
...          [ { "_id": 1, "name": "Никита Кологривый" },
...            { "_id": 2, "name": "Мила Ершова" },
...            { "_id": 3, "name": "Алина Алексеева" }
...          ] },
...    { "_id": 2,
...      "title": "Вызов",
...      "category": "драма",
...      "actors":
...          [ { "_id": 4, "name": "Юлия Пересильд" },
...            { "_id": 5, "name": "Милош Бикович" },
...            { "_id": 6, "name": "Владимир Машков" }
...          ] }
...    ])
{ acknowledged: true, insertedIds: { '0': 1, '1': 2 } }
test>
```

Рис. 52

В документы *film* вложены массивы *actors*, что позволяет извлечь фильм и занятых в нем актеров за одну операцию. Одна из проблем – дублирование актеров в нескольких фильмах, что вызывает трудности с изменением каких-либо атрибутов у такого актера. Выше уже упоминалось о проблемах, связанных с увеличением количества встроенных документов.

Вместо этого можно выбрать связывание документов с использованием их идентификаторов (подобно тому, как в реляционной БД связываются строки с помощью внешних ключей). В этом случае в документы, соответствующие фильмам, вставляются массивы идентификаторов актеров (рис. 53).

```
test> db.actor.insertMany( [
...       { "_id": 1, "name": "Никита Кологривый" },
...       { "_id": 2, "name": "Мила Ершова" },
...       { "_id": 3, "name": "Алина Алексеева" },
...       { "_id": 4, "name": "Юлия Пересильд" },
...       { "_id": 5, "name": "Милош Бикович" },
...       { "_id": 6, "name": "Владимир Машков" }
...    ] )
{
  acknowledged: true,
  insertedIds: { '0': 1, '1': 2, '2': 3, '3': 4, '4': 5, '5': 6 }
}
test>
```

Рис. 53

Результат связывания документов представлен на рис. 54.

```
test> db.film.insertMany( [
...    { "_id": 1,
...      "title": "По щучьему велению",
...      "category": "фэнтези",
...      "actors": [1, 2, 3]
...    },
...    { "_id": 2,
...      "title": "Вызов",
...      "category": "драма",
...      "actors": [4, 5, 6 ] }
...    ] )
{ acknowledged: true, insertedIds: { '0': 1, '1': 2 } }
test>
```

Рис. 54

Выполнение работы

Расширить документную БД, созданную в лабораторной работе № 3, дополнительными коллекциями и создать связи с этими коллекциями путем использования ссылок.

Содержание отчета

1. Номер и название лабораторной работы.
2. Цели лабораторной работы.
3. Снимки экранных форм, показывающие порядок выполнения лабораторной работы.
4. Результаты лабораторной работы.

Лабораторная работа № 5
ПОИСК, СОРТИРОВКА И ОБНОВЛЕНИЕ ДАННЫХ В MONGODB

Цель работы: освоить формирование простых и сложных запросов с использованием методов поиска, сортировки и обновления данных, а также применение логических операторов (модификаторов).

Теоретические сведения

Простые запросы в MongoDB

Основной способ получения документов из MongoDB – вызов метода *find()* для рассматриваемой коллекции (аналог оператора SELECT в языке SQL). Он возвращает в виде коллекции массив документов. Если найденных документов нет, метод вернет пустую коллекцию. Синтаксис:

```
db.<коллекция>.find(<критерий>, <поля>),
```

где *<критерий>* – критерий отбора;

<поля> – перечень полей, которые будут возвращены в результате обработки запроса, и поле *_id*, если оно не указано.

Если метод вызывается без параметров, то он вернет все документы коллекции. Получим все документы коллекции *actor* (рис. 55).

```
test> db.actor.find()
[
  { _id: 1, name: 'Никита Кологривый' },
  { _id: 2, name: 'Мила Ершова' },
  { _id: 3, name: 'Алина Алексеева' },
  { _id: 4, name: 'Юлия Пересильд' },
  { _id: 5, name: 'Милош Бикович' },
  { _id: 6, name: 'Владимир Машков' }
]
test>
```

Рис. 55

Метод *limit()* позволяет ограничить количество документов, получаемых в результате запроса. Синтаксис:

```
limit(<количество документов>)
```

Для получения первых трех документов коллекции *actor* необходимо вызвать метод с параметром 3 (рис. 56).

```
test> db.actor.find().limit(3)
[
  { _id: 1, name: 'Никита Кологривый' },
  { _id: 2, name: 'Мила Ершова' },
  { _id: 3, name: 'Алина Алексеева' }
]
test>
```

Рис. 56

Метод *skip()* используется для пропуска в результирующей выборке некоторого количества документов. Синтаксис:

```
skip(<количество документов>)
```

Сделаем пропуск первых двух документов коллекции *actor* (рис. 57).

```
test> db.actor.find().skip(2)
[
  { _id: 3, name: 'Алина Алексеева' },
  { _id: 4, name: 'Юлия Пересильд' },
  { _id: 5, name: 'Милош Бикович' },
  { _id: 6, name: 'Владимир Машков' }
]
test>
```

Рис. 57

Методы *limit()* и *skip()* можно использовать для постраничного вывода данных. Например, для постраничного получения записей некоторой коллекции *clients* можно использовать следующий прием:

```
db.clients.find().limit(10)
db.clients.find().limit(10).skip(10)
db.clients.find().limit(10).skip(20) …
```

Метод *countDocuments()* позволяет подсчитать количество документов в результирующей выборке. Синтаксис:

```
countDocuments(<skip/limit>),
```

где <*skip/limit*> – указывает, следует ли учитывать влияние методов *skip()* и *limit()* (по умолчанию игнорируется применение методов *skip()* и *limit()*).

Получим количество документов в результирующей выборке (рис. 58).

```
test> db.actor.find().count()
6
test>
```

Рис. 58

Метод *count()* можно использовать применительно к коллекции, передавая в него критерий отбора из метода *find()* (рис. 59).

```
test> db.actor.count({"name": "Мила Ершова"})
1
test>
```

Рис. 59

ВНИМАНИЕ! Метод *count()* считается устаревшим. Вместо него рекомендуется использовать метод *countDocuments()*, обеспечивающий более точные результаты в сегментированных кластерах и распределенных средах

Подсчитаем количество документов в коллекции *actor*, пропуская при этом три документа (рис. 60).

```
test> db.actor.countDocuments({}, { skip: 3 })
3
test>
```

Рис. 60

Теперь приведем пример запроса с критерием отбора документов. Получим документы коллекции *actor*, в которых поле *name* имеет значение "Владимир Машков" (рис. 61).

```
test> db.actor.find({ "name": "Владимир Машков"})
[ { _id: 6, name: 'Владимир Машков' } ]
test>
```

Рис. 61

Для отображения только одного поля документа для всех документов коллекции следует ввести имя этого поля (рис. 62).

```
test> db.actor.find({}, {"name": 1})
[
  { _id: 1, name: 'Никита Кологривый' },
  { _id: 2, name: 'Мила Ершова' },
  { _id: 3, name: 'Алина Алексеева' },
  { _id: 4, name: 'Юлия Пересильд' },
  { _id: 5, name: 'Милош Бикович' },
  { _id: 6, name: 'Владимир Машков' }
]
test>
```

Рис. 62

ВНИМАНИЕ! Поле идентификатора *_id* возвращается всегда, но при необходимости можно явно его исключить (рис. 63)

```
test> db.actor.find( {}, {"name": 1, "_id": 0} )
[
  { name: 'Никита Кологривый' },
  { name: 'Мила Ершова' },
  { name: 'Алина Алексеева' },
  { name: 'Юлия Пересильд' },
  { name: 'Милош Бикович' },
  { name: 'Владимир Машков' }
]
test>
```

Рис. 63

ВНИМАНИЕ! В запросе нельзя смешивать включения и исключения полей. Можно или включить, или исключить определенные поля. Исключением из этого правила является только поле идентификатора *_id* (рис. 64)

```
test> db.film.find( {}, {"name": 1, "category": 0} )
MongoServerError[Location31254]: Cannot do exclusion on field category
in inclusion projection
test>
```

Рис. 64

Сортировка результатов запроса

Для сортировки используется метод *sort()*. Синтаксис:

`sort(<параметры сортировки>),`

где <*параметры сортировки*> – документ, содержащий поля, по которым будет производиться сортировка на результирующем наборе данных.

Для полей, по которым требуется сортировать, следует указывать 1 в случае сортировки по возрастанию (рис. 65) и –1 – для сортировки по убыванию (рис. 66).

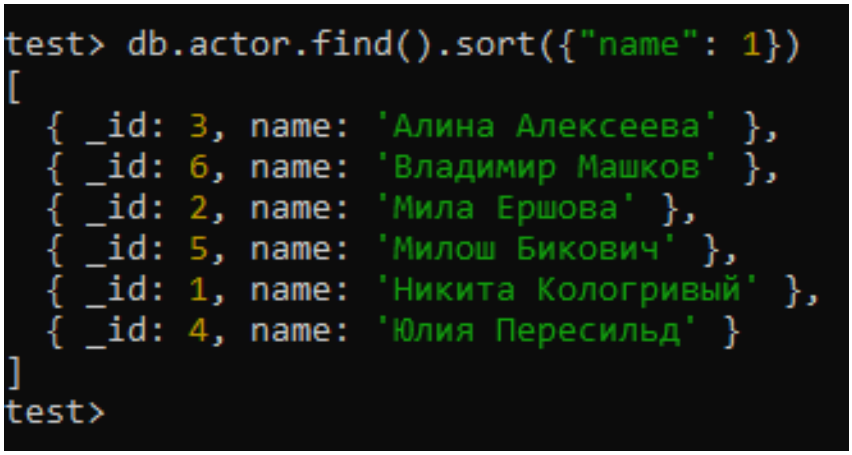


Рис. 65

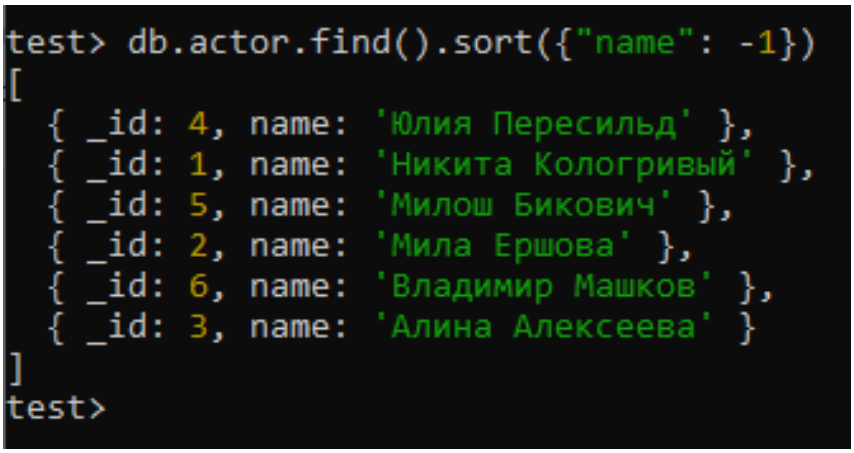


Рис. 66

MongoDB, как и реляционная СУБД, может использовать для сортировки индексы. При этом нужно иметь в виду, что без использования индексов MongoDB ограничивает размер сортируемых данных.

Запросы с условием

Для создания запросов с условием применяются условные операторы (модификаторы):

- *$eq* – равно;
- *$ne* – не равно;
- *$gt* – больше чем;
- *$lt* – меньше чем;

- *$gte* – больше или равно;
- *$lte* – меньше или равно;
- *$in* – вхождение в массив значений;
- *$nin* – определяет массив значений, которые не должно иметь поле.

Модификаторы задают условие, которому должно соответствовать значение поля документа. Синтаксис:

```
{ <поле>: {<модификатор>: <значение>}},
```

где *<поле>* – имя поля, для которого применяется модификатор;

<модификатор> – модификатор;

<значение> – значение модификатора.

Примеры запросов с условием разберем с использованием коллекции *customers* БД *e_shop*. Построим запрос для выбора всех клиентов, у которых сумма бонусных баллов составляет от 200 до 300 (рис. 67).

```
e_shop> db.customers.find({ "loyalty_points": { $gte: 200, $lte: 300 }})
[
  {
    _id: ObjectId('686e77e1db26fb6ae68bf20c'),
    customerId: 4,
    name: 'Медведев Антон Александрович',
    email: 'medvedev@internet.ru',
    city: 'Москва',
    date_of_birth: ISODate('1980-11-27T00:00:00.000Z'),
    loyalty_points: 280,
    purchases: [
      { name: 'Электроника', year: 2022, value: 105000 },
      { name: 'Одежда и обувь', year: 2023, value: 57000 },
      { name: 'Детские товары', year: 2025, value: 35000 }
    ]
  }
]
e_shop>
```

Рис. 67

Получим список всех клиентов, живущих не в Москве (рис. 68).

```
e_shop> db.customers.find({"city": {$ne:"Москва"}}, {"name":1, "_id":0})
[
  { name: 'Баранов Владимир Николаевич' },
  { name: 'Курбанов Денис Игоревич' },
  { name: 'Романова Василиса Викторовна' }
]
```

Рис. 68

Получим список клиентов с идентификаторами 1, 5, 6 (рис. 69).

```
e_shop> db.customers.find({"customerId": {$in: [1, 5, 6]}}, {"name":1, "_id":0})
[
  { name: 'Баранов Владимир Николаевич' },
  { name: 'Петренко Анастасия Максимовна' },
  { name: 'Романова Василиса Викторовна' }
]
```

Рис. 69

Оператор *$nin* действует противоположным образом – определяет массив возможных выражений и ищет те ключи, значение которых отсутствует в этом массиве (рис. 70).

```
e_shop> db.customers.find({"customerId": {$nin: [1, 5, 6]}}, {"name":1, "_id":0})
[
  { name: 'Иванова Ольга Сергеевна' },
  { name: 'Курбанов Денис Игоревич' },
  { name: 'Медведев Антон Александрович' }
]
e_shop>
```

Рис. 70

Логические операторы выполняются над условиями выборки:

- *$or* – соединяет два условия, причем документ должен соответствовать одному из этих условий (является реализацией логической операции ИЛИ);
- *$and* – соединяет два условия, причем документ должен соответствовать обоим условиям (является реализацией логической операции И);

- *$not* – документ должен не соответствовать условию;
- *$nor* – соединяет два условия, причем документ должен не соответствовать обоим условиям (рис. 71).

```
e_shop> db.customers.find({$nor: [{"city":"Москва"}, {"loyalty_points":{$lt: 300}}]})
[
  {
    _id: ObjectId('686e74d5db26fb6ae68bf209'),
    customerId: 1,
    name: 'Баранов Владимир Николаевич',
    email: 'baranov@mail.ru',
    city: 'Новосибирск',
    date_of_birth: ISODate('1975-10-01T00:00:00.000Z'),
    loyalty_points: 340,
    purchases: [
      { name: 'Электроника', year: 2023, value: 60000 },
      { name: 'Одежда и обувь', year: 2021, value: 13000 },
      { name: 'Одежда и обувь', year: 2022, value: 18000 },
      { name: 'Билеты и отели', year: 2021, value: 130000 },
      { name: 'Дом и сад', year: 2024, value: 40000 },
      { name: 'Книги', year: 2024, value: 5500 }
    ]
  }
]
e_shop>
```

Рис. 71

В конце рассмотрим несколько сложных запросов, которые докажут, что связи между коллекциями, реализованные с помощью идентификаторов в лабораторной работе № 4, созданы правильно. Здесь же покажем пример создания вложенных запросов.

В коллекциях *person* и *address* реализована связь «один-к-одному» (рис. 72).

```
test> db.person.find({"_id": db.address.findOne({"city":"Тюмень"}).person_id})
[
  {
    _id: ObjectId('68751742f26ea02c3c8bf202'),
    first_name: 'Иван',
    last_name: 'Ипатьев',
    dob: ISODate('2010-10-30T00:00:00.000Z')
  }
]
test>
```

Рис. 72

Команда выполняет два запроса, вложенных один в другой, первый запрос извлекает значение *person_id*, которое используется для запроса к коллекции *person* по идентификатору *_id*.

ВНИМАНИЕ! В этом запросе следует обязательно использовать метод *findOne()*

В коллекциях *person* и *address* реализована связь «один-ко-многим». Таким образом, можно получить массив элементов *addresses*, а затем выполнить запрос внутри, чтобы получить все документы со стороны «много» (рис. 73, 74).

```
test> person = db.person.findOne({"last_name": "Ипатьев"})
{
  _id: ObjectId('68752618f26ea02c3c8bf20b'),
  first_name: 'Иван',
  last_name: 'Ипатьев',
  dob: ISODate('2010-10-30T00:00:00.000Z'),
  addresses: [
    ObjectId('6875232df26ea02c3c8bf208'),
    ObjectId('6875232df26ea02c3c8bf209')
  ]
}
test>
```

Рис. 73

```
test> addresses = db.address.find({_id: {$in: person.addresses} })
[
  {
    _id: ObjectId('6875232df26ea02c3c8bf208'),
    street: { name: 'улица Комаровская', number: '18' },
    city: 'Тюмень',
    zip: '625032'
  },
  {
    _id: ObjectId('6875232df26ea02c3c8bf209'),
    street: { name: 'улица Автомобилистов', number: '21' },
    city: 'Кемерово',
    zip: '650902'
  }
]
test>
```

Рис. 74

Выполним еще одну проверку связи для этих же коллекций (рис. 75, 76).

```
test> person = db.person.findOne({"last_name": "Птицына"})
{
  _id: ObjectId('68752618f26ea02c3c8bf20c'),
  first_name: 'Ольга',
  last_name: 'Птицына',
  dob: ISODate('2008-12-04T00:00:00.000Z'),
  addresses: [ ObjectId('6875232df26ea02c3c8bf20a') ]
}
test>
```

Рис. 75

```
test> addresses = db.address.find({_id: {$in: person.addresses} })
[
  {
    _id: ObjectId('6875232df26ea02c3c8bf20a'),
    street: { name: 'улица Дружбы', number: '33' },
    city: 'Кемерово',
    zip: '650040'
  }
]
test>
```

Рис. 76

В коллекциях *actor* и *film* реализована связь «один-ко-многим» (рис. 77).

```
test> movie = db.film.findOne({"title": "Вызов"})
{ _id: 2, title: 'Вызов', category: 'драма', actors: [ 4, 5, 6 ] }

test> actors = db.actor.find({"_id": {$in: movie.actors} })
[
  { _id: 4, name: 'Юлия Пересильд' },
  { _id: 5, name: 'Милош Бикович' },
  { _id: 6, name: 'Владимир Машков' }
]
test>
```

Рис. 77

Обновление существующих документов

Часто требуется изменять данные в БД: обновить поле или добавить дополнительные поля к имеющемуся документу. Обновить или заменить документы в коллекции позволяют методы, перечисленные ранее:

- *updateOne()* – обновляет один документ в коллекции на основе предоставленного фильтра;
- *updateMany()* – обновляет несколько документов в коллекции, соответствующих указанному фильтру;
- *replaceOne()* – заменяет весь документ в коллекции на основе предоставленного фильтра.

Для обновления одного документа в коллекции используется метод *updateOne(),* имеющий следующий синтаксис:

```
db.<коллекция>.updateOne(<фильтр>, <обновление>, {<дополнительные параметры>}),
```

где <*фильтр*> – критерии выбора для обновления (обязательный);

<*обновление*> – документ, содержащий изменения, которые необходимо применить (обязательный);

<*дополнительные параметры*> – документ, определяющий необязательные аргументы.

Первый аргумент – документ, задающий условия фильтра, которые будут применяться для выбора документов. Так как метод *updateOne()* изменяет не более одного документа в коллекции, будет использоваться первый документ, удовлетворяющий условиям фильтра.

Второй аргумент – указывает операцию обновления, которую следует выполнить для изменения содержимого соответствующего документа.

Третий аргумент – документ с различными вариантами изменения поведения метода. Наиболее важные из них:

- *upsert* – превращает операцию в процедуру обновления путем вставки нового документа, если фильтр не соответствует ни одному существующему документу;

- *collation* – определяет правила для конкретного языка, которые должны применяться к операции.

Для примеров обновления и замены документов создадим простую демонстрационную коллекцию *student*, содержащую данные о трех студентах (рис. 78).

```
test> db.student.insertMany([
...    { "_id": 1,
...      "name": "Астафьев Владимир",
...      "age": 20
...      },
...    { "_id": 2,
...      "name": "Ипатьев Иван",
...      "age": 19
...      },
...    { "_id": 3,
...      "name": "Громыхало Наталья",
...      "age": 19
...      },
...  ] )
{ acknowledged: true, insertedIds: { '0': 1, '1': 2, '2': 3 } }
test>
```

Рис. 78

Обновим возраст студентки Натальи Громыхало (рис. 79), а затем ее фамилию (рис. 80).

```
test> db.student.updateOne({"name": "Громыхало Наталья"}, {$set:{"age": 20}})
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
test> db.student.findOne({"name": "Громыхало Наталья"})
{ _id: 3, name: 'Громыхало Наталья', age: 20 }
test>
```

Рис. 79

```
test> db.student.updateOne({"name":"Громыхало Наталья"},
... {$set:{"name":"Сидорова Наталья"}})
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
test> db.student.findOne({"_id": 3})
{ _id: 3, name: 'Сидорова Наталья', age: 20 }
test>
```

Рис. 80

Добавим в документ Владимира Астафьева новое поле с именем *curs* и значением 3 (рис. 81).

```
test> db.student.updateOne({"name": "Астафьев Владимир"},
... {$set: {"curs": 3}})
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
test> db.student.findOne({"_id": 1})
{ _id: 1, name: 'Астафьев Владимир', age: 20, curs: 3 }
test>
```

Рис. 81

Обновим все документы, где указан возраст 20, установив его равным 21 (рис. 82).

```
test> db.student.updateMany({ "age": 20 },
... { $set: {"age": 21 }})
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 2,
  modifiedCount: 2,
  upsertedCount: 0
}
test> db.student.find()
[
  { _id: 1, name: 'Астафьев Владимир', age: 21, curs: 3 },
  { _id: 2, name: 'Ипатьев Иван', age: 19 },
  { _id: 3, name: 'Сидорова Наталья', age: 21 }
]
test>
```

Рис. 82

Обновим документ с именем "Дроздова Мария", если он существует, в противном случае вставим новый документ (рис. 83, 84).

```
test> db.student.updateOne({ "name": "Скворцова Мария"},
... { $set: { "age": 20 }}, { upsert: true })
{
  acknowledged: true,
  insertedId: ObjectId('687bc7e60ffafb65b1b12148'),
  matchedCount: 0,
  modifiedCount: 0,
  upsertedCount: 1
}
```

Рис. 83

```
test> db.student.find()
[
  { _id: 1, name: 'Астафьев Владимир', age: 21, curs: 3 },
  { _id: 2, name: 'Ипатьев Иван', age: 19 },
  { _id: 3, name: 'Сидорова Наталья', age: 21 },
  {
    _id: ObjectId('687bc7e60ffafb65b1b12148'),
    name: 'Скворцова Мария',
    age: 20
  }
]
test>
```

Рис. 84

Метод *replaceOne ()* возвращает первый документ, удовлетворяющий критериям фильтра, и полностью заменяет его новым документом (рис. 85). Он не поддерживает атомарные операторы обновления, такие как *$set*. Вместо этого заменяется весь документ. Синтаксис:

`db.replaceOne(<фильтр>, <новый документ>, <опции>, <обратный вызов>),`

где <*фильтр*> – критерии выбора документа, который нужно заменить;

<*новый документ*> – документ, который заменяет найденный;

<*опции*> – необязательные параметры, такие как *upsert;*

<*обратный вызов*> – функция, которая будет выполнена после завершения операции (необязательно).

```
test> db.student.replaceOne({"name": "Скворцова Мария"},
... {"name": "Баева Ольга", "age": 25})
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
test> db.student.find()
[
  { _id: 1, name: 'Астафьев Владимир', age: 21, curs: 3 },
  { _id: 2, name: 'Ипатьев Иван', age: 19 },
  { _id: 3, name: 'Сидорова Наталья', age: 21 },
  {
    _id: ObjectId('687bc7e60ffafb65b1b12148'),
    name: 'Баева Ольга',
    age: 25
  }
]
test>
```

Рис. 85

Выполнение работы

В созданной ранее документной базе данных необходимо:

- выполнить четыре запроса фильтрации с использованием операторов сравнения;
- выполнить два сложных запроса к коллекции документов, содержащих встроенные документы;
- выполнить два сложных запроса к коллекциям, содержащим документы, связанные по ссылкам;
- провести сортировку данных коллекции по одному полю и по дополнительным полям;
- обновить один документ в коллекции на основе фильтра и несколько документов в коллекции, соответствующих указанному фильтру.
- заменить весь документ в коллекции на основе предоставленного фильтра.

Содержание отчета

1. Номер и название лабораторной работы.
2. Цели лабораторной работы.
3. Снимки экранных форм, показывающие порядок выполнения лабораторной работы.
4. Результаты лабораторной работы.

Лабораторная работа № 6
РАБОТА СО ВЛОЖЕННЫМИ СТРУКТУРАМИ И МАССИВАМИ

Цель работы:

– освоить методы работы со вложенными документами и массивами в MongoDB, включая создание, обновление, запросы и агрегацию данных со сложной структурой;

– познакомиться с модификаторами массивов.

ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ

MongoDB позволяет хранить данные в виде документов со вложенными структурами:

- вложенные документы;
- массивы значений;
- массивы документов.

В паре «поле-значение» в качестве значения может использоваться документ, он называется вложенным. MongoDB поддерживает вложенные документы (*embedded documents*), что позволяет хранить в одном документе сложные иерархические данные. Это одна из ключевых особенностей документоориентированных БД.

Основные возможности вложенных документов в MongoDB:

- они содержатся внутри других документов, при этом поддерживается до 100 уровней вложенности (нужно учитывать, что сложные запросы к глубоко вложенным структурам могут быть менее эффективными);
- все связанные данные извлекаются за одну операцию, что обеспечивает высокую производительность;
- связанные данные хранятся вместе, за счет чего обеспечивается их целостность (при этом максимальный размер документа MongoDB, включая все вложения, равен 16 Мб);
- каждый вложенный документ может иметь свою структуру, ее изменение возможно без изменения схемы, чем достигается гибкость.

Вложенные документы обычно используют в следующих ситуациях:

- данные имеют отношение «содержит»;
- вложенные данные часто запрашиваются вместе с родительским документом;
- вложенные данные имеют смысл только в контексте родительского документа.

Запросы к вложенным документам выполняются с использованием точечной нотации (*dot notation*). Например, для коллекции *address* найдем все адреса по улице Автомобилистов (рис. 86).

```
test> db.address.find({"street.name": "улица Автомобилистов"})
[
  {
    _id: ObjectId('6875232df26ea02c3c8bf209'),
    street: { name: 'улица Автомобилистов', number: '21' },
    city: 'Кемерово',
    zip: '650902'
  }
]
test>
```

Рис. 86

Переименуем дом 21 по улице Автомобилистов в дом 21а (рис. 87).

```
test> db.address.updateOne(
...   {"_id": ObjectId("6875232df26ea02c3c8bf209")},
...   {$set: {"street.number": "21a"}}
... )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
test>
```

Рис. 87

Одним из самых мощных типов данных в MongoDB является массив. Это тип данных, который может содержать множество значений, включая

другие документы, массивы и смесь различных типов данных. Массивы позволяют пользователям хранить в документах упорядоченные данные. Такая универсальность делает их краеугольным камнем моделирования данных.

В качестве примера рассмотрим коллекцию *student*, которая содержит следующие данные о трех учениках: имя и фамилию, возраст, хобби и оценки по разным предметам. В структуре данных два массива: массив строковых значений *hobbies* и массив документов *scores* (рис. 88).

```
test> db.student.insertMany( [
...    { "name": "Иван Петров",
...      "age": 15,
...      "hobbies": ["чтение", "плавание"],
...      "scores": [
...        {"subject": "математика", "score": 4},
...        {"subject": "физика", "score": 3}] },
...    { "name": "Ольга Сергеева",
...      "age": 14,
...      "hobbies": ["рисование", "музыка", "путешествия"],
...      "scores": [
...        {"subject": "математика", "score": 5},
...        {"subject": "литература", "score": 5}] },
...    { "name": "Татьяна Николаева",
...      "age": 15,
...      "hobbies": ["программирование", "шахматы"],
...      "scores": [
...        {"subject": "информатика", "score": 4},
...        {"subject": "математика", "score": 4}] }
...    ] )
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('687e39c30465e09ef88bf205'),
    '1': ObjectId('687e39c30465e09ef88bf206'),
    '2': ObjectId('687e39c30465e09ef88bf207')
  }
}
test>
```

Рис. 88

Запросы к массивам

Имеются различные методы для запросов к элементам массива. Точечная нотация обеспечивает доступ к определенному элементу массива по его индексу. Синтаксис:

```
db.<коллекция>.find({ "<массив>.<индекс>": <значение> })
```

Найдем учащихся, имеющих оценки по информатике (рис. 89).

```
test> db.student.find({ "scores.subject": "информатика" })
[
  {
    _id: ObjectId('687e39c30465e09ef88bf207'),
    name: 'Татьяна Николаева',
    age: 15,
    hobbies: [ 'программирование', 'шахматы' ],
    scores: [
      { subject: 'информатика', score: 4 },
      { subject: 'математика', score: 4 }
    ]
  }
]
test>
```

Рис. 89

Можно в запрос добавить условие: например, найти учащихся с оценкой «4» по математике (рис. 90).

```
test> db.student.find({"scores.subject": "математика",
... "scores.score": {$eq: 4}})
[
  {
    _id: ObjectId('687e39c30465e09ef88bf205'),
    name: 'Иван Петров',
    age: 15,
    hobbies: [ 'чтение', 'плавание' ],
    scores: [
      { subject: 'математика', score: 4 },
      { subject: 'физика', score: 3 }
    ]
  },
  {
    _id: ObjectId('687e39c30465e09ef88bf207'),
    name: 'Татьяна Николаева',
    age: 15,
    hobbies: [ 'программирование', 'шахматы' ],
    scores: [
      { subject: 'информатика', score: 4 },
      { subject: 'математика', score: 4 }
    ]
  }
]
test>
```

Рис. 90

$elemMatch – оператор запроса, сопоставляющий документы, в которых хотя бы один элемент массива удовлетворяет нескольким условиям (рис. 91):

```
db.<коллекция>.find({ <массив>: { $elemMatch: { <условие> } } })
```

```
test> db.student.find({"scores":
... { $elemMatch:
... { "subject": "математика", "score": {$eq: 4}}}
... })
[
  {
    _id: ObjectId('687e39c30465e09ef88bf205'),
    name: 'Иван Петров',
    age: 15,
    hobbies: [ 'чтение', 'плавание' ],
    scores: [
      { subject: 'математика', score: 4 },
      { subject: 'физика', score: 3 }
    ]
  },
  {
    _id: ObjectId('687e39c30465e09ef88bf207'),
    name: 'Татьяна Николаева',
    age: 15,
    hobbies: [ 'программирование', 'шахматы' ],
    scores: [
      { subject: 'информатика', score: 4 },
      { subject: 'математика', score: 4 }
    ]
  }
]
test>
```

Рис. 91

$slice – оператор запроса, ограничивающий количество возвращаемых элементов массива:

```
db.<коллекция>.find({}, { <массив>: { $slice: n } })
```

Выведем только по одному значению хобби для каждого учащегося. Кроме хобби выведем только идентификатор и имя учащегося (рис. 92).

```
test> db.student.find({}, { "name": 1, "hobbies": { $slice: 1}})
[
  {
    _id: ObjectId('687e39c30465e09ef88bf205'),
    name: 'Иван Петров',
    hobbies: [ 'чтение' ]
  },
  {
    _id: ObjectId('687e39c30465e09ef88bf206'),
    name: 'Ольга Сергеева',
    hobbies: [ 'рисование' ]
  },
  {
    _id: ObjectId('687e39c30465e09ef88bf207'),
    name: 'Татьяна Николаева',
    hobbies: [ 'программирование' ]
  }
]
test>
```

Рис. 92

$in – оператор запроса, позволяющий найти документы, в которых хотя бы один элемент массива соответствует указанным значениям:

`db.<коллекция>.find({ <массив>: { $in: [значение1, значение2] } })`

Найдем учащихся, у которых есть оценки или по информатике, или по литературе (рис. 93).



Рис. 93

$all – оператор запроса, позволяющий найти документы, в которых все указанные значения существуют в массиве:

```
db.<коллекция>.find({ <массив>: { $all: [<значение1>, <значение2>] } })
```

Найдем учащихся, у которых есть оценки и по информатике, и по литературе (таких нет) (рис. 94).

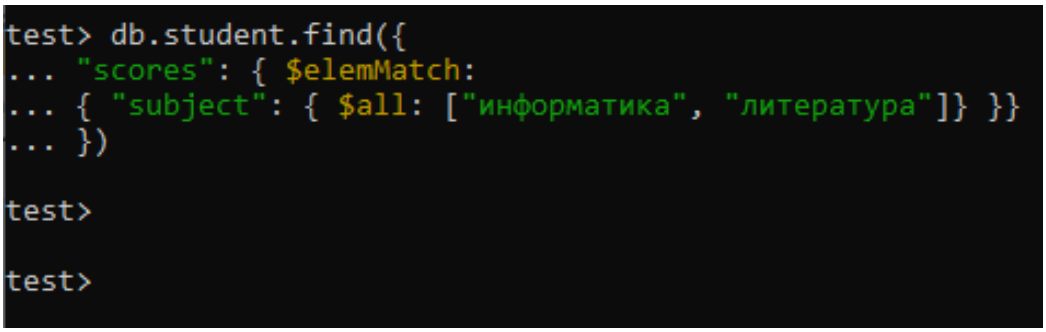


Рис. 94

$unwind – оператор запроса, позволяющий деконструировать массив, чтобы вернуть каждый его элемент как отдельный документ:

```
db.<коллекция>.aggregate([{ $unwind: "$<массив>" }])
```

Этот оператор используется во фреймворке агрегации и будет подробно разобран в одной из следующих лабораторных работ.

Модификаторы массивов в запросах

Для добавления элемента в массив используется модификатор *$push* как параметр метода *update()*. Синтаксис:

```
{ $push: { <массив>: <значение> },
```

где <*массив*> – массив, в который добавляется элемент;

<*значение*> – значение добавляемого элемента.

На рис. 95 приведен пример использования модификатора.

```
test> db.student.updateOne({ "name": "Иван Петров" },
... { $push: { "hobbies": "рыбалка" } })
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
test>
```

Рис. 95

Этот запрос добавит в массив *hobbies* учащегося Ивана Петрова элемент "рыбалка". Если такой массив отсутствует, то он будет создан. Если обновляемое поле не является массивом, то в результате выполнения метода возникнет ошибка. Если значение является массивом, *$push* добавляет весь массив как отдельный элемент. Чтобы добавить несколько элементов, следует использовать модификатор *$each*.

Рассмотрим добавление значений из массива *["музыка", "кино"]* как отдельных элементов в массив *hobbies* (рис. 96).

```
test> db.student.updateOne({ "name": "Татьяна Николаева" },
... {$push: {"hobbies": {$each: ["музыка", "кино"]}}})
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
test>
```

Рис. 96

Модификатор *$push* добавляет элементы в массив, не проверяя их на уникальность. Чтобы добавить в массив только уникальные элементы, используется модификатор *$addToSet*. Синтаксис этого модификатора аналогичен синтаксису *$push*.

Для удаления элемента из массива используется модификатор *$pop* (рис. 97). Синтаксис:

```
{ $pop: { <поле>: <параметр удаления> } },
```

где <*массив*> – имя массива;

<*параметр удаления*> – 1, если требуется удалить последний элемент в массиве, –1, если требуется удалить первый элемент в массиве.

```
test> db.student.updateOne({ "name": "Татьяна Николаева" },
... { $pop: { "hobbies": 1 } })
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
test>
```

Рис. 97

Для удаления элемента массива по определенному критерию используется модификатор *$pull*. Синтаксис:

```
{ $pull: { <массив>: <запрос> } },
```

где <*массив*> – имя массива;

<*запрос*> – запрос для поиска требуемого элемента.

Например, удалим элемент "музыка" из массива *hobbies* (рис. 98).

```
test> db.student.updateOne({ "name": "Татьяна Николаева" },
... { $pull: { "hobbies": "музыка" } })
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
test>
```

Рис. 98

Позиционные модификаторы массивов

Для доступа к конкретному элементу массива используют два способа:

- по конкретной позиции элемента;
- с помощью позиционного оператора ($).

В качестве примера рассмотрим коллекцию из двух документов, содержащих массивы числовых данных целого типа (рис. 99).

```
test> db.product.insertMany([
...   { "name": "m12896", "cost": [1000, 2000, 3000] },
...   { "name": "m12900", "cost": [4000, 5000, 6000] }
... ])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('687f5fbc55fa0cf8768bf207'),
    '1': ObjectId('687f5fbc55fa0cf8768bf208')
  }
}
test>
```

Рис. 99

ВНИМАНИЕ! Не используйте коллекции, содержащие один документ. При работе с ними возникают проблемы

Для доступа к элементу массива по номеру после запроса на выборку документа в качестве имени изменяемого поля используется:

`<массив>.<№ элемента>`

Увеличим нулевой элемент массива на 200 (рис. 100).

```
test> db.product.updateOne({ "name": "m12896" },
... {$inc: {"cost.0": 200}})
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
test>
```

Рис. 100

Убедимся в правильности внесенных изменений (рис. 101).

```
test> db.product.find({"name": "m12896"})
[
  {
    _id: ObjectId('687f5fbc55fa0cf8768bf207'),
    name: 'm12896',
    cost: [ 1200, 2000, 3000 ]
  }
]
test>
```

Рис. 101

Если необходимо изменить элемент массива, не зная его позиции в массиве, его можно отыскать с помощью запроса, и применить нужный модификатор. В качестве имени изменяемого поля будет выступать:

`<массив>.<символ $>`

Увеличим элемент массива, равный 2000, на 100 (рис. 102).

```
test> db.product.updateOne({ "cost": 2000 }, {$inc: {"cost.$": 100}})
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
test>
```

Рис. 102

Убедимся в правильности внесенных изменений (рис. 103).

```
test> db.product.find({"name": "m12896"})
[
  {
    _id: ObjectId('687f5fbc55fa0cf8768bf207'),
    name: 'm12896',
    cost: [ 1200, 2100, 3000 ]
  }
]
test>
```

Рис. 103

Увеличим все элементы массива на 1 (рис. 104).

```
test> db.product.updateOne({ "name": "m12896" },
... { $inc: { "cost.$[]": 1 }})
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
test>
```

Рис. 104

Выполнение работы

С помощью модификаторов *$push, $pop, $pull, $inc, $all, $slice* в документной базе данных необходимо:

- создать несколько запросов для вставки данных в массив;
- создать запросы, производящие обновление данных в массиве (по позиции элемента в массиве и по его значению);
- создать запросы, удаляющие элементы из массива (по позиции элемента в массиве и по его значению);
- составить отчет.

Содержание отчета

1. Номер и название лабораторной работы.
2. Цели лабораторной работы.
3. Снимки экранных форм, показывающие порядок выполнения лабораторной работы.
4. Результаты лабораторной работы.

Лабораторная работа № 7
ФРЕЙМВОРК АГРЕГАЦИИ В MONGODB

Цель работы: изучить возможности фреймворка агрегации MongoDB для выполнения сложных операций обработки данных, включая фильтрацию, группировку, сортировку и преобразование данных.

Теоретические сведения

Доступ к данным в MongoDB осуществляется с помощью языка запросов MongoDB (MQL). При этом по мере масштабирования базы данных возникают все более сложные запросы. Фреймворк агрегации MongoDB – мощный инструмент обработки данных с использованием конвейера операций. Это способ реализации запросов, позволяющий разбить сложные запросы на шаги. Сложная логика разделяется на последовательные операции (рис. 105).



Рис. 105

Каждый этап принимает в качестве входных данных то, что получено на предыдущем в качестве выходных данных. Входными и выходными данными для всех этапов являются документы.

В конце конвейера осуществляется вывод, как если бы был выполнен запрос на поиск. Полученный поток документов можно использовать для решения каких-либо задач (создание отчета, веб-сайта и др.).

Отдельный этап конвейера принимает поток входных документов по одному, обрабатывает каждый из них по одному и создает выходной поток документов. Каждый этап предоставляет набор настраиваемых параметров, которыми можно управлять. Этап выполняет обобщенную задачу. Его параметризируют для определенной коллекции, и конкретно для того, что он должен делать с документами. Настраиваемые параметры имеют форму операторов, которые будут изменять поля, выполнять операции и т. д.

Можно включать в конвейер один и тот же тип этапа несколько раз, например, выполнять фильтрацию. Тогда после дополнительной обработки вновь осуществляется фильтрация с другим набором критериев.

Конвейеры агрегации работают с методом *aggregate()*. Синтаксис:

```
db.<коллекция>.aggregate()
```

Этапы (*операции*) *агрегации*

В MongoDB доступно около 40 этапов агрегации (весь список можно найти в официальной документации). Рассмотрим следующие из них:

- *$project* – изменяет форму каждого документа в потоке, например, добавляя новые поля или удаляя существующие. Для каждого входного документа выводится один документ;
- *$match* – фильтрует поток документов, позволяя только совпадающим документам проходить без изменений на следующий этап. Использует стандартные запросы MongoDB. Для каждого входного документа выводится либо один документ (совпадение), либо ноль документов (нет совпадения);
- *$group* – группирует входные документы по указанному идентификатору и применяет к каждой группе аккумуляторы, если они указаны. Потребляет все входные документы и выводит по одному документу для каждой группы. Выходные документы содержат только поле идентификатора и, если указано, накопительные поля;
- *$sort* – изменяет порядок потока документов по указанному ключу сортировки. Меняется только порядок, документы остаются неизменными. Для каждого входного документа выводится один документ;
- *$skip* – пропускает первые *n* документов с указанным номером пропуска и передает оставшиеся в конвейер без изменений. Для каждого входного документа выводит либо ноль документов (для первых *n* документов), либо один документ (если после первых *n* документов);
- *$limit* – передает первые *n* неизмененных документов в конвейер, где *n* – указанный предел. Для каждого входного документа выводится либо один документ (для первых *n* документов), либо ноль документов (после первых *n* документов);

- *$unwind* – деконструирует поле массива из входных документов для вывода документа для каждого элемента. Для каждого входного документа выводятся выходные *n* документов, где *n* – количество элементов массива, которое для пустого массива может быть равно нулю;
- *$out* – заключительный этап конвейера агрегации, на котором данные выводятся в выходную коллекцию (она будет полностью удалена и перезаписана, если уже существует). Не может выполнить вывод в сегментированную коллекцию;
- *$merge* – завершающий этап конвейера агрегации, может вставлять, объединять, заменять или сохранять существующие документы. Обрабатывает документы с помощью пользовательского конвейера обновления. Может заменить все документы в коллекции (только если выходные данные результатов агрегирования соответствуют всем существующим документам в коллекции), выводить данные в сегментированную коллекцию.

ВНИМАНИЕ! Этапы *$out* и *$merge* (а также *$geoNear*) могут появиться в конвейере агрегации только один раз

Рассмотрим использование фреймворка агрегации на примере БД *e_shop*, содержащей одну коллекцию *customers*, которая представляет собой список клиентов интернет-магазина из разных городов страны.

Проведем простую операцию агрегации в MongoDB. В списке покупателей нужно выбрать москвичей, упорядочить их по бонусным баллам и отобразить только имя и бонусный балл. Эта агрегация состоит из трех этапов:

- *$match:* – фильтрация только совпадающих документов. В примере нужно отфильтровать людей, имеющих в поле *city* значение "Москва";
- *$sort:* – сортировка по возрастанию по полю *loyalty_points*;
- *$project:* – выбор полей для отображения (1 – для отображения, 0 – для сокрытия).

Позиции этапов в массиве агрегации определяют порядок работы. Это означает, что в приведенном примере сначала запускается *$match*, затем *$sort* и, наконец, *$project* (рис. 106).

```
e_shop> db.customers.aggregate([
... { $match: { "city": "Москва" }},
... { $sort: { "loyalty_points": 1 }},
... { $project: { "_id": 0, "name": 1, "loyalty_points": 1 }}
... ])
[
  { name: 'Иванова Ольга Сергеевна', loyalty_points: 80 },
  { name: 'Петренко Анастасия Максимовна', loyalty_points: 120 },
  { name: 'Медведев Антон Александрович', loyalty_points: 280 }
]
e_shop>
```

Рис. 106

Можно использовать *$match* как *find()*. В следующем примере *$match* использован для фильтрации покупателей только из Санкт-Петербурга (рис. 107).

```
e_shop> db.customers.aggregate([
... { $match: {"city": "Санкт-Петербург" }},
... { $project: { "_id": 0, "purchases": 0 }}
... ])
[
  {
    customerId: 3,
    name: 'Курбанов Денис Игоревич',
    email: 'kurbanov@internet.ru',
    city: 'Санкт-Петербург',
    date_of_birth: ISODate('1966-07-21T00:00:00.000Z'),
    loyalty_points: 120
  },
  {
    customerId: 6,
    name: 'Романова Василиса Викторовна',
    email: 'romanova@inbox.ru',
    city: 'Санкт-Петербург',
    date_of_birth: ISODate('2003-10-01T00:00:00.000Z'),
    loyalty_points: 100
  }
]
e_shop>
```

Рис. 107

Выполним фильтрацию документов, в которые вложены покупки, сделанные позже 2024 года. Результат неожиданный. Фильтр по-прежнему выводит полные документы с покупками до 2024 года (если такие есть) (рис. 108). Единственным требованием является наличие в документе хотя бы одной покупки после 2024 года.

```
e_shop> db.customers.aggregate([
... { $match: { "purchases.year": { $gt: 2024}}},
... { $project: { "_id": 0, "purchases": 1, "name": 1 }}
... ])
[
  {
    name: 'Иванова Ольга Сергеевна',
    purchases: [
      { name: 'Электроника', year: 2021, value: 50000 },
      { name: 'Электроника', year: 2025, value: 14000 },
      { name: 'Одежда и обувь', year: 2022, value: 45000 },
      { name: 'Одежда и обувь', year: 2025, value: 36000 }
    ]
  },
  {
    name: 'Медведев Антон Александрович',
    purchases: [
      { name: 'Электроника', year: 2022, value: 105000 },
      { name: 'Одежда и обувь', year: 2023, value: 57000 },
      { name: 'Детские товары', year: 2025, value: 35000 }
    ]
  },
  {
    name: 'Петренко Анастасия Максимовна',
    purchases: [
      { name: 'Путешествия', year: 2021, value: 150000 },
      { name: 'Одежда и обувь', year: 2023, value: 75000 },
      { name: 'Детские товары', year: 2024, value: 14500 },
      { name: 'Детские товары', year: 2025, value: 3500 }
    ]
  },
  {
    name: 'Романова Василиса Викторовна',
    purchases: [
      { name: 'Дом и сад', year: 2021, value: 25000 },
      { name: 'Дом и сад', year: 2022, value: 54000 },
      { name: 'Дом и сад', year: 2023, value: 35000 },
      { name: 'Билеты и отели', year: 2024, value: 130000 },
      { name: 'Путешествия', year: 2025, value: 60000 }
    ]
  }
]
e_shop>
```

Рис. 108

$project используют для изменения полей, создания новых, а также для отображения и сокрытия полей. Отобразим только фамилию, имя, отчество покупателя и его идентификатор (рис. 109). Для показываемых полей установлено значение 1. Следует явно скрыть поле *_id*. Поля скрыты, если для них нет настроек.

```
e_shop> db.customers.aggregate([
... { $project: { "_id": 0, "customerId": 1, "name": 1 }}
... ])
[
  { customerId: 1, name: 'Баранов Владимир Николаевич' },
  { customerId: 2, name: 'Иванова Ольга Сергеевна' },
  { customerId: 3, name: 'Курбанов Денис Игоревич' },
  { customerId: 4, name: 'Медведев Антон Александрович' },
  { customerId: 5, name: 'Петренко Анастасия Максимовна' },
  { customerId: 6, name: 'Романова Василиса Викторовна' }
]
e_shop>
```

Рис. 109

Можно использовать *$project* для добавления нового вычисляемого поля в документ. Например, перевести бонусные баллы в рубли, приравняв каждый балл к 10 рублям, и отобразить это новое поле в документе (рис. 110).

```
e_shop> db.customers.aggregate([
... { $project: {"_id": 0, "name": 1, "loyalty_points": 1,
... "calculated_money": {$multiply: [ "$loyalty_points", 10]}}}
... ])
[
  {
    name: 'Баранов Владимир Николаевич',
    loyalty_points: 340,
    calculated_money: 3400
  },
  {
    name: 'Иванова Ольга Сергеевна',
    loyalty_points: 80,
    calculated_money: 800
  },
  {
    name: 'Курбанов Денис Игоревич',
    loyalty_points: 120,
    calculated_money: 1200
  },
```

Рис. 110 (окончание см. на с. 93)

```
  {
    name: 'Медведев Антон Александрович',
    loyalty_points: 280,
    calculated_money: 2800
  },
  {
    name: 'Петренко Анастасия Максимовна',
    loyalty_points: 120,
    calculated_money: 1200
  },
  {
    name: 'Романова Василиса Викторовна',
    loyalty_points: 100,
    calculated_money: 1000
  }
]
e_shop>
```

Рис. 110. Окончание (начало см. на с. 92)

С помощью *$unwind* можно показать только покупки в категории "Книги", сделанные всеми покупателями. В результате есть один документ для каждого упоминания этой категории (рис. 111).

```
e_shop> db.customers.aggregate([
... { $unwind: "$purchases" },
... { $match: { "purchases.name": "Книги" }}
... ])
[
  {
    _id: ObjectId('686e74d5db26fb6ae68bf209'),
    customerId: 1,
    name: 'Баранов Владимир Николаевич',
    email: 'baranov@mail.ru',
    city: 'Новосибирск',
    date_of_birth: ISODate('1975-10-01T00:00:00.000Z'),
    loyalty_points: 340,
    purchases: { name: 'Книги', year: 2024, value: 5500 }
  },
  {
    _id: ObjectId('686e7646db26fb6ae68bf20b'),
    customerId: 3,
    name: 'Курбанов Денис Игоревич',
    email: 'kurbanov@internet.ru',
    city: 'Санкт-Петербург',
    date_of_birth: ISODate('1966-07-21T00:00:00.000Z'),
    loyalty_points: 120,
    purchases: { name: 'Книги', year: 2023, value: 3500 }
  }
]
e_shop>
```

Рис. 111

Если запустить только этап *$unwind*, то получим разделение исходных документов на документы, каждый из которых содержит только данные о покупках в одной категории (для экономии места вывод оборван) (рис. 112).

```
e_shop> db.customers.aggregate([{ $unwind: "$purchases" }])
[
  {
    _id: ObjectId('686e74d5db26fb6ae68bf209'),
    customerId: 1,
    name: 'Баранов Владимир Николаевич',
    email: 'baranov@mail.ru',
    city: 'Новосибирск',
    date_of_birth: ISODate('1975-10-01T00:00:00.000Z'),
    loyalty_points: 340,
    purchases: { name: 'Электроника', year: 2023, value: 60000 }
  },
  {
    _id: ObjectId('686e74d5db26fb6ae68bf209'),
    customerId: 1,
    name: 'Баранов Владимир Николаевич',
    email: 'baranov@mail.ru',
    city: 'Новосибирск',
    date_of_birth: ISODate('1975-10-01T00:00:00.000Z'),
    loyalty_points: 340,
    purchases: { name: 'Одежда и обувь', year: 2021, value: 13000 }
  },
```

Рис. 112

$group – один из самых мощных операторов в структуре агрегации. Его используют, когда нужно сгруппировать в один документ документы, имеющие общие характеристики. Несколько важных вещей, которые нужно знать об операторе *$group*:

- необходимо определить групповой ключ, который является общей чертой. Он может быть полем, группой полей или выражением;
- групповой ключ определяют, используя *_id*.

Например, можно получить уникальные города покупателей (рис. 113).

```
e_shop> db.customers.aggregate([{ $group: {"_id": "$city"}} ])
[
  { _id: 'Новосибирск' },
  { _id: 'Москва' },
  { _id: 'Санкт-Петербург' }
]
e_shop>
```

Рис. 113

Использование аккумуляторов в *$group* помогает получить хорошие результаты с минимальным количеством кода. Например, необходимо рассчитать общее количество бонусных баллов покупателей каждого города. В этом случае используют в *$group* агрегатор суммы (рис. 114).

```
e_shop> db.customers.aggregate([
... { $group: {"_id": "$city", "total_points":
... { $sum: "$loyalty_points"}}}
... ])
[
  { _id: 'Москва', total_points: 480 },
  { _id: 'Санкт-Петербург', total_points: 220 },
  { _id: 'Новосибирск', total_points: 340 }
]
e_shop>
```

Рис. 114

Можно использовать *$avg* для расчета среднего бонусного балла покупателей в каждом городе (рис. 115).

```
e_shop> db.customers.aggregate([
... { $group: {"_id": "$city", "avg_points":
... { $avg: "$loyalty_points"}}}
... ])
[
  { _id: 'Новосибирск', avg_points: 340 },
  { _id: 'Москва', avg_points: 160 },
  { _id: 'Санкт-Петербург', avg_points: 110 }
]
e_shop>
```

Рис. 115

Иногда приходится иметь дело с большими наборами данных, где количество документов исчисляется миллионами. В таких случаях можно поработать с частью этой коллекции с помощью оператора *$sample* (рис. 116).

Например, коллекция *customers* содержит шесть документов, но необходимо работать только с двумя. Обратите внимание на то, что при по-

вторных запусках запроса эти два документа каждый раз будут выбираться произвольно.

```
e_shop> db.customers.aggregate([{ $sample: { size: 2 }}])
[
  {
    _id: ObjectId('686e78cddb26fb6ae68bf20d'),
    customerId: 5,
    name: 'Петренко Анастасия Максимовна',
    email: 'petrenko@mail.ru',
    city: 'Москва',
    date_of_birth: ISODate('2000-08-14T00:00:00.000Z'),
    loyalty_points: 120,
    purchases: [
      { name: 'Путешествия', year: 2021, value: 150000 },
      { name: 'Одежда и обувь', year: 2023, value: 75000 },
      { name: 'Детские товары', year: 2024, value: 14500 },
      { name: 'Детские товары', year: 2025, value: 3500 }
    ]
  },
  {
    _id: ObjectId('686e7646db26fb6ae68bf20b'),
    customerId: 3,
    name: 'Курбанов Денис Игоревич',
    email: 'kurbanov@internet.ru',
    city: 'Санкт-Петербург',
    date_of_birth: ISODate('1966-07-21T00:00:00.000Z'),
    loyalty_points: 120,
    purchases: [
      { name: 'Билеты и отели', year: 2021, value: 100000 },
      { name: 'Дом и сад', year: 2024, value: 52000 },
      { name: 'Книги', year: 2023, value: 3500 }
    ]
  }
]
e_shop>
```

Рис. 116

Приведем еще несколько полезных примеров практической агрегации в MongoDB.

Пример 1. Определим уникальные названия категорий товаров интернет-магазина. Для этого используем (рис. 117):

- *$unwind* – для деконструкции покупок;
- *$group* – для группировки категорий покупок.

```
e_shop> db.customers.aggregate([
... { $unwind: "$purchases"},
... { $group: {"_id": "$purchases.name"}}
... ])
[
  { _id: 'Книги' },
  { _id: 'Детские товары' },
  { _id: 'Путешествия' },
  { _id: 'Билеты и отели' },
  { _id: 'Одежда и обувь' },
  { _id: 'Электроника' },
  { _id: 'Дом и сад' }
]
e_shop>
```

Рис. 117

Пример 2. Определим, в каком городе сделано больше всего покупок в категории "Электроника". Для этого используем (рис. 118):

- *$unwind* – для деконструкции покупок;
- *$match* – для фильтрации категории;
- *$group* – для группировки фамилий покупателей и *$sum* – для расчета общей суммы.

```
e_shop> db.customers.aggregate([
... { $unwind: "$purchases" },
... { $match: { "purchases.name": "Электроника"} },
... { $group: { "_id": "$city", "total":
... { $sum: "$purchases.value"}}}
... ])
[
  { _id: 'Москва', total: 169000 },
  { _id: 'Новосибирск', total: 60000 }
]
e_shop>
```

Рис. 118

Пример 3. Выполним ранжирование покупателей по общей стоимости их покупок. Следует рассчитать сумму покупок каждого человека, а затем ранжировать значение по убыванию. Для этого используем (рис. 119):

- *$unwind* – для деконструкции покупок;
- *$group* – для группировки по фамилии, имени, отчеству и *$sum* – для расчета общей стоимости покупок;
- *$sort* – чтобы упорядочить общие значения по убыванию.

```
e_shop> db.customers.aggregate([
... { $unwind: "$purchases"},
... { $group: { "_id": ["$name"],
... "total_value": { $sum: "$purchases.value" } } },
... { $sort: {"total_value": -1} }
... ])
[
  { _id: [ 'Романова Василиса Викторовна' ], total_value: 304000 },
  { _id: [ 'Баранов Владимир Николаевич' ], total_value: 266500 },
  { _id: [ 'Петренко Анастасия Максимовна' ], total_value: 243000 },
  { _id: [ 'Медведев Антон Александрович' ], total_value: 197000 },
  { _id: [ 'Курбанов Денис Игоревич' ], total_value: 155500 },
  { _id: [ 'Иванова Ольга Сергеевна' ], total_value: 145000 }
]
e_shop>
```

Рис. 119

Пример 4. Рассчитаем общее количество бонусных баллов и среднее количество бонусных баллов покупателей в каждом городе и ранжируем их по этому среднему количеству. Для этого используем (рис. 120):

- *$group*, *$sum* и *$avg* – для вычисления значений;
- *$sort* – для ранжирования среднего количества.

```
e_shop> db.customers.aggregate([
... { $group: {"_id": "$city", "total": {$sum: "$loyalty_points"},
... "average": {$avg: "$loyalty_points"}}},
... { $sort: {"average": -1}}
... ])
[
  { _id: 'Новосибирск', total: 340, average: 340 },
  { _id: 'Москва', total: 480, average: 160 },
  { _id: 'Санкт-Петербург', total: 220, average: 110 }
]
e_shop>
```

Рис. 120

Пример 5. Ранжируем 3 лучшие категории покупок по стоимости. Для этого используем (рис. 121):

- *$unwind* – для деконструкции массива категорий покупок;
- *$sort* – для сортировки покупок по убыванию значений;
- *$limit* – чтобы получить только 3 лучших значения.

```
e_shop> db.customers.aggregate([
... { $unwind: "$purchases" },
... { $sort: { "purchases.value": -1 }},
... { $limit: 3}
... ])
[
  {
    _id: ObjectId('686e78cddb26fb6ae68bf20d'),
    customerId: 5,
    name: 'Петренко Анастасия Максимовна',
    email: 'petrenko@mail.ru',
    city: 'Москва',
    date_of_birth: ISODate('2000-08-14T00:00:00.000Z'),
    loyalty_points: 120,
    purchases: { name: 'Путешествия', year: 2021, value: 150000 }
  },
  {
    _id: ObjectId('686e7a81db26fb6ae68bf20e'),
    customerId: 6,
    name: 'Романова Василиса Викторовна',
    email: 'romanova@inbox.ru',
    city: 'Санкт-Петербург',
    date_of_birth: ISODate('2003-10-01T00:00:00.000Z'),
    loyalty_points: 100,
    purchases: { name: 'Билеты и отели', year: 2024, value: 130000 }
  },
  {
    _id: ObjectId('686e74d5db26fb6ae68bf209'),
    customerId: 1,
    name: 'Баранов Владимир Николаевич',
    email: 'baranov@mail.ru',
    city: 'Новосибирск',
    date_of_birth: ISODate('1975-10-01T00:00:00.000Z'),
    loyalty_points: 340,
    purchases: { name: 'Билеты и отели', year: 2021, value: 130000 }
  }
]
e_shop>
```

Рис. 121

Пример 6. Определим, в каком году была сделана наибольшая сумма покупок в определенной категории. Для этого необходимо использовать *$unwind*, *$group*, *$sort* и *$limit* (рис. 122).

```
e_shop> db.customers.aggregate([
... { $unwind: "$purchases" },
... { $group: { "_id": "$purchases.year",
... "amount": {$sum: "$purchases.value"}} },
... { $sort: {"amount": -1} },
... { $limit: 1}
... ])
[ { _id: 2021, amount: 468000 } ]
e_shop>
```

Рис. 122

Пример 7. Определим количество покупателей в каждом городе, общая стоимость покупок которых превышает 180 тыс. рублей. Для этого используем (рис. 123):

- *$unwind* – для деконструкции массива покупок;
- *$group* по *_id* и *$sum* – чтобы рассчитать общую стоимость покупок каждого человека, а также аккумулятор *$first* – для извлечения названия города и сохранения его в поле *city_name*;
- *$match* – чтобы отфильтровать покупателей со стоимостью покупок более 180 тыс. рублей;
- *$group* – для названия города и *$sum* – чтобы подсчитать количество покупателей.

```
e_shop> db.customers.aggregate([
... { $unwind: "$purchases" },
... { $group: {"_id": "$_id", "city_name": {$first: "$city"},
... "purchase_value": { $sum: "$purchases.value"} } },
... { $match: { "purchase_value": { $gt: 180080} } },
... { $group: {"_id": "$city_name", "people_count": {$sum: 1}}}
... ])
[
  { _id: 'Новосибирск', people_count: 1 },
  { _id: 'Москва', people_count: 2 },
  { _id: 'Санкт-Петербург', people_count: 1 }
]
e_shop>
```

Рис. 123

В заключение приведем пример использования *$out* – заключительного этапа конвейера агрегации. Снова определим уникальные названия категорий товаров интернет-магазина, но теперь результаты запроса записываются в БД *test* и новую коллекцию с именем *new* (рис. 124).

```
e_shop> db.customers.aggregate([
... { $unwind: "$purchases"},
... { $group: {"_id": "$purchases.name"}},
... { $out: {db: "test", coll: "new"}}
... ])

e_shop> use test
switched to db test
test> show collections
actor
address
film
new
person
student
test> db.new.find()
[
  { _id: 'Билеты и отели' },
  { _id: 'Электроника' },
  { _id: 'Дом и сад' },
  { _id: 'Книги' },
  { _id: 'Детские товары' },
  { _id: 'Путешествия' },
  { _id: 'Одежда и обувь' }
]
test>
```

Рис. 124

Выполнение работы

В документной базе данных необходимо:

- создать четыре конвейера агрегации с использованием различных этапов, аккумуляторов и операторов конвейерных выражений агрегации;
- создать конвейер с сохранением результатов в новой коллекции.

Содержание отчета

1. Номер и название лабораторной работы.
2. Цели лабораторной работы.
3. Снимки экранных форм, показывающие порядок выполнения лабораторной работы.
4. Результаты лабораторной работы.

Лабораторная работа № 8
ИСПОЛЬЗОВАНИЕ РЕГУЛЯРНЫХ ВЫРАЖЕНИЙ В MONGODB

Цель работы:

– познакомиться с регулярными выражениями;

– научиться использовать регулярные выражения при составлении сложных поисковых запросов в коллекциях.

Теоретические сведения

MongoDB предоставляет функционал для поиска шаблона в строке во время запроса путем написания регулярного выражения. Регулярные выражения – формальный язык поиска и осуществления манипуляций с подстроками в тексте, основанный на использовании метасимволов.

Регулярные выражения в MongoDB используют Perl-синтаксис (Perl-совместимые регулярные выражения). При их составлении необходимо руководствоваться следующими правилами:

1. Любой символ обозначает себя самого, если это не метасимвол. Чтобы отменить действие метасимвола, следует поставить перед ним \.

2. Строка символов обозначает строку этих символов.

3. Множество возможных символов (класс) заключается в квадратные скобки [ ]. Это означает, что в данном месте может стоять один из указанных в скобках символов. Если первый символ в скобках ^, то ни один из указанных символов не может стоять в данном месте выражения. Внутри класса можно употреблять символ -, обозначающий диапазон символов. Например, a-z – одна из малых букв латинского алфавита, 0-9 – цифра и т. д.

4. Все символы, включая специальные, можно обозначить как \.

5. Альтернативные последовательности разделяются символом |. Но внутри квадратных скобок это обычный символ.

6. Внутри регулярного выражения можно указывать подшаблоны, заключая их в круглые скобки, и ссылаться на них как \номер. Первая скобка обозначается как \1.

Основные метасимволы в регулярных выражениях:

- . (точка) – любой одиночный символ, кроме перевода строки (\n) (например, a.c соответствует abc, aac, a9c и т. д.);
- ^ (карет) – начало строки (или начало текста в зависимости от флагов) (например, ^abc соответствует abc в начале строки);
- $ (доллар) – конец строки (или конец текста в зависимости от флагов) (например, abc$ соответствует abc в конце строки);
- [ ] (квадратные скобки) – любой символ из указанных в скобках (группа или класс символов):
  - [abc] соответствует a, b или c;
  - [a-z] – любая строчная буква;
- [^] (отрицание в скобках) – любой символ, кроме указанных (например, [^abc] соответствует любому символу, кроме a, b, c);
- | (вертикальная черта) – логическое ИЛИ (например, a|b соответствует a или b);
- () (круглые скобки) – группировка символов (например, (ab)+ соответствует ab, abab, ababab и т. д.).

Можно выделить группу так называемых модификаторов:

- * (звездочка) – 0 или более повторений предыдущего символа или группы (например, ab*c соответствует ac, abc, abbc и т. д.);
- + (плюс) – 1 или более повторений предыдущего символа или группы (например, ab+c соответствует abc, abbc, но не ac);
- ? (вопрос) – 0 или 1 повторение предыдущего символа или группы (делает его необязательным) (например, ab?c соответствует ac и abc);
- {n} – ровно *n* повторений предыдущего символа или группы (например, a{3} соответствует aaa);
- {n,} – *n* или более повторений (например, a{2,} соответствует aa, aaa, aaaa и т. д.);
- {n,m} – от *n* до *m* повторений (например, a{2,4} соответствует aa, aaa, aaaa и т. д.).

ВНИМАНИЕ! Во всех других случаях фигурные скобки считаются обычными символами

При составлении регулярных выражений можно использовать обратный слеш (\) – специальный символ для экранирования специальных символов. Если символ имеет особое значение в регулярных выражениях (. (точка), * , ? , + , | , { , [, (, ^ , $), то обратный слеш перед ним делает его обычным символом. Например, \. означает точку (а не любой символ, как . без экранирования), * означает символ * (а не повторение 0 или более раз).

Кроме того, обратный слеш используется для обозначения специальных последовательностей и в управляющих конструкциях:

- \d – любая цифра ([0-9]);
- \D – не цифра ([^0-9]);
- \w – буква, цифра или подчеркивание ([a-zA-Z0-9_]);
- \W – не \w;
- \s – пробельный символ (пробел, табуляция, \n и др.);
- \S – не \s;
- \b – граница слова;
- \n – перенос строки;
- \t – табуляция.

Все вышеперечисленные бэкслеш-символы при распознавании регулярного выражения воспринимаются как один символ. Чтобы обозначить последовательности, необходимо комбинировать бэкслэш-символы с модификаторами метасимволов, например:

- \d+ – одна или больше цифр (то же, что и [0-9]+);
- \w+ – слово (то же, что и [a-zA-Z0-9]+).

При поиске обратного слеша в тексте его экранируют двумя обратными слешами, потому что в строках \ тоже экранирующий символ.

Примеры использования метасимволов в регулярных выражениях:

- российский почтовый индекс (6 цифр): ^\d{6}$;
- электронная почта: ^[\w.-]+@[\w.-]+\.[a-zA-Z]{2,}$;
- дата в формате DD.MM.YYYY: ^\d{2}\.\d{2}\.\d{4}$.

В MongoDB сопоставление с образцом можно выполнять:

- используя оператор *$regex*;
- без оператора *$regex*.

Сопоставление с шаблоном с использованием оператора $regex

Оператор *$regex* – мощный инструмент, предоставляющий возможность использования регулярных выражений в строках запросов. Он полезен, когда точное значение поля неизвестно, что обеспечивает гибкий и эффективный поиск в коллекциях.

Синтаксис:

```
{ <поле>: { $regex: /шаблон/, $options: "<опции>" } }
{ <поле>: { $regex: 'шаблон', $options: "<опции>" } }
```

В MongoDB для использования доступны следующие опции:

- *i* – соответствует шаблону и строчных, и заглавных букв в строке;
- *m* – многострочная строка (включить ^ и $ в шаблон при поиске, т. е. искать именно ^ и $ внутри строки. Без данной опции эти метасимволы будут совпадать с началом или концом строки);
- *x* – игнорировать все пробельные символы в шаблоне *$regex*;
- *s* – разрешить символу точки (.) соответствовать всем символам.

Важные моменты, которые следует учитывать:

- нельзя использовать оператор *$regex* внутри оператора *$in*;
- если нужно добавить регулярное выражение в список условий запроса, разделенных запятыми, необходимо использовать оператор *$regex*;

- если нужно задать параметры *x* и *s*, то необходимо использовать выражение оператора *$regex* с *$options*;
- можно использовать оператор *$not* с выражением оператора *$regex*;
- если нужно использовать регулярные выражения, совместимые с Perl, но не поддерживаемые JavaScript, то необходим оператор *$regex*.

Рассмотрим конкретные примеры. Пусть имеется коллекция *employee*, содержащая шесть документов, в которых приведены сведения о сотрудниках в виде пар «поле-значение» (рис. 125).

```
test> db.employee.insertMany([
... { "name": "Юрий", "age": 32,
... "position": "бэкенд-разработчик" },
... { "name": "Алексей", "age": 25,
... "position": "фронтенд-разработчик" },
... { "name": "Татьяна", "age": 30,
... "position": "менеджер по персоналу"},
... { "name": "Ольга", "age": 32,
... "position": "аналитик данных" },
... { "name": "Владимир", "age": 30,
... "position": "старший программист"},
... { "name": "Алексей", "age": 42,
... "position": "Ведущий Программист" }
... ])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('6880e5926856c29e4a8bf208'),
    '1': ObjectId('6880e5926856c29e4a8bf209'),
    '2': ObjectId('6880e5926856c29e4a8bf20a'),
    '3': ObjectId('6880e5926856c29e4a8bf20b'),
    '4': ObjectId('6880e5926856c29e4a8bf20c'),
    '5': ObjectId('6880e5926856c29e4a8bf20d')
  }
}
test>
```

Рис. 125

С помощью оператора *$regex* создадим шаблон, который вернет только те документы, которые содержат строку "разработчик" (рис. 126).

```
test> db.employee.find({"position": {$regex: "разработчик"}})
[
  {
    _id: ObjectId('6880e5926856c29e4a8bf208'),
    name: 'Юрий',
    age: 32,
    position: 'бэкенд-разработчик'
  },
  {
    _id: ObjectId('6880e5926856c29e4a8bf209'),
    name: 'Алексей',
    age: 25,
    position: 'фронтенд-разработчик'
  }
]
test>
```

Рис. 126

Отобразим сведения о сотруднике, который является программистом, без учета регистра с помощью параметра *i* (рис. 127).

```
test> db.employee.find({"position": {$regex: "программист",
... $options: "i"}})
[
  {
    _id: ObjectId('6880e5926856c29e4a8bf20c'),
    name: 'Владимир',
    age: 30,
    position: 'старший программист'
  },
  {
    _id: ObjectId('6880e5926856c29e4a8bf20d'),
    name: 'Алексей',
    age: 42,
    position: 'Ведущий Программист'
  }
]
test>
```

Рис. 127

Отобразим информацию о сотрудниках, имена которых начинаются на букву А (рис. 128).

```
test> db.employee.find({ "name": {$regex:"^A"}})
[
  {
    _id: ObjectId('6880e5926856c29e4a8bf209'),
    name: 'Алексей',
    age: 25,
    position: 'фронтенд-разработчик'
  },
  {
    _id: ObjectId('6880e5926856c29e4a8bf20d'),
    name: 'Алексей',
    age: 42,
    position: 'Ведущий Программист'
  }
]
test>
```

Рис. 128

Отобразим сведения о сотрудниках, имена которых заканчиваются на букву р (рис. 129).

```
test> db.employee.find({ "name": {$regex:"р$"}})
[
  {
    _id: ObjectId('6880e5926856c29e4a8bf20c'),
    name: 'Владимир',
    age: 30,
    position: 'старший программист'
  }
]
test>
```

Рис. 129

Сопоставление с образцом без использования оператора $regex

В этом случае достаточно указать регулярное выражение в объекте регулярного выражения. Его использование позволяет применять регулярное выражение внутри оператора *$in*.

Синтаксис:

```
{ <поле>: /шаблон/<опции> },
```

где */шаблон/* – указание на критерии поиска;

<опции> – значение данных.

Отобразим сведения о сотрудниках, в названиях должностей которых есть сочетание букв «тик», с помощью объекта регулярного выражения (рис. 130).

```
test> db.employee.find({ "position": /тик/})
[
  {
    _id: ObjectId('6880e5926856c29e4a8bf20b'),
    name: 'Ольга',
    age: 32,
    position: 'аналитик данных'
  }
]
test>
```

Рис. 130

Возможны комбинированные запросы. Например, в демонстрационной БД *e_shop* в коллекции *customers* найти покупателей с именем, содержащим слог «ан», имеющих электронную почту в домене *internet.ru*. Поскольку регулярные выражения здесь добавляются в список условий запроса, разделенных запятыми, необходимо использовать оператор *$regex* (рис. 131).

```
e_shop> db.customers.find({
...   "name": { $regex: /ан/ },
...   "email": { $regex: /\@internet.ru$/i }
... })
[
  {
    _id: ObjectId('686e7646db26fb6ae68bf20b'),
    customerId: 3,
    name: 'Курбанов Денис Игоревич',
    email: 'kurbanov@internet.ru',
    city: 'Санкт-Петербург',
    date_of_birth: ISODate('1966-07-21T00:00:00.000Z'),
    loyalty_points: 120,
    purchases: [
      { name: 'Билеты и отели', year: 2021, value: 100000 },
      { name: 'Дом и сад', year: 2024, value: 52000 },
      { name: 'Книги', year: 2023, value: 3500 }
    ]
  },
  {
    _id: ObjectId('686e77e1db26fb6ae68bf20c'),
    customerId: 4,
    name: 'Медведев Антон Александрович',
    email: 'medvedev@internet.ru',
    city: 'Москва',
    date_of_birth: ISODate('1980-11-27T00:00:00.000Z'),
    loyalty_points: 280,
    purchases: [
      { name: 'Электроника', year: 2022, value: 105000 },
      { name: 'Одежда и обувь', year: 2023, value: 57000 },
      { name: 'Детские товары', year: 2025, value: 35000 }
    ]
  }
]
e_shop>
```

Рис. 131

Облегчить работу по построению и проверке регулярных выражений позволяют онлайн-инструменты. Одним из лучших является Regex 101 [4]. Он обеспечивает полную среду для работы с регулярными выражениями. С его помощью можно создавать, тестировать и отлаживать регулярные выражения в удобном интерфейсе (рис. 132).

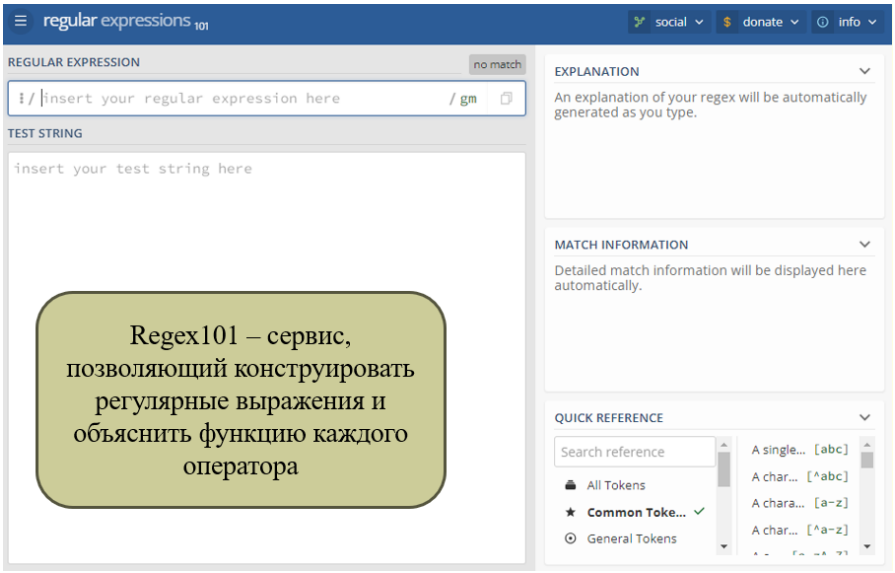


Рис. 132

Регулярное выражение задается непосредственно в поле и мгновенно видны результаты его работы. Тестовые данные вводятся прямо в инструменте, чтобы проверить правильность регулярного выражения.

Выполнение работы

Сформировать различные типы запросов с использованием регулярных выражений в документной базе данных:

- создать несколько запросов с использованием оператора *$regex*;
- создать несколько запросов без использования оператора *$regex*;
- создать несколько комбинированных запросов.

В запросах следует предусмотреть как можно более широкое использование возможностей регулярных выражений, рассмотренных в работе.

Содержание отчета

1. Номер и название лабораторной работы.
2. Цели лабораторной работы.
3. Снимки экранных форм, показывающие порядок выполнения лабораторной работы.
4. Результаты лабораторной работы.

Лабораторная работа № 9
РАБОТА С КУРСОРОМ

Цель работы:

– изучить принципы работы с курсором в MongoDB;

– освоить методы обработки результатов запросов и оптимизации производительности.

ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ

Курсор в MongoDB – это указатель на результат запроса. При выполнении запроса к коллекции с использованием таких методов, как *find(),* MongoDB возвращает курсор, который позволяет эффективно извлекать и обрабатывать документы из базы данных один за другим. Курсоры предоставляют различные методы для манипулирования и доступа к результатам запроса, что делает их необходимыми для работы с большими наборами данных и сложными запросами.

Особенности курсоров в MongoDB:

- эффективность – курсоры позволяют эффективно извлекать большие наборы данных без одновременной загрузки в память всех документов;
- автоматическая и ручная итерация – MongoDB может автоматически выполнять итерацию по курсору или позволить вручную управлять ей;
- ограничение по времени активности – по умолчанию MongoDB закрывает курсор после 10 минут бездействия, поэтому важно правильно управлять курсорами при длительных операциях.

Как следствие, курсор обеспечивает следующие возможности:

- «ленивая» загрузка – данные подгружаются по мере необходимости;
- итерация – возможность перебирать документы один за другим;
- пакетная обработка – MongoDB возвращает документы партиями.

Рассмотрим использование курсоров при работе с демонстрационной БД *e_shop*. Для отображения всех документов, имеющихся в коллекции покупателей, используется метод *find().* Этот метод возвращает курсор, содержащий все документы, имеющиеся в коллекции. MongoDB автомати-

чески выполняет итерацию по курсору для извлечения документов, но можно выполнить итерацию вручную.

Базовые операции с курсором

Простой запрос с использованием курсора представлен на рис. 133.

```
e_shop> var mycursor = db.customers.find({"customerId":3}).pretty()

e_shop> mycursor
[
  {
    _id: ObjectId('686e7646db26fb6ae68bf20b'),
    customerId: 3,
    name: 'Курбанов Денис Игоревич',
    email: 'kurbanov@internet.ru',
    city: 'Санкт-Петербург',
    date_of_birth: ISODate('1966-07-21T00:00:00.000Z'),
    loyalty_points: 120,
    purchases: [
      { name: 'Билеты и отели', year: 2021, value: 100000 },
      { name: 'Дом и сад', year: 2024, value: 52000 },
      { name: 'Книги', year: 2023, value: 3500 }
    ]
  }
]
e_shop>
```

Рис. 133

Другой пример простого запроса с использованием курсора представлен на рис. 134.

```
e_shop> const cursor = db.customers.find({"city": "Санкт-Петербург"})

e_shop> cursor
[
  {
    _id: ObjectId('686e7646db26fb6ae68bf20b'),
    customerId: 3,
    name: 'Курбанов Денис Игоревич',
    email: 'kurbanov@internet.ru',
    city: 'Санкт-Петербург',
    date_of_birth: ISODate('1966-07-21T00:00:00.000Z'),
    loyalty_points: 120,
    purchases: [
      { name: 'Билеты и отели', year: 2021, value: 100000 },
      { name: 'Дом и сад', year: 2024, value: 52000 },
      { name: 'Книги', year: 2023, value: 3500 }
    ]
  },
```

Рис. 134 (окончание см. на с. 114)

```
  {
    _id: ObjectId('686e7a81db26fb6ae68bf20e'),
    customerId: 6,
    name: 'Романова Василиса Викторовна',
    email: 'romanova@inbox.ru',
    city: 'Санкт-Петербург',
    date_of_birth: ISODate('2003-10-01T00:00:00.000Z'),
    loyalty_points: 100,
    purchases: [
      { name: 'Дом и сад', year: 2021, value: 25000 },
      { name: 'Дом и сад', year: 2022, value: 54000 },
      { name: 'Дом и сад', year: 2023, value: 35000 },
      { name: 'Билеты и отели', year: 2024, value: 130000 },
      { name: 'Путешествия', year: 2025, value: 60000 }
    ]
  }
]
e_shop>
```

Рис. 134. Окончание (начало см. на с. 113)

Если курсор не назначен с помощью ключевых слов *var* или *const*, то оболочка MongoDB автоматически перебирает в курсоре до 20 документов.

На рис. 135 представлен перебор результатов курсора в цикле.

```
e_shop> var mycursor = db.customers.find({"customerId": {$gt:4}})

e_shop> while(mycursor.hasNext()) {
... printjson(mycursor.next());
... }
{
  _id: ObjectId('686e78cddb26fb6ae68bf20d'),
  customerId: 5,
  name: 'Петренко Анастасия Максимовна',
  email: 'petrenko@mail.ru',
  city: 'Москва',
  date_of_birth: ISODate('2000-08-14T00:00:00.000Z'),
  loyalty_points: 120,
  purchases: [
    {
      name: 'Путешествия',
      year: 2021,
      value: 150000
    },
    {
      name: 'Одежда и обувь',
      year: 2023,
      value: 75000
    },
```

Рис. 135 (окончание см. на с. 115)

```
    {
      name: 'Детские товары',
      year: 2024,
      value: 14500
    },
    {
      name: 'Детские товары',
      year: 2025,
      value: 3500
    }
  ]
}
{
  _id: ObjectId('686e7a81db26fb6ae68bf20e'),
  customerId: 6,
  name: 'Романова Василиса Викторовна',
  email: 'romanova@inbox.ru',
  city: 'Санкт-Петербург',
  date_of_birth: ISODate('2003-10-01T00:00:00.000Z'),
  loyalty_points: 100,
  purchases: [
    {
      name: 'Дом и сад',
      year: 2021,
      value: 25000
    },
    {
      name: 'Дом и сад',
      year: 2022,
      value: 54000
    },
    {
      name: 'Дом и сад',
      year: 2023,
      value: 35000
    },
    {
      name: 'Билеты и отели',
      year: 2024,
      value: 130000
    },
    {
      name: 'Путешествия',
      year: 2025,
      value: 60000
    }
  ]
}
```

Рис. 135. Окончание (начало см. на с. 114)

Здесь мы вручную перебираем курсор, чтобы найти документ, *customerId* которого равен 3. Для этого мы присвоили курсор, возвращенный методом *find(),* переменной JavaScript *mycursor*. Метод курсора *next()* использован для доступа к следующему документу. Метод *printjson()* применяется для отображения результата. Мы взяли курсор, чтобы начать с *customerId* > 4, поэтому извлечены оставшиеся документы *customerId* 5 и 6.

Преобразуем курсор в массив: можно итерировать курсор и отображать результирующий документ в массиве с помощью метода *toArray()* (рис. 136).

```
e_shop> var mycursor = db.customers.find().pretty()

e_shop> var docs = mycursor.toArray()

e_shop> var resultdoc = docs[0]

e_shop> resultdoc
{
  _id: ObjectId('686e74d5db26fb6ae68bf209'),
  customerId: 1,
  name: 'Баранов Владимир Николаевич',
  email: 'baranov@mail.ru',
  city: 'Новосибирск',
  date_of_birth: ISODate('1975-10-01T00:00:00.000Z'),
  loyalty_points: 340,
  purchases: [
    { name: 'Электроника', year: 2023, value: 60000 },
    { name: 'Одежда и обувь', year: 2021, value: 13000 },
    { name: 'Одежда и обувь', year: 2022, value: 18000 },
    { name: 'Билеты и отели', year: 2021, value: 130000 },
    { name: 'Дом и сад', year: 2024, value: 40000 },
    { name: 'Книги', year: 2024, value: 5500 }
  ]
}
e_shop>
```

Рис. 136

Мы присваиваем возвращаемый курсор переменной *mycursor*, затем создаем массив из результирующего курсора с помощью метода *toArray()* и присваиваем результат переменной *docs*. Это обеспечивает доступ к документам в соответствии с их индексом. В данном случае отображаем документ, индекс которого равен 0.

Подсчет количества документов представлен на рис. 137.

```
e_shop> var count = db.customers.countDocuments({"city": "Москва"})

e_shop> print(count)
3

e_shop>
```

Рис. 137

Основные методы курсора

Рассмотрим основные методы курсора.

1. Сортировка.

Обычно проще проверять документы, если вывод отсортирован по возрастанию или убыванию. Для сортировки документов используют метод *sort()*. Чтобы отсортировать документы по возрастанию, устанавливают значение поля на 1, по убыванию – на –1 (рис. 138).

```
e_shop> const sortedCursor = db.customers.find().sort({"customerId": -1})

e_shop> sortedCursor
[
  {
    _id: ObjectId('686e7a81db26fb6ae68bf20e'),
    customerId: 6,
    name: 'Романова Василиса Викторовна',
    email: 'romanova@inbox.ru',
    city: 'Санкт-Петербург',
    date_of_birth: ISODate('2003-10-01T00:00:00.000Z'),
    loyalty_points: 100,
    purchases: [
      { name: 'Дом и сад', year: 2021, value: 25000 },
      { name: 'Дом и сад', year: 2022, value: 54000 },
      { name: 'Дом и сад', year: 2023, value: 35000 },
      { name: 'Билеты и отели', year: 2024, value: 130000 },
      { name: 'Путешествия', year: 2025, value: 60000 }
    ]
  },
  {
    _id: ObjectId('686e78cddb26fb6ae68bf20d'),
    customerId: 5,
    name: 'Петренко Анастасия Максимовна',
    email: 'petrenko@mail.ru',
    city: 'Москва',
    date_of_birth: ISODate('2000-08-14T00:00:00.000Z'),
    loyalty_points: 120,
    purchases: [
      { name: 'Путешествия', year: 2021, value: 150000 },
      { name: 'Одежда и обувь', year: 2023, value: 75000 },
      { name: 'Детские товары', year: 2024, value: 14500 },
      { name: 'Детские товары', year: 2025, value: 3500 }
    ]
  },
```

Рис. 138 (продолжение и окончание см. на с. 118 и 119)

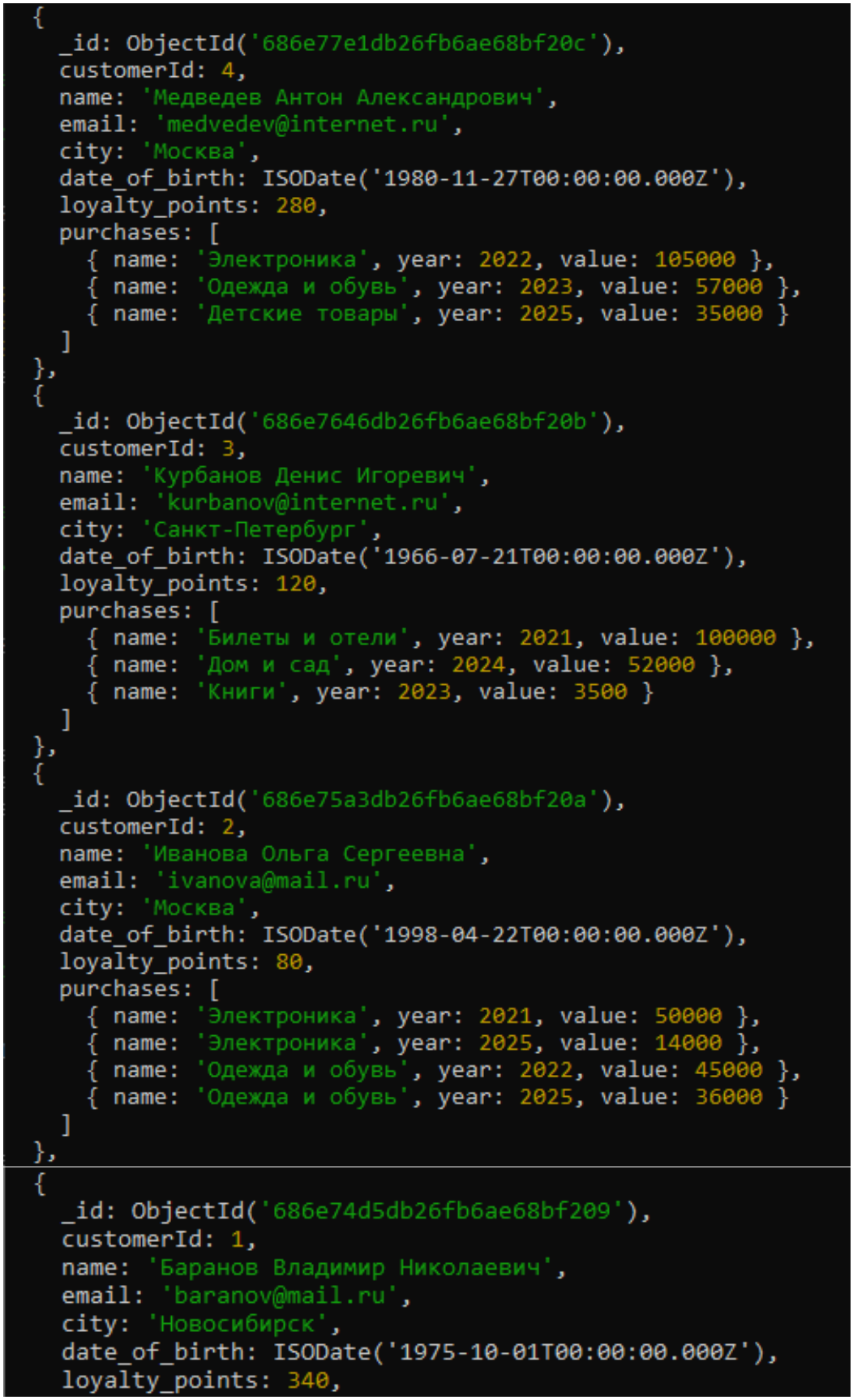


Рис. 138 (продолжение (начало см. на с. 117, окончание – на с. 119)

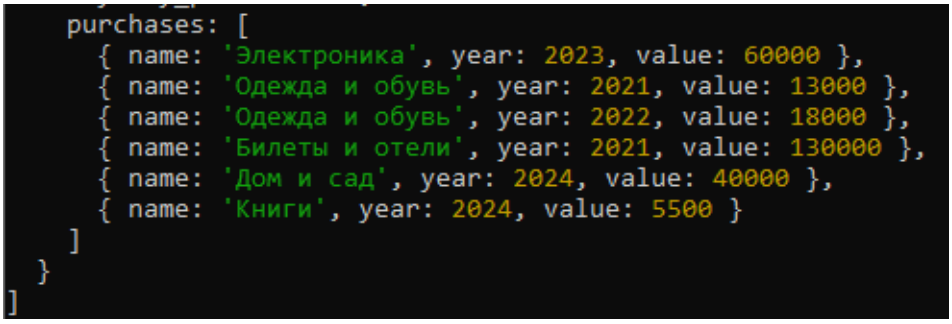


Рис. 138. Окончание (начало см. на с. 117 и 118)

2. Лимитирование.

Оно помогает извлекать из коллекции ограниченное число документов. Предположим, нам необходимо получить только два первых документа. Добиться этого позволяет метод *limit()* (рис. 139).



Рис. 139

3. Пропуск документов.

Во время отображения документов иногда требуется пропустить несколько документов по разным причинам. Метод *skip()* помогает отображать результаты после пропуска ряда документов (рис. 140).

```
e_shop> const skippedCursor = db.customers.find().skip(4)

e_shop> skippedCursor
[
  {
    _id: ObjectId('686e78cddb26fb6ae68bf20d'),
    customerId: 5,
    name: 'Петренко Анастасия Максимовна',
    email: 'petrenko@mail.ru',
    city: 'Москва',
    date_of_birth: ISODate('2000-08-14T00:00:00.000Z'),
    loyalty_points: 120,
    purchases: [
      { name: 'Путешествия', year: 2021, value: 150000 },
      { name: 'Одежда и обувь', year: 2023, value: 75000 },
      { name: 'Детские товары', year: 2024, value: 14500 },
      { name: 'Детские товары', year: 2025, value: 3500 }
    ]
  },
  {
    _id: ObjectId('686e7a81db26fb6ae68bf20e'),
    customerId: 6,
    name: 'Романова Василиса Викторовна',
    email: 'romanova@inbox.ru',
    city: 'Санкт-Петербург',
    date_of_birth: ISODate('2003-10-01T00:00:00.000Z'),
    loyalty_points: 100,
    purchases: [
      { name: 'Дом и сад', year: 2021, value: 25000 },
      { name: 'Дом и сад', year: 2022, value: 54000 },
      { name: 'Дом и сад', year: 2023, value: 35000 },
      { name: 'Билеты и отели', year: 2024, value: 130000 },
      { name: 'Путешествия', year: 2025, value: 60000 }
    ]
  }
]
e_shop>
```

Рис. 140

Возможна комбинация нескольких методов (рис. 141).

```
e_shop> const combCursor = db.customers.find().sort({"customerId":1}).limit(2).skip(1)

e_shop> combCursor
[
  {
    _id: ObjectId('686e75a3db26fb6ae68bf20a'),
    customerId: 2,
    name: 'Иванова Ольга Сергеевна',
    email: 'ivanova@mail.ru',
    city: 'Москва',
    date_of_birth: ISODate('1998-04-22T00:00:00.000Z'),
    loyalty_points: 80,
    purchases: [
      { name: 'Электроника', year: 2021, value: 50000 },
      { name: 'Электроника', year: 2025, value: 14000 },
      { name: 'Одежда и обувь', year: 2022, value: 45000 },
      { name: 'Одежда и обувь', year: 2025, value: 36000 }
    ]
  },
  {
    _id: ObjectId('686e7646db26fb6ae68bf20b'),
    customerId: 3,
    name: 'Курбанов Денис Игоревич',
    email: 'kurbanov@internet.ru',
    city: 'Санкт-Петербург',
    date_of_birth: ISODate('1966-07-21T00:00:00.000Z'),
    loyalty_points: 120,
    purchases: [
      { name: 'Билеты и отели', year: 2021, value: 100000 },
      { name: 'Дом и сад', year: 2024, value: 52000 },
      { name: 'Книги', year: 2023, value: 3500 }
    ]
  }
]
e_shop>
```

Рис. 141

Существуют и другие методы курсора, улучшающие функциональность запроса. Информацию о них можно найти в справочной литературе.

Оптимизация работы с курсором

Можно осуществить проекцию, т. е. выбор полей документа, которые должны отображаться (рис. 142).

```
e_shop> const cursor = db.customers.find({}, {"name":1, "date_of_birth":1, _id:0})

e_shop> cursor
[
  {
    name: 'Баранов Владимир Николаевич',
    date_of_birth: ISODate('1975-10-01T00:00:00.000Z')
  },
  {
    name: 'Иванова Ольга Сергеевна',
    date_of_birth: ISODate('1998-04-22T00:00:00.000Z')
  },
  {
    name: 'Курбанов Денис Игоревич',
    date_of_birth: ISODate('1966-07-21T00:00:00.000Z')
  },
  {
    name: 'Медведев Антон Александрович',
    date_of_birth: ISODate('1980-11-27T00:00:00.000Z')
  },
  {
    name: 'Петренко Анастасия Максимовна',
    date_of_birth: ISODate('2000-08-14T00:00:00.000Z')
  },
  {
    name: 'Романова Василиса Викторовна',
    date_of_birth: ISODate('2003-10-01T00:00:00.000Z')
  }
]
e_shop>
```

Рис. 142

Оптимизация запроса с помощью индекса представлена на рис. 143.

```
e_shop> db.customers.createIndex({"customerId": 1})
customerId_1
e_shop>
```

Рис. 143

При использовании метода *forEach()* можно применять функцию JavaScript к каждому документу в курсоре. Это простой способ обработки каждого документа без явного использования цикла (рис. 144).

```
e_shop> var mycursor = db.customers.find({"customerId": 2}).pretty()

e_shop> mycursor.forEach(printjson)
{
  _id: ObjectId('686e75a3db26fb6ae68bf20a'),
  customerId: 2,
  name: 'Иванова Ольга Сергеевна',
  email: 'ivanova@mail.ru',
  city: 'Москва',
  date_of_birth: ISODate('1998-04-22T00:00:00.000Z'),
  loyalty_points: 80,
  purchases: [
    {
      name: 'Электроника',
      year: 2021,
      value: 50000
    },
    {
      name: 'Электроника',
      year: 2025,
      value: 14000
    },
    {
      name: 'Одежда и обувь',
      year: 2022,
      value: 45000
    },
    {
      name: 'Одежда и обувь',
      year: 2025,
      value: 36000
    }
  ]
}
```

Рис. 144

Сначала сохраняем курсор, возвращаемый методом *find()* (т. е. покупатель с *customerId*: 2), в переменной *mycursor*. После этого используем метод *forEach()* для итерации курсора и отображения результирующего документа с помощью функции *printjson* (рис. 145).



Рис. 145 (окончание см. на с. 125)

```
    purchases: [
      { name: 'Дом и сад', year: 2021, value: 25000 },
      { name: 'Дом и сад', year: 2022, value: 54000 },
      { name: 'Дом и сад', year: 2023, value: 35000 },
      { name: 'Билеты и отели', year: 2024, value: 130000 },
      { name: 'Путешествия', year: 2025, value: 60000 }
    ]
  }
]
e_shop>
```

Рис. 145. Окончание (начало см. на с. 124)

Метод *next()* возвращает следующий документ в курсоре. Обычно возвращает первый документ (рис. 146).

```
e_shop> db.customers.find().next()
{
  _id: ObjectId('686e74d5db26fb6ae68bf209'),
  customerId: 1,
  name: 'Баранов Владимир Николаевич',
  email: 'baranov@mail.ru',
  city: 'Новосибирск',
  date_of_birth: ISODate('1975-10-01T00:00:00.000Z'),
  loyalty_points: 340,
  purchases: [
    { name: 'Электроника', year: 2023, value: 60000 },
    { name: 'Одежда и обувь', year: 2021, value: 13000 },
    { name: 'Одежда и обувь', year: 2022, value: 18000 },
    { name: 'Билеты и отели', year: 2021, value: 130000 },
    { name: 'Дом и сад', year: 2024, value: 40000 },
    { name: 'Книги', year: 2024, value: 5500 }
  ]
}
e_shop>
```

Рис. 146

Выполнение работы

В документной базе данных необходимо:

- выполнить итерацию по курсору для извлечения части документов;
- использовать рассмотренные способы итерации курсора и методы, улучшающие функциональность запроса.

Содержание отчета

1. Номер и название лабораторной работы.
2. Цели лабораторной работы.
3. Снимки экранных форм, показывающие порядок выполнения лабораторной работы.
4. Результаты лабораторной работы.

Лабораторная работа № 10
ИНТЕГРАЦИЯ MONGODB С ПРИЛОЖЕНИЯМИ

Цель работы: научиться подключать и использовать MongoDB в приложении, выполнять основные CRUD-операции и работать с данными в NoSQL-формате.

Теоретические сведения

MongoDB можно интегрировать с Node.js для создания масштабируемых и высокопроизводительных веб-приложений. Node.js – мощная кросс-платформенная среда выполнения JavaScript с открытым исходным кодом. Дает возможность запускать код JavaScript вне браузера [8].

Интеграция MongoDB с Node.js позволяет:

- работать с данными в формате JSON-подобных документов, что удобно для приложений Node.js, взаимодействующих с данными в этом формате;
- выполнять CRUD-операции с данными посредством API, предоставляемого драйвером MongoDB для Node.js, который следует установить.

Установка Node.js в Windows производится с помощью msi-инсталлятора. Он загружается с сайта *https://nodejs.org*. *Node.js* и все компоненты (пакетный менеджер *npm*, ссылка на документацию) в Windows устанавливают в папку *C:\Program Files\nodejs*. Путь к *node* и *npm* прописывается в переменную среды *PATH*.

Проверяем корректность установки (рис. 147).

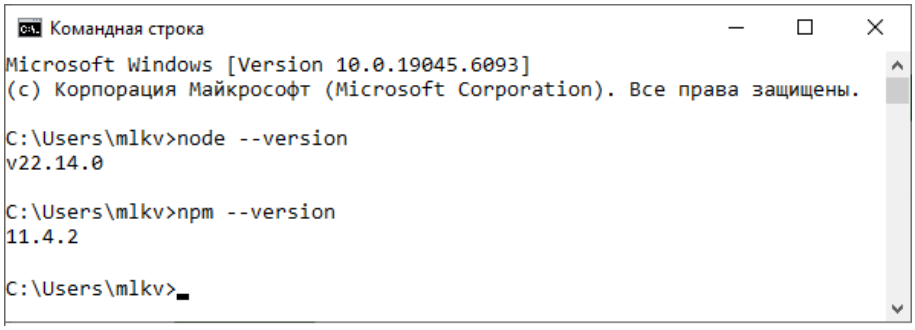


Рис. 147

Для работы с проектом создадим папку *C:\node\mongoapp*, в которой будем размещать разработанные приложения. Перейдем к этому каталогу в командной строке, установим пакет *mongodb* и проверим его версию (рис. 148).

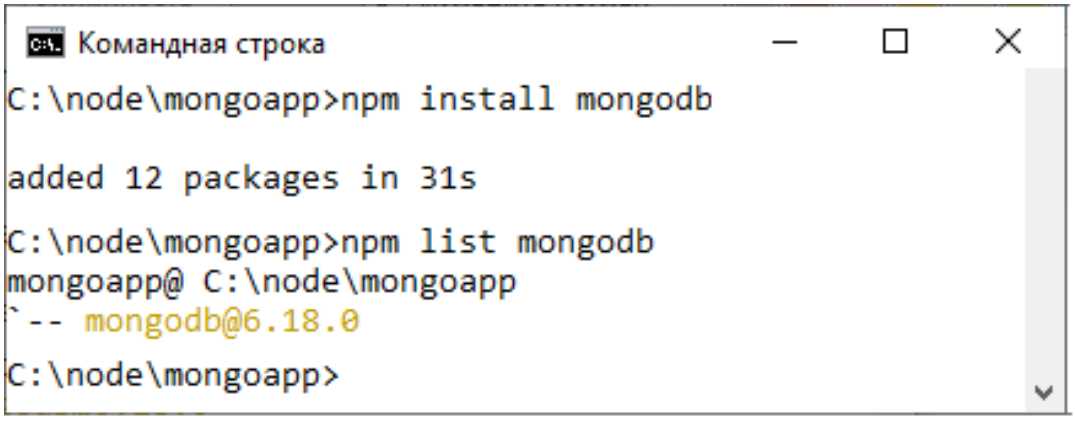


Рис. 148

ВНИМАНИЕ! Драйвер устанавливается в той же папке, в которой будут размещены файлы приложений, иначе возникнут проблемы с подключением

При работе с базой данных в MongoDB можно выделить следующие этапы:

1. Подключение к серверу.
2. Получение объекта БД на сервере.
3. Получение объекта коллекции в БД.
4. Взаимодействие с коллекцией (добавление, удаление, получение, изменение данных).

Подключение к базе данных

Перед подключением к MongoDB из приложения *Node.js* необходимо сначала запустить сам сервер *mongodb*.

Вначале нужно создать объект *MongoClient*, через который будут осуществляться все взаимодействия с БД. В конструктор *MongoClient* передается строка подключения к серверу. Элементы строки подключения:

- протокол, в качестве которого обычно применяется *"mongodb"*;
- имя пользователя и пароль через двоеточие (*"user: password"*) (если предусмотрена парольная аутентификация);
- хост и порт, по которому запущен сервер MongoDB (*host:port*);
- дополнительные параметры после вопросительного знака.

Стандартная строка подключения к локальному серверу MongoDB, запущенному на текущем компьютере:

```
"127.0.0.1:27017/"
```

В папке *mongoapp* создадим файл *connect.js*, который будет осуществлять подключение к базе данных, выводить имя текущей базы данных и закрывать подключение:

```
const MongoClient = require("mongodb").MongoClient;

const mongoClient = new MongoClient("mongodb://127.0.0.1:27017/");
async function run() {
    try {
        // Подключаемся к серверу
        await mongoClient.connect();
        console.log("Подключение установлено");
        // получаем имя базы данных
        console.log(mongoClient.options.dbName);
        // выполняем операции с базой данных
    }catch(err) {
        console.log(err);
    } finally {
        // Закрываем подключение при завершении работы или ошибке
        await mongoClient.close();
        console.log("Подключение закрыто");
    }
}
run().catch(console.log);
```

Запустим файл *connect.js* из командной строки (рис. 149).

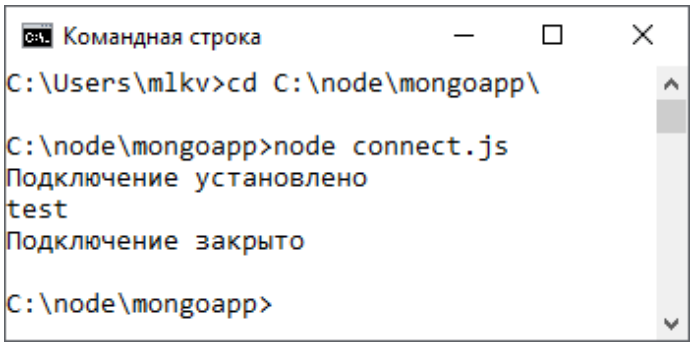


Рис. 149

Использована асинхронная функция JavaScript, которая позволяет выполнять другие задачи, пока ожидается завершение длительных операций, таких как запросы к серверу или чтение файлов. Это предотвращает блокировку основного потока выполнения. Конструкция *async/await* позволяет писать асинхронный код, который выглядит и читается как синхронный. Перед объявлением асинхронной функции ставится *async*. Внутри функции используется *await* для приостановки ее выполнения, пока не завершится промис (успешно или с ошибкой). Если функция возвращает значение, оно автоматически оборачивается в промис с этим значением.

Промис (*Promise*) в JavaScript – это объект, представляющий результат асинхронной операции: либо успешный, либо ошибочный. Это обещание, что код когда-нибудь выполнится и выдаст результат. Промис может находиться в одном из трех состояний:

- *pending* – ожидание (еще не завершился);
- *fulfilled* – успешно завершен;
- *rejected* – ошибка.

Промисы нужны для работы с асинхронными задачами – загрузкой данных с сервера, чтением файлов, чем-либо еще, что занимает время. Пока промис в ожидании, остальной код выполняется дальше и не зависает.

Для подключения к серверу MongoDB применяется метод *connect()*. Асинхронная функция *run()* приостанавливает выполнение, пока не завершится промис, возвращаемый функцией *connect()*. По завершении ре-

зультат передается в переменную, и выполнение продолжается. Для обработки ошибок в асинхронных функциях используется традиционная конструкция *try/catch/finally*. Блок *catch* перехватывает ошибку и позволяет ее обработать должным образом.

Свойство *mongoClient.options.dbName* позволяет получить название БД, к которой мы подключились (по умолчанию *test*). По завершении работы соединение закрывается с помощью метода *close()*. Этот метод также возвращает промис. Необходимая информация отображается в консоли с помощью метода *concole.log()*.

Работа с коллекциями

Для взаимодействия с БД (операций CRUD) нам потребуется получить объект коллекции. Для этого применяется метод *db.collection("коллекция")*, в который передается название коллекции и который возвращает объект класса *Collection*, позволяющий взаимодействовать с этой коллекцией.

Обратимся к коллекции *employee* в БД *test*. Для этого создадим в папке *mongoapp* файл *collection.js*:

```
const MongoClient = require("mongodb").MongoClient;

const url = "mongodb://127.0.0.1:27017/";
const mongoClient = new MongoClient(url);

async function run() {
    try {
        await mongoClient.connect();
        const db = mongoClient.db("test");
        const collection = db.collection("employee");
        const count = await collection.countDocuments();
        console.log(`В коллекции users ${count} документов`);
    }catch(err) {
        console.log(err);
    } finally {
        await mongoClient.close();
    }
}
run().catch(console.error);
```

Если на сервере MongoDB нет такой БД, то при первом обращении к ней сервер ее автоматически создаст. После подключения мы обращаемся к коллекции *employee*. Если такой коллекции нет в БД *test*, она также будет создана. После получения коллекции можно использовать ее методы. Мы применили метод *countDocuments()*. Он возвращает промис, из которого можно получить собственно результат операции – количество документов коллекции.

Запустим файл *collection.js* из командной строки (рис. 150).

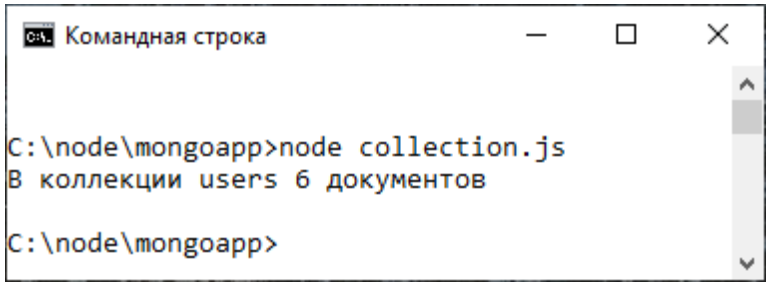


Рис. 150

Добавление документов в коллекцию

Добавить один документ в коллекцию позволяет метод *insertOne()*. В качестве параметра он принимает добавляемый документ. Создадим в папке *mongoapp* файл *insertone.js*:

```
const MongoClient = require("mongodb").MongoClient;

const url = "mongodb://127.0.0.1:27017/";
const mongoClient = new MongoClient(url);

async function run() {
    try {
        await mongoClient.connect();
        const db = mongoClient.db("test");
        const collection = db.collection("employee");
        const person = {"name": "Василий", "age": 19, "position": "лаборант"};
        const result = await collection.insertOne(person);
        console.log(result);
        console.log(person);
    }catch(err) {
        console.log(err);
    } finally {
        await mongoClient.close();
    }
}
run().catch(console.error);
```

Запустим файл *insertone.js* из командной строки (рис. 151).

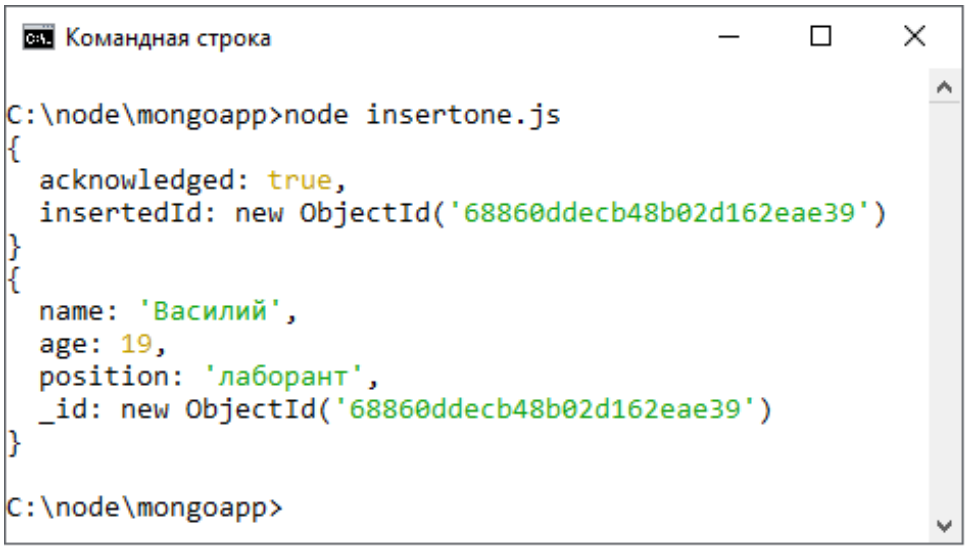


Рис. 151

С помощью свойства *result* можно увидеть результат операции, в частности, *insertedId* – идентификатор добавленного объекта.

Для добавления нескольких объектов используется метод *insertMany()*. Создадим в папке *mongoapp* файл *insertmany.js*:

```
const MongoClient = require("mongodb").MongoClient;

const url = "mongodb://127.0.0.1:27017/";
const mongoClient = new MongoClient(url);

const persons = [{"name": "Игорь", "age": 18, "position": "студент"},
                 {"name": "Ирина", "age": 18, "position": "студент"}];

async function run() {
    try {
        await mongoClient.connect();
        const db = mongoClient.db("test");
        const collection = db.collection("employee");
        const results = await collection.insertMany(persons);
        console.log(results);
        console.log(persons);
    }catch(err) {
        console.log(err);
    } finally {
        await mongoClient.close();
    }
}
run().catch(console.error);
```

Запустим файл *insertmany.js* из командной строки. Как и в случае с *insertOne()*, метод *insertMany()* в качестве параметра принимает добавляемые данные – массив объектов, а в качестве результата возвращает результат операции. При корректном добавлении объектов из массива *persons* они получают свойство – идентификаторы *_id*.

Объект результата в свойстве *insertedCount* содержит количество добавленных документов, а через свойство *insertedIds* можно получить идентификаторы добавленных документов (рис. 152).

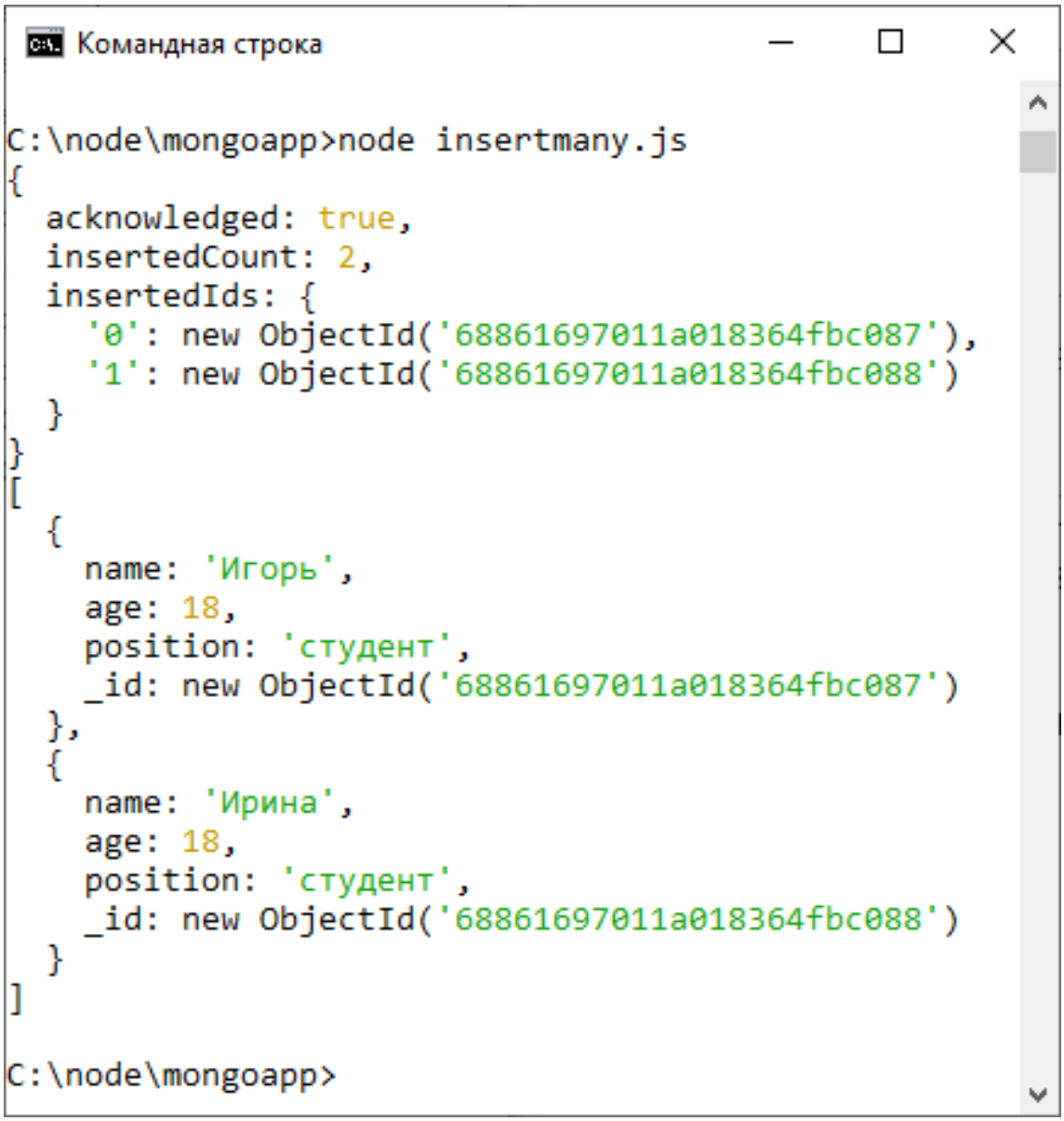


Рис. 152

Получение данных из коллекции

Для получения данных из коллекции применяется метод *find()*. Метод возвращает специальный объект *FindCursor*. Чтобы получить данные, у этого объекта вызывается метод *toArray()*. Создадим файл *find.js*:

```
const MongoClient = require("mongodb").MongoClient;

const url = "mongodb://127.0.0.1:27017/";
const mongoClient = new MongoClient(url);

async function run() {
    try {
        await mongoClient.connect();
        const db = mongoClient.db("test");
        const collection = db.collection("employee");
        const results = await collection.find().toArray();
        console.log(results);

    }catch(err) {
        console.log(err);
    } finally {
        await mongoClient.close();
    }
}
run().catch(console.error);
```

Запустим файл *find.js* из командной строки и увидим все документы коллекции *employee* (рис. 153).



Рис. 153

Фильтрация документов

С помощью метода *find()* можно дополнительно отфильтровать извлекаемые документы, например, найти сотрудников с именем Алексей:

```
const MongoClient = require("mongodb").MongoClient;

const url = "mongodb://127.0.0.1:27017/";
const mongoClient = new MongoClient(url);

async function run() {
    try {
        await mongoClient.connect();
        const db = mongoClient.db("test");
        const collection = db.collection("employee");
        const results = await collection.find({"name": "Алексей"}).toArray();
        console.log(results);

    }catch(err) {
        console.log(err);
    } finally {
        await mongoClient.close();
    }
}
run().catch(console.error);
```

Запустим файл *find_.js* из командной строки (рис. 154).

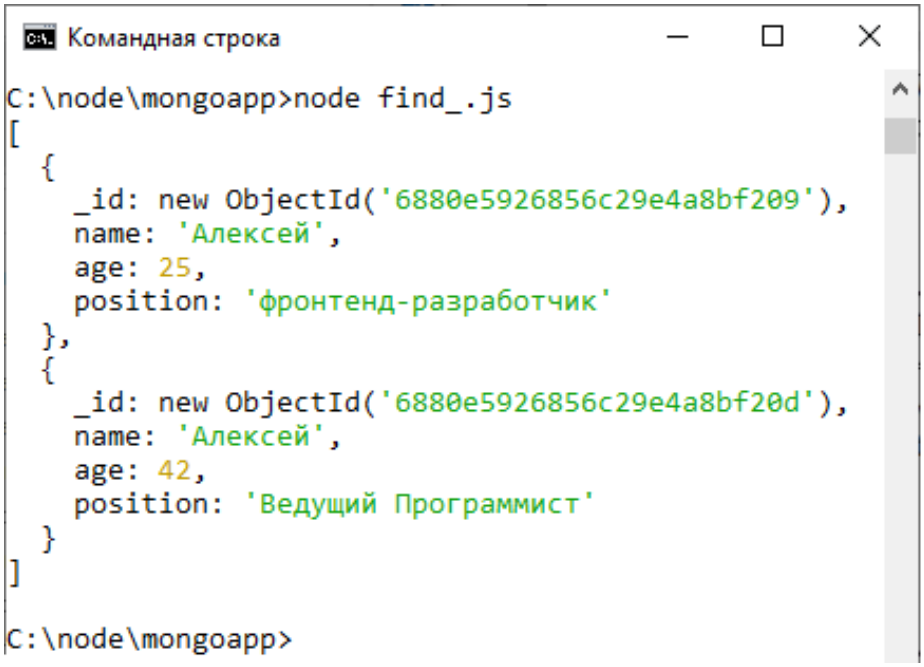


Рис. 154

Можно задать дополнительные критерии фильтрации, например фильтрацию по полю *age*. Метод *findOne()* работает аналогично, но позволяет получить только один документ.

Удаление документов

Удалять документы можно методами, рассмотренными в предыдущих лабораторных работах: *deleteOne(), deleteMany(), findOneAndDelete()*.

Метод *deleteOne()* удаляет только один объект. В качестве параметра метод принимает фильтр удаляемых документов (файл *deleteone.js*):

```
const MongoClient = require("mongodb").MongoClient;

const url = "mongodb://127.0.0.1:27017/";
const mongoClient = new MongoClient(url);

async function run() {
    try {
        await mongoClient.connect();
        const db = mongoClient.db("test");
        const collection = db.collection("employee");
        const result = await collection.deleteOne({"name": "Василий"});
        console.log(result);

    }catch(err) {
        console.log(err);
    } finally {
        await mongoClient.close();
    }
}
run().catch(console.error);
```

Запустим файл *deleteone.js* из командной строки (рис. 155).

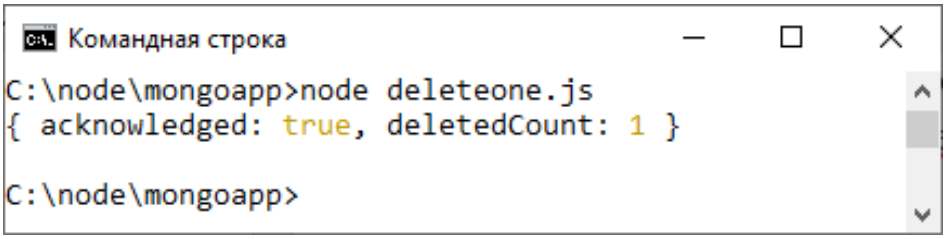


Рис. 155

Метод возвращает результат операции – объект, в котором свойство *deletedCount* позволяет увидеть число удаленных документов.

Метод *deleteMany()* работает аналогично, но удаляет все документы, соответствующие фильтру. Метод *findOneAndDelete()* удаляет один доку-

мент по критерию, но в отличие от метода *deleteOne()* позволяет получить удаленный документ (файл *findand.js*):

```
const MongoClient = require("mongodb").MongoClient;

const url = "mongodb://127.0.0.1:27017/";
const mongoClient = new MongoClient(url);

async function run() {
    try {
        await mongoClient.connect();
        const db = mongoClient.db("test");
        const collection = db.collection("employee");
        const result = await collection.findOneAndDelete({"age": 30});
        console.log(result);

    }catch(err) {
        console.log(err);
    } finally {
        await mongoClient.close();
    }
}
run().catch(console.error);
```

Запустим файл *findand.js* из командной строки (рис. 156).

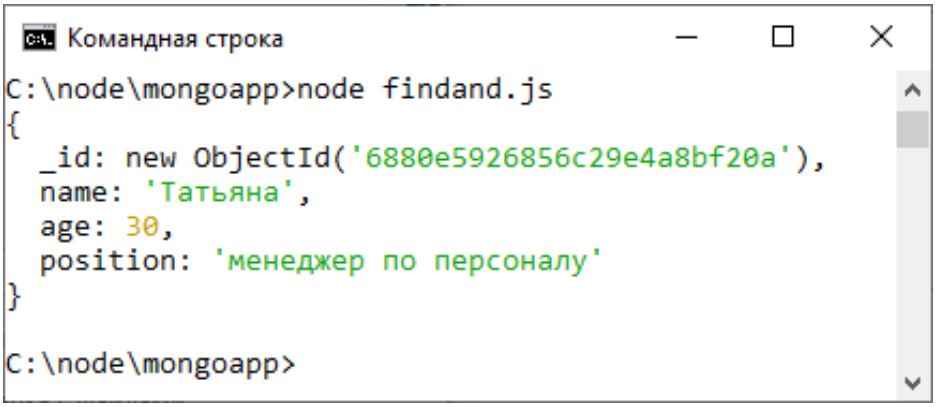


Рис. 156

Обновление документов

Обновлять документы можно методами, рассмотренными в предыдущих лабораторных работах: *updateOne(), updateMany (), findOneAndDelete().*

Метод *updateOne()* обновляет только один документ. В качестве параметра метод принимает критерий фильтрации (файл *updateone.js*). В отличие от метода *findOneAndUpdate()* он не возвращает измененный документ:

```
const MongoClient = require("mongodb").MongoClient;

const url = "mongodb://127.0.0.1:27017/";
const mongoClient = new MongoClient(url);

async function run() {
    try {
        await mongoClient.connect();
        const db = mongoClient.db("test");
        const collection = db.collection("employee");
        const result = await collection.updateOne({"name": "Игорь"},
              { $set: {"name": "Игорь", "age": 18, "position": "лаборант"}});
        console.log(result);

    }catch(err) {
        console.log(err);
    } finally {
        await mongoClient.close();
    }
}
run().catch(console.error);
```

Запустим файл *updateone.js* из командной строки (рис. 157).

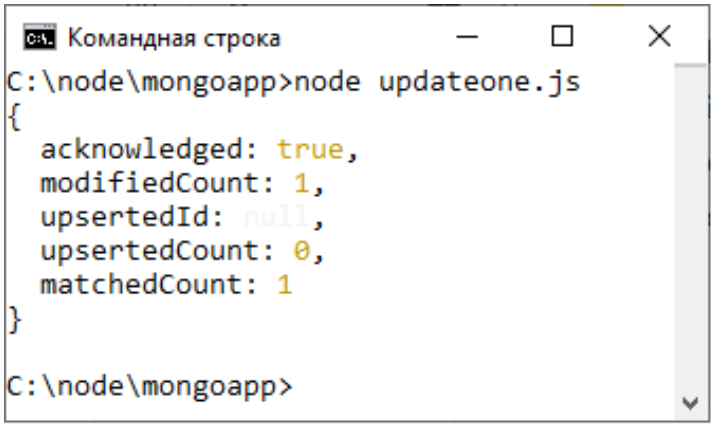


Рис. 157

Свойство *modifiedCount* показывает количество измененных документов. Свойство *upsertedId* содержит *id* добавленного документа (для операций замены с добавлением). Свойство *upsertedCount* содержит количество

добавленных строк. Свойство *matchedCount* указывает на количество документов, соответствующих параметру фильтрации.

Метод *updateMany()* аналогичен методу *updateOne()* за исключением того, что позволяет обновить все документы коллекции, соответствующие критерию фильтрации. Он также не возвращает измененные документы.

Метод *findOneAndUpdate()* обновляет один документ, соответствующий критерию фильтрации, и возвращает обновленный документ. Он принимает следующие параметры:

- критерий фильтрации для документа, который должен быть обновлен;
- параметр обновления;
- дополнительные опции обновления (по умолчанию имеют значение *null*);
- функция обратного вызова, которая выполняется при обновлении.

Например, обновим данные первого сотрудника, возраст которого 18 лет (файл *findandupd.js*):

```
const MongoClient = require("mongodb").MongoClient;

const url = "mongodb://127.0.0.1:27017/";
const mongoClient = new MongoClient(url);

async function run() {
    try {
        await mongoClient.connect();
        const db = mongoClient.db("test");
        const collection = db.collection("employee");
        const result = await collection.findOneAndUpdate({"age": 18},
                        { $set: {"age": 19}});
        console.log(result);

    }catch(err) {
        console.log(err);
    } finally {
        await mongoClient.close();
    }
}
run().catch(console.error);
```

Запустим файл *findandupd.js* из командной строки (рис. 158).

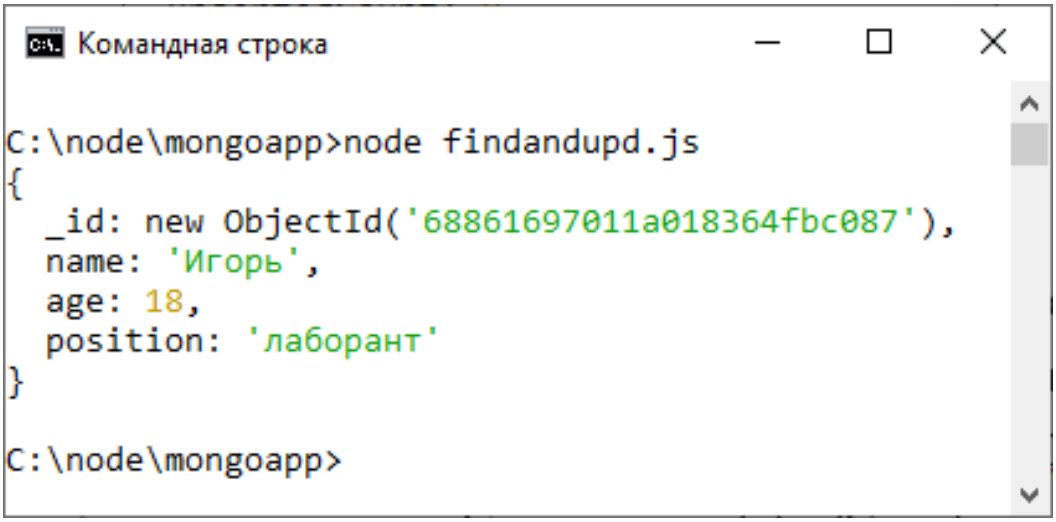


Рис. 158

Для обновления применен объект *{ $set: {"age": 19}}*. Параметр *$set* позволяет обновить значения одного поля или группы полей. В нашем случае изменяется поле *age*. Метод *findOneAndUpdate()* возвращает результат обновления – обновленный документ. По умолчанию возвращается старое состояние измененного документа, что мы и видим.

Если после обновления нужно получить не старое, а новое состояние измененного документа, то следует задать дополнительную опцию *returnDocument: "after"*, которая указывает, что надо получить новое состояние документа. По умолчанию этот параметр имеет значение *"before"*, т. е. возвращает состояние документа перед обновлением (файл *before.js*):

```
const MongoClient = require("mongodb").MongoClient;

const url = "mongodb://127.0.0.1:27017/";
const mongoClient = new MongoClient(url);

async function run() {
    try {
        await mongoClient.connect();
        const db = mongoClient.db("test");
        const collection = db.collection("employee");
        const result = await collection.findOneAndUpdate({"age": 18},
                    { $set: {"age": 19}}, { returnDocument: "after" });
        console.log(result);

    }catch(err) {
        console.log(err);
    } finally {
        await mongoClient.close();
    }
}
run().catch(console.error);
```

Запустим файл *before.js* из командной строки (рис. 159).

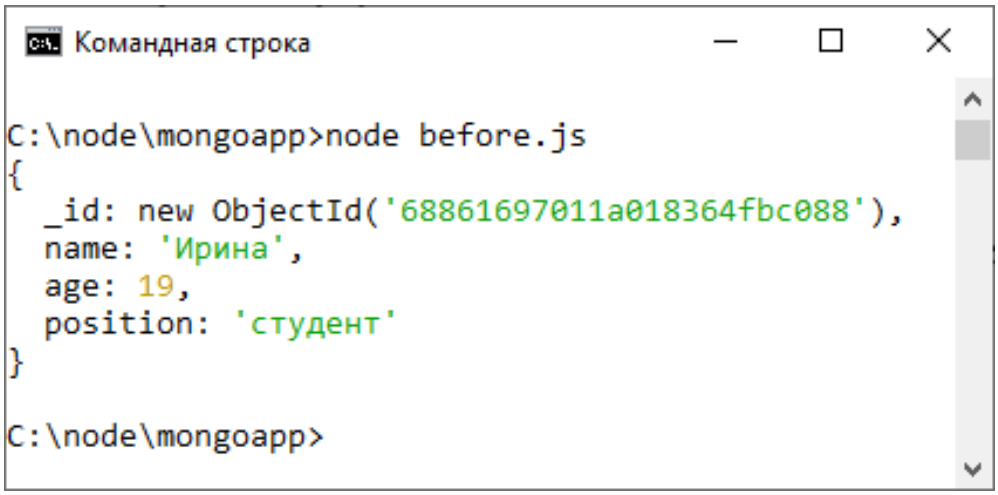


Рис. 159

Использование курсора

Выше мы рассматривали метод *find()*, возвращавший объект *FindCursor*. Получить все данные у этого объекта позволял метод *toArray()*. Можно непосредственно осуществлять итерацию по курсору.

Рассмотрим на примере коллекции *customers* базы данных *e_shop* (файл *cursor.js*) (рис. 160):

```
const MongoClient = require("mongodb").MongoClient;

const url = "mongodb://127.0.0.1:27017/";
const mongoClient = new MongoClient(url);

async function run() {
    try {
        await mongoClient.connect();
        const db = mongoClient.db("e_shop");
        const collection = db.collection("customers");
        const documents = collection.find().sort(["city", 1]).limit(3);
        // итерация по курсору
        while(await documents.hasNext()) {
            const document = await documents.next();
            console.log(document.name);
        }
    } catch (err) {
         console.log(err);
    } finally {
         await mongoClient.close();
    }
}
run().catch(console.error);
```

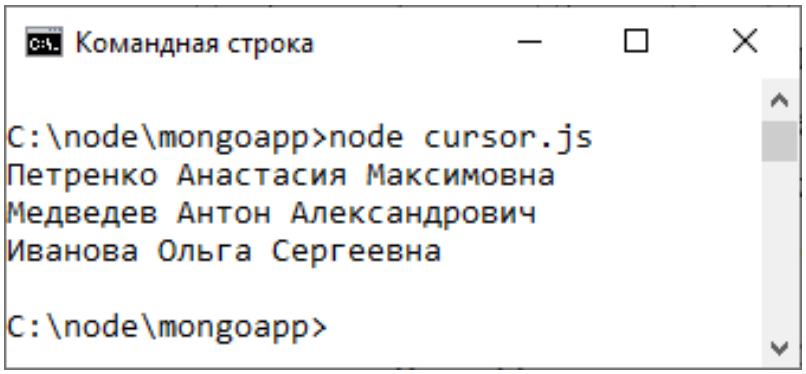


Рис. 160

К запросу *find()* добавлена сортировка *sort()* и предел *limit()*. Обратно получаем курсор *documents*, который перебираем в цикле *while()*. Для управления циклом используется результат метода *hasNext()* курсора. Это позволяет проверить, остались ли какие-либо документы в курсоре, назначить следующий документ и вывести его.

Агрегация с использованием Node.js

В *Node.js* можно использовать фреймворк агрегации. В коллекции *customers* базы данных *e_shop* подсчитаем общее количество бонусных баллов покупателей каждого города (файл *aggreg.js*) (рис. 161):

```
const MongoClient = require("mongodb").MongoClient;

const url = "mongodb://127.0.0.1:27017/";
const mongoClient = new MongoClient(url);

async function run() {
    try {
        await mongoClient.connect();
        const db = mongoClient.db("e_shop");
        const collection = db.collection("customers");
        const pipeline = [
            { $group: { "_id": "$city", "total_points":
                        { $sum: "$loyalty_points" } } },
            { $sort:  { "total_points": -1 } }
        ];
        const cursor = collection.aggregate(pipeline);
        // итерация по курсору
        while (await cursor.hasNext()) {
            const doc = await cursor.next();
            console.log(`${doc._id} - ${doc.total_points} points`);
        }
    } catch (err) {
         console.log(err);
    } finally {
        await mongoClient.close();
    }
}
run().catch(console.error);
```

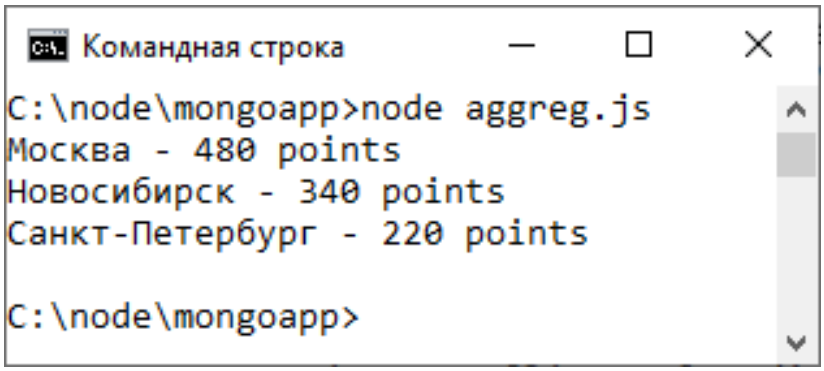


Рис. 161

Двухэтапный конвейер запускает *$group* по полю *city* и суммирует бонусные баллы покупателей. Суммарные бонусные баллы сортируем по убыванию. Конвейерный массив отправляется в метод *aggregate()*.

ВЫПОЛНЕНИЕ РАБОТЫ

Для документной базы данных необходимо:

- выполнить подключение к MongoDB с помощью *Node.js*;
- написать на языке JavaScript приложения, реализующие основные CRUD-операции, запустить и отладить их в среде *Node.js*;
- написать на языке JavaScript приложение, использующее курсор, запустить и отладить его в среде *Node.js*;
- написать на языке JavaScript приложение, реализующее конвейер с использованием фреймворка агрегации, запустить и отладить его в среде Node.js.

СОДЕРЖАНИЕ ОТЧЕТА

1. Номер и название лабораторной работы.
2. Цели лабораторной работы.
3. Снимки экранных форм, показывающие порядок выполнения лабораторной работы.
4. Результаты лабораторной работы.

ЗАКЛЮЧЕНИЕ

Настоящее пособие знакомит студентов с MongoDB – одной из наиболее популярных баз данных NoSQL. Она обладает уникальными качествами, отличающими ее от традиционных реляционных баз данных. Документная модель MongoDB революционизирует данные организации и их поиск, что выводит ее на передовые позиции в сфере информационных технологий.

В работе рассмотрена установка и первичная настройка MongoDB и представлены ее основные инструменты, такие как оболочка MongoDB Shell и графический клиент Compass. Приведены многочисленные и разнообразные примеры, демонстрирующие, как MongoDB позволяет эффективно организовать и хранить сложные структуры данных.

В рамках данного пособия рассмотрены CRUD-операции, основная задача которых – ввод, обновление и удаление данных. Показано использование фреймворка агрегации, регулярных выражений в запросах.

Подробно освещены вопросы интеграции MongoDB с приложением, написанном на языке JavaScript. Рассмотрены вопросы практического использования курсора, показаны примеры кода в Node.js, что позволит реализовать возможности MongoDB в создаваемых приложениях.

Используя полученные знания, далее можно уже самостоятельно изучать следующие темы:

- индексирование и управление коллекциями, способы повышения производительности запросов;
- более мощные и сложные методы импорта и экспорта данных;
- эффективная настройка и мониторинг сервера MongoDB;
- масштабирование данных, репликация и сегментирование;
- защита данных, включая миграцию данных, безопасность, стратегии резервного копирования и шифрование.

БИБЛИОГРАФИЧЕСКИЙ СПИСОК

1. Брэдшоу, Ш. MongoDB: полное руководство. Мощная и масштабируемая система управления базами данных / Ш. Брэдшоу, Й. Брэзил, К. Ходоров ; пер. с англ. – Москва : ДМК Пресс, 2020. – 540 с.

2. Николаев, Е. И. Основы NOSQL СУБД : учеб. пособие (лабораторный практикум) для студентов специальности 09.03.02 «Информационные системы и технологии» / Е. И. Николаев ; Северо-Кавказский федеральный ун-т. – Ставрополь : Изд-во СКФУ, 2016. – 130 с.

3. Редмонд, Э. Семь баз данных за семь недель. Введение в современные базы данных и идеологию NoSQL / Э. Редмонд, Дж. Р. Уилсон ; под ред. Ж. Картер ; пер. с англ. – Москва : ДМК Пресс, 2013. – 384 с.

4. Форта, Б. Изучаем регулярные выражения / Б. Форта ; пер. с англ. – Санкт-Петербург : Диалектика, 2019. – 192 с.

5. Giamas, A. Mastering MongoDB 6.x. Expert techniques to run high-volume and fault-tolerant database solutions using MongoDB 6.x / A. Giamas. – 3rd ed. – Birmingham – Mumbai : Packt Publishing, 2022. – 439 p.

6. Jenkins, J. MongoDB for Jobseekers / J. Jenkins. – London : BPB Online, 2023. – 399 p.

7. GeeksforGeeks. A comprehensive educational portal : site. – URL: https://www.geeksforgeeks.org (дата обращения: 04.08.2025).

8. METANIT.COM : сайт. – URL: https://metanit.com (дата обращения: 04.08.2025).

9. Prisma’s Data Guide : site. – URL: https://www.prisma.io/dataguide (дата обращения: 04.08.2025).